

Deen Associate Management System

Developed By

Muhammad Junaid Riaz

BS (CS) in Software Systems

Supervised By

Mr. Khawar Maqsood

Project Coordinator

Mr. Khawaja Tahir

Department of Computer Science

Federal Urdu University of Arts, Science & Technology, Islamabad

SESSION [2022-2026]

A Project Presented to

Federal Urdu University of Arts, Science & Technology

In Partial Fulfillment of the Requirement for the Degree

**Bachelor of Computer Science
(Software Systems)**

By

Muhammad Junaid Riaz

15th October, 2025

(Federal Urdu University of Arts, Science & Technology, Islamabad)

Declaration

I hereby declare that this software, neither as a whole nor as a part, has been copied from any source. I have developed this software and the accompanied report entirely on the basis of my personal efforts. If any part of this project is proved to be copied from any source, I will stand by the consequences. No portion of the work presented has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Junaid Riaz

Dedication

Dedicated to my beloved parents,

Teachers

And

The fellows

Acknowledgement

Thanks to Almighty Allah for giving me knowledge, power and strength to accomplish this task. I learned a lot while doing this project and this will certainly help me in my forthcoming life.

I am thankful to my supervisor Mr. Khawar Maqsood for his help and support in all phases of my project. His encouragement helped me during times of difficulty.

I would also like to thank the project coordinator and all of my teachers and friends for their support.

Project in Brief

Project Title: Deen Associate Management System

Organization: Federal Urdu University of Arts, Science & Technology, Islamabad

Client: Deen Associate

Objectives:

- Develop a digital platform to manage real-estate projects, units, bookings, and payments.
- Provide separate admin and client interfaces for transparent project and sales management.
- Automate installment schedules, employee management, and finance reporting.
- Replace manual record keeping with a centralized web application.

ABSTRACT

The Deen Associate Management System (DAMS) is a web-based real estate management platform designed for Deen Associate. It supports project publishing, unit inventory, customer booking requests, confirmed sales, installment tracking, payment receipts, employee attendance and salary management, and finance dashboards.

The system uses React for the frontend, .NET 8 Web API for the backend, and Microsoft SQL Server for data storage. JWT authentication and role-based access control protect admin and client operations.

By replacing manual spreadsheets and paper forms with a centralized application, DAMS improves transparency for customers and efficiency for administrators.

Revision History

Version 1.0 — Complete project documentation for Deen Associate Management System.

Author: Muhammad Junaid Riaz

Date: 15th October, 2025

Table of Contents

Chapter # 1 Introduction	11
Chapter # 2 Analysis	17
2.8 External Interface Requirements	36
Chapter # 3 Design Phase	40
3.1 Use Case Diagram	42
3.2 Fully Dressed Use Cases	46
3.3 Domain Model	90
3.4 Package Diagram	93
3.5 System Sequence Diagram	95
3.6 ER Diagram	115
3.7 Normalization	117
3.8 Relational Model	125
Chapter # 4 Construction	130
4.1 Class Diagram	132
4.2 Project Code	134
4.3 Sample Queries	147
Chapter # 5 Testing	159
Chapter # 6 User Manual	204

Chapter # 1

Introduction

Introduction

“Deen Associate Management System” is a complete digital platform designed to manage multiple real estate projects under the Deen Associate brand, including Urban Complex, Deen Villas, and Floria Heights. The purpose of the system is to streamline booking, client management, expense handling, and construction updates, ensuring transparency and efficiency between the admin and clients. The system provides a role-based access model, allowing both admins and clients to interact within a secure, scalable environment.

Problem Statement

As this project is being developed for an established real estate firm, the main goal is to provide an automated system that helps management handle all operations efficiently. Currently, **“Deen Associate”** manages bookings, client data, and financial records manually, which often leads to data inconsistency, delays, and inefficiency. Most real estate companies are now shifting toward digital management platforms to ensure transparency, accuracy, and real-time updates. However, **“Deen Associate”** still relies on traditional methods without a centralized database for projects, and progress tracking. While some competitors offer real estate management systems, many of them cover only basic functionalities like booking or client registration. Our proposed system will go beyond that by including modules for project management, client handling, expense management, and progress monitoring. The main objective is to automate daily operations, minimize manual errors, and enhance overall efficiency through a centralized and user-friendly platform.

Proposed System

“Deen Associate Management System” will provide a complete online solution for managing real estate operations efficiently. The project will include a web-based platform that automates daily business tasks such as project management, client registration, property booking, and progress monitoring. This system will make it easier for both management and clients to handle all activities through a single centralized portal. Clients will be able to view project details, book properties online, and track construction progress in real time. Admins will have access to tools for managing projects, updating progress reports, recording financial transactions, and generating analytical summaries. What sets this system apart from many other real estate management platforms is its focus on real-time updates, transparent financial tracking, and detailed project monitoring, which are often missing in traditional systems. Additionally, the system will feature modules for expense management and reporting, helping Deen Associate reduce manual workload, prevent data inconsistencies, and enhance operational efficiency. The main goal is to enable 24/7 online accessibility, allowing both management and clients to interact seamlessly and ensuring a smooth, transparent, and modern real estate experience.

Benefits of proposed system

1. Deen Associate Management System is used to manage projects, clients, and financial records online.
2. Simple and user-friendly dashboard for Admin and Clients.
3. Centralized platform for handling multiple projects and client data efficiently.

4. Real-time updates on construction progress and project status.
5. Secure JWT-based authentication with role-based access control for data protection.
6. Transparent financial tracking with automated reporting features.
7. Improved decision-making through analytical and visual dashboards.
8. Admins can monitor overall project performance, and client activities.
9. Reduced manual errors and increased operational efficiency.
10. Scalable for future modules like marketing, lead management, and expense tracking.

Primary Actors

- 1 Super Admin
- 2 Admin
- 3 client

Scope

The real estate industry is rapidly evolving with the growing adoption of digital management and property booking systems. Clients today prefer transparent, quick, and convenient ways to explore properties, make bookings, and track progress online. As technology advances, real estate companies are shifting from manual processes to automated systems that ensure accuracy, efficiency, and better client engagement. People now expect digital platforms that provide real-time updates, and easy access to project details without the need for physical visits. The trend is moving toward smart real estate management that emphasizes automation, data security, and transparency. Our target market includes both local and overseas clients who value professionalism, convenience, and reliability in property dealings. The Deen Associate Management System is designed to meet these modern needs by offering an all-in-one web-based solution to manage projects, client records, payments, expenses, and progress tracking. With its scalable structure, the system can be expanded to include marketing, staff, and lead management modules, helping Deen Associate stay competitive and aligned with the digital transformation in Pakistan's real estate sector.

Survey Analysis

Companies →	Zameen CRM (zameencrm.com)	Aiksol360 (aiksol360.com)	Dean Associate
System Features ↓			
Online Booking	✓	✓	✓
Unit Availability	✓	✓	✓
Project & Plaza Management	✓	✓	✓
Construction Progress	✗	✗	✓

Client Notification System	✗	✓	✓
Staff Management	✗	✓	✓
Expense Management	✓	✓	✓

Modules & Sub Modules

1. Security Management

- 1.1 Sign Up
- 1.2 Login
- 1.3 Change Password
- 1.4 Forget Password
- 1.5 Manage user role and profile

2. Staff Management

- 2.1 Add Staff
- 2.2 Edit Staff
- 2.3 View Staff
- 2.5 Assign Staff to Project

3. Project Management

- 3.1 Add Project
- 3.2 Edit Project
- 3.3 View Project
- 3.4 Search project

4. Client Management

- 4.1 Add Client
- 4.2 Edit Client
- 4.3 View Client
- 4.4 Search client

5. Booking Management

- 5.1 Add Booking
- 5.2 Edit Booking
- 5.3 View Booking
- 5.4 Cancel Booking

6. Construction Progress

- 6.1 Add Progress
- 6.2 Edit Progress
- 6.3 View Progress
- 6.4 Upload Media

7. Notification System

- 7.1 Send Notifications
- 7.2 View Notification

8. Expense Management

- 8.1 Add Expense
- 8.2 Edit Expense
- 8.3 View Expense
- 8.4 Delete Expense

9. System Interface

- 9.1 Dashboard Overview
- 9.2 Access All Modules

10. Income Management

- 10.1 Add Income Record
- 10.2 Edit Income Record
- 10.3 View Income record
- 10.4 Filter Income (by project , date wise , client wise)
- 10.5 Income summary

Tools & Technologies

1. Frontend Tools

Angular 16 /React

2. Backend Tools

. NET 8 (Web API)

3. Database

SQL Server / Firebase

4. Authentication

JWT (JSON Web Token)

5. Hosting & Deployment

Azure / Firebase / Local Server

6. Technique

Web Application

System Design Approach

➤ MVC Pattern

We will follow the N-tier design pattern to ensure better performance, maintainability, scalability and clear separation of concerns in our project.

➤ UML (Unified Model Language)

Process Model Used

➤ RUP (Rational Unified Process Model)

Limitations/Constraint

- Must have a good internet connection.
- For admin/employees, Login must be performed.
- Hosting setup and authentication configuration needed.
- “Dean Associate Management System” will be available only in Web Application.
- Future modules will be integrated gradually.

References

- <https://zameencrm.com>

- <https://aiksol360.com>

CHAPTER #02

Analysis

Software Requirement Specification

FOR

Deen Associate Management System

VERSION 1.0

Proposed By

Muhammad Junaaid Riaz

27th November, 2025

Revision History

Version	Description	Author	Date
1.0	This document contains SRS of Deen Associate Management System	Muhammad Junaid Riaz	27 th November 2025

Introduction

The real estate sector in Pakistan is rapidly transforming as businesses move from manual record-keeping to fully digital management systems. Clients today expect transparent, fast, and reliable ways to view project details, check property availability, book units, and track construction progress online. **Deen Associate**, which manages multiple projects such as Urban Complex, Deen Villas, and Floria Heights, currently handles most of its operations manually, including booking, client records, payment tracking, project progress updates, and staff coordination. Manual handling of such operations often leads to delays, errors, data loss, and reduced operational efficiency.

The **Deen Associate Management System (DAMS)** will be a complete web-based platform that automates all major real estate operations. The system will provide real-time project information, digital booking management, client handling, expense and income tracking, construction progress updates, and staff organization. By offering a centralized platform with role-based access, DAMS aims to improve transparency, reduce human errors, enhance user accessibility, and provide smooth, modern digital services to both clients and administrators.

Purpose

The purpose of this SRS is to define the detailed requirements, system behavior, and operational boundaries for the **Deen Associate Management System (DAMS)**. This document ensures that all stakeholders—including project owners, supervisors, developers, and evaluators—share a clear understanding of system goals and functionalities

The DAMS aims to:

- Provide a centralized platform for project, client, booking, and financial managementSimplify room reservations, check-in/check-out, and payment processing
- Digitalize unit booking, availability tracking, and construction progress updatesOffer secure access to users based on their roles
- Improve internal management through staff assignment and system-wide reportsOffer secure login and role-based access to Admin, Super Admin, and Clients
- Support scalability for future modules such as marketing or lead management

This SRS will serve as a reference throughout the development, implementation and testing phases

Scope

The Deen Associate Management System (DAMS) is designed for real estate companies that want to automate and centralize their business operations. This system will allow both administrators and clients to interact with the platform efficiently and securely. DAMS covers the entire workflow of Deen Associate, including project management, client handling, bookings, construction updates, staff management, expenses, and income.

The DBMS aims to:

- Provide a centralized online booking and management platform
- Provide online project and property management
- Allow clients and admins to manage bookings digitally
- Maintain client records with complete history
- Track construction progress with photos, videos, and stage-wise percentages
- Manage staff assignments and system roles
- Maintain transparent records of expenses and income
- Generate financial and operational reports
- Provide secure login using JWT authentication
- Reduce manual operations and shift Deen Associate toward complete digital automation

This SRS will serve as a reference throughout the development, implementation, and testing phases.

Definitions, Acronyms, and Abbreviations

Term	Definition
DAMS	DAMS Management System
GUI	Graphical User Interface
RUP	Rational Unified Process
UML	Unified Modeling Language
N-Tier Architecture	Presentation Layer, Business Logic Layer, and Data Access Layer

Overview

This document outlines the complete requirements of the Deen Associate Management System, including purpose, scope, actors, constraints, functional requirements, and non-functional requirements. DAMS is designed to automate project management, client handling, property bookings, construction updates, staff assignment, and financial tracking. The system will use modern technologies such as Angular/React, .NET Web API, SQL Server, and JWT authentication. The architecture will follow N-tier and the RUP process model to ensure scalability, maintainability, and high performance.

FUNCTIONAL REQUIREMENTS SPECIFICATION

Deen Associate Management System

1. SECURITY MANAGEMENT

1.1 Sign Up

SRS-1	The system shall allow new users to register by entering required details such as name, email, contact number, password, role, and company name.
SRS-2	The system shall validate all required fields before submission, including email format, password strength requirements, and valid phone number format.
SRS-3	The system shall prevent duplicate user registrations by checking unique identifiers such as email and phone number.
SRS-4	The system shall confirm successful registration with a success message and send a verification email to the user.
SRS-5	The system shall implement email verification before account activation.

1.2 Login

SRS-6	The system shall allow registered users to log in using their email and password.
SRS-7	The system shall validate credentials and allow access only if they are correct.
SRS-8	The system shall display an error message for incorrect login attempts.
SRS-9	The system shall track login timestamps and log all authentication attempts for security audit purposes.

1.3 Change Password

SRS-10	The system shall allow logged in users to change their password after providing the current password.
SRS-11	The system shall validate the current password before allowing password change.
SRS-12	The system shall validate the new password for strength requirements (minimum length, special characters, uppercase/lowercase).
SRS-13	The system shall update the password in the database and confirm the change with a success message.

1.4 Forget Password

SRS-14	The system shall allow users to reset their password by verifying their registered email address.
SRS-15	The system shall send a secure, time limited code reset link to the user's email (valid for 5 minute).
SRS-16	The system shall allow users to create a new password after matching user entered code (from email sendd by system)
SRS-17	The system shall confirm the password reset with a success message and send a notification email.

2. STAFF MANAGEMENT

2.1 Add Staff

SRS-32	The system shall allow administrators to add a new staff member by entering details such as name, email, contact number, designation, experience level, skills, and department.
SRS-33	The system shall validate all required fields before submission, including proper phone number format and valid email.
SRS-34	The system shall prevent duplicate staff entries by checking unique identifiers such as email or phone number.
SRS-35	The system shall automatically assign a unique staff ID upon successful addition.
SRS-36	The system shall confirm successful staff addition with a success message.

2.2 Edit Staff

SRS-37	The system shall allow administrators to update staff details including name, contact number, designation, skills, department, and status.
SRS-38	The system shall validate updated information before saving changes.
SRS-39	The system shall maintain a history of changes made to staff records for audit purposes.
SRS-40	The system shall confirm successful updates with a success message.

2.3 View Staff

SRS-41	The system shall display a list of all active staff members with details such as staff ID, name, designation, contact number, department, and employment status.
SRS-42	The system shall provide pagination options to navigate through large staff lists.
SRS-43	The system shall allow viewing individual staff profiles with complete details including work history and assigned projects.
SRS-44	The system shall display availability status of staff members.

2.4 Search Staff

SRS-45	The system shall allow users to search for staff using criteria such as name, staff ID,
---------------	---

	designation, department, or skills.
SRS-46	The system shall display search results in a clear list format with key staff details.
SRS-47	The system shall indicate if no matching staff members are found.
SRS-48	The system shall allow advanced search with multiple filter combinations.

2.5 Assign Staff to Project

SRS-49	The system shall allow project managers to assign staff members to specific projects.
SRS-50	The system shall display available staff based on skills required and current workload.
SRS-51	The system shall update staff assignment status and associate the staff member with the project.
SRS-52	The system shall prevent assigning staff to conflicting projects or exceed maximum workload limits.
SRS-53	The system shall confirm successful assignment with a success message.
SRS-54	The system shall allow administrators to view all staff-project assignments.

3. PROJECT MANAGEMENT

3.1 Add Project

SRS-55	The system shall allow authorized users to add a new project by entering details such as project name, description, location, client name, start date, end date, budget, and project status.
SRS-56	The system shall validate all required fields before submission, including logical date ranges (start date before end date).
SRS-57	The system shall prevent duplicate project entries by checking project name and location.
SRS-58	The system shall automatically assign a unique project ID upon successful addition.
SRS-59	The system shall confirm successful project addition with a success message.

3.2 Edit Project

SRS-60	The system shall allow authorized users to update project details including name, description, location, budget, timeline, and status.
SRS-61	The system shall validate updated information before saving changes.
SRS-62	The system shall maintain a complete history of all project modifications including timestamps and user who made changes.
SRS-63	The system shall confirm successful updates with a success message.

3.3 View Project

SRS-64	The system shall display a list of all projects with key details such as project ID,
---------------	--

	name, location, client, budget, start date, end date, and status.
SRS-65	The system shall provide filtering options by project status (Active, Completed, On-Hold, Cancelled).
SRS-66	The system shall allow viewing detailed project information including assigned staff, current progress, expenses, and timeline.
SRS-67	The system shall display project dashboard showing key metrics like budget utilization, timeline progress, and resource allocation.

3.4 Search Project

SRS-68	The system shall allow users to search for projects using criteria such as project name, project ID, client name, location, or status.
SRS-69	The system shall display search results in a clear list format with key project details.
SRS-70	The system shall indicate if no matching projects are found.
SRS-71	The system shall provide advanced search with date range filters and budget range filters.

4. CLIENT MANAGEMENT

4.1 Add Client

SRS-72	The system shall allow authorized users to add a new client by entering details such as client name, contact person name, email, phone number, address, company registration number, and payment terms.
SRS-73	The system shall validate all required fields before submission, including valid email format and phone number format.
SRS-74	The system shall prevent duplicate client entries by checking unique identifiers such as email, phone number, or company registration number.
SRS-75	The system shall automatically assign a unique client ID upon successful addition.
SRS-76	The system shall confirm successful client addition with a success message.

4.2 Edit Client

SRS-77	The system shall allow authorized users to update client details including contact person, email, phone number, address, and payment terms.
SRS-78	The system shall validate updated information before saving changes.
SRS-79	The system shall maintain a history of client information changes for audit purposes.
SRS-80	The system shall confirm successful updates with a success message.

4.3 View Client

SRS-81	The system shall display a list of all clients with details such as client ID, name, contact person, email, phone number, and address.
---------------	--

SRS-82	The system shall allow viewing individual client profiles including associated projects, booking history, and transaction records.
SRS-83	The system shall display client status (Active, Inactive, Blacklisted).
SRS-84	The system shall provide pagination options for large client lists.

4.4 Search Client

SRS-85	The system shall allow users to search for clients using criteria such as client name, client ID, email, phone number, or company name.
SRS-86	The system shall display search results in a clear list format with key client details.
SRS-87	The system shall indicate if no matching clients are found.
SRS-88	The system shall allow advanced search with multiple filter combinations.

5. BOOKING MANAGEMENT

5.1 Add Booking

SRS-89	The system shall allow users to create a new booking by selecting project, client, required resources, start date, end date, and booking purpose.
SRS-90	The system shall validate all required fields and ensure logical date ranges before submission.
SRS-91	The system shall check resource availability before confirming the booking.
SRS-92	The system shall automatically assign a unique booking ID and generate confirmation details.
SRS-93	The system shall confirm successful booking with a success message and send confirmation email to client.

5.2 View Booking

SRS-94	The system shall display all bookings with details including booking ID, project name, client name, resource type, start date, end date, and booking status.
SRS-95	The system shall provide pagination and sorting options for easier booking management.
SRS-96	The system shall allow viewing detailed booking information including resource allocation and special requirements.

5.3 Search Booking

SRS-97	The system shall allow users to search for bookings using criteria such as booking ID, project name, client name, date range, or status.
SRS-98	The system shall display search results in a clear list format with key booking details.
SRS-99	The system shall indicate if no matching bookings are found.

5.4 Cancel Booking

SRS-100	The system shall allow authorized users to cancel a booking within defined cancellation policies.
SRS-101	The system shall update the booking status to "Cancelled" immediately.
SRS-102	The system shall send cancellation notification email to the client with cancellation reason.
SRS-103	The system shall trigger refund processing if applicable based on cancellation policy.

5.5 Edit Booking

SRS-104	The system shall allow authorized users to modify booking details including dates, resources, and requirements.
SRS-105	The system shall validate all changes before saving and check resource availability.
SRS-106	The system shall maintain a history of booking modifications with timestamps and user details.
SRS-107	The system shall send booking modification notification to the client.

6. CONSTRUCTION PROGRESS

6.1 Add Progress

SRS-108	The system shall allow project managers to add progress updates by entering details such as project ID, progress percentage, status description, work completed, challenges faced, and planned next steps.
SRS-109	The system shall validate all required fields including progress percentage (0-100%) before submission.
SRS-110	The system shall timestamp each progress entry automatically with current date and time.
SRS-111	The system shall confirm successful progress addition with a success message.

6.2 Edit Progress

SRS-112	The system shall allow project managers to update progress details including percentage, description, and status.
SRS-113	The system shall validate updated information before saving changes.
SRS-114	The system shall maintain revision history of all progress updates with previous values and modification timestamps.
SRS-115	The system shall confirm successful updates with a success message.

6.3 View Progress

SRS-116	The system shall display construction progress for all active projects with progress
----------------	--

	percentage, current status, and completion timeline.
SRS-117	The system shall provide visual progress indicators (progress bars, charts) for better understanding.
SRS-118	The system shall allow filtering progress records by project, date range, or status.
SRS-119	The system shall display milestone tracking with planned vs actual completion dates.

6.4 Upload Media

SRS-120	The system shall allow users to upload progress-related media including images and videos to project records.
SRS-121	The system shall validate file types (JPG, PNG, MP4, MOV) and file size (maximum 50MB per file).
SRS-122	The system shall organize uploaded media by project and date for easy retrieval.
SRS-123	The system shall allow adding captions or descriptions to uploaded media.
SRS-124	The system shall generate automatic thumbnails for image preview.

7. NOTIFICATION SYSTEM

7.1 Send Notification

SRS-125	The system shall allow authorized users to send notifications to individual users or groups.
SRS-126	The system shall support notification types including project updates, booking confirmations, attendance alerts, and payment reminders.
SRS-127	The system shall deliver notifications via email, in-app messages, and SMS (if configured).
SRS-128	The system shall validate recipient information before sending notifications.
SRS-129	The system shall allow scheduling notifications for future delivery.

7.2 View Notifications

SRS-130	The system shall display all received notifications in a notification center with timestamps and read/unread status.
SRS-131	The system shall allow users to mark notifications as read or archive them.
SRS-132	The system shall provide filtering options by notification type, date, or sender.
SRS-133	The system shall maintain notification history for at least 90 days.

7.3 Manage Announcements

SRS-134	The system shall allow administrators to create and publish announcements for all users or specific user groups.
SRS-135	The system shall validate announcement content and allow formatting (bold, italic, lists).

SRS-136	The system shall allow scheduling announcements for automatic publishing at specified dates and times.
SRS-137	The system shall display announcements on dashboard and send as notifications to relevant users.
SRS-138	The system shall allow administrators to track announcement read receipts and engagement.

8. EXPENSE MANAGEMENT

8.1 Add Expense

SRS-139	The system shall allow authorized users to add expenses by entering details such as expense type (e.g., Materials, Labor, Equipment, Utilities, Transportation, Other), amount, date, related project, vendor/supplier, and description.
SRS-140	The system shall validate all required fields including positive amount and logical date (not in future).
SRS-141	The system shall allow attaching receipts or supporting documents to expense records.
SRS-142	The system shall automatically assign a unique expense ID upon successful addition.
SRS-143	The system shall confirm successful expense addition with a success message.

8.2 Edit Expense

SRS-144	The system shall allow authorized users to update expense details including type, amount, date, and description.
SRS-145	The system shall validate updated information before saving changes.
SRS-146	The system shall maintain a complete audit trail of expense modifications with timestamps and user information.
SRS-147	The system shall require additional approval if expense amount changes significantly (e.g., more than 20%).
SRS-148	The system shall confirm successful updates with a success message.

8.3 Delete Expense

SRS-149	The system shall allow administrators to delete expenses with proper authorization and approval workflow.
SRS-150	The system shall require confirmation before permanently deleting an expense record.
SRS-150	The system shall maintain soft delete records in archive for compliance and audit purposes.
SRS-152	The system shall send notification to project manager when an expense is deleted.

8.4 View Expenses

SRS-153	The system shall display all expenses with details such as expense ID, type, amount, date, project name, vendor, and status.
SRS-154	The system shall provide pagination and sorting options for easier expense management.
SRS-155	The system shall allow filtering expenses by expense type, date range, project, or amount range.
SRS-156	The system shall display expense category breakdowns in visual format (pie charts, bar graphs).

8.5 View Profit & Loss Report

SRS-157	The system shall allow users to generate profit and loss reports for selected date ranges and projects.
SRS-158	The system shall calculate total revenue from all projects and bookings automatically.
SRS-159	The system shall include all expenses (direct and indirect) in the P&L calculation.
SRS-160	The system shall display net profit/loss with detailed breakdown by category.
SRS-161	The system shall allow exporting P&L reports in PDF and Excel formats.
SRS-162	The system shall provide comparison between planned budget and actual expenses.

9. INCOME MANAGEMENT

10.1 Record Income

SRS-163	The system shall allow authorized users to record income by entering details such as income type (Project Billing, Service Charges, Deposits, Refunds), amount, date, related project/client, payment method, and description.
SRS-164	The system shall validate all required fields including positive amount and logical date (not in future).
SRS-165	The system shall link income records to specific projects and clients for tracking.
SRS-166	The system shall automatically assign a unique income ID upon successful addition.
SRS-167	The system shall confirm successful income recording with a success message.

10.2 Edit Income

SRS-168	The system shall allow authorized users to update income details including type, amount, date, and description.
SRS-169	The system shall validate updated information before saving changes.
SRS-170	The system shall maintain complete modification history with timestamps and user information.
SRS-171	The system shall confirm successful updates with a success message.

10.4 View Income Records

SRS-172	The system shall display all income records with details such as income ID, type, amount, date, project/client name, and payment method.
SRS-173	The system shall provide pagination and sorting options for easier management.
SRS-174	The system shall allow viewing detailed income information including payment verification status and reconciliation details.

10.5 Filter Income

SRS-175	The system shall allow filtering income records by income type, project, client, date range, payment method, or amount range.
SRS-176	The system shall support advanced filtering with multiple criteria combinations.
SRS-177	The system shall display filtered results with summary statistics (total amount, count of records).
SRS-178	The system shall allow saving frequently used filters for quick access.

10.6 Income Summary Report

SRS-179	The system shall generate comprehensive income summary reports for selected date ranges.
SRS-180	The system shall calculate total received amount, pending amount, and payment status breakdown.
SRS-181	The system shall display income breakdown by type, project, and client in visual format.
SRS-182	The system shall allow comparing projected vs actual income.
SRS-183	The system shall allow exporting income summary reports in PDF and Excel formats.

9. SYSTEM INTERFACE

9.1 Dashboard Overview

SRS-184	The system shall display a comprehensive dashboard showing key metrics such as total projects, active projects, total staff, upcoming bookings, and total revenue.
SRS-185	The system shall provide visual representations (charts, graphs, cards) of project status, budget utilization, and progress indicators.
SRS-186	The system shall display recent activities including new bookings, completed projects, and recent expenses.
SRS-187	The system shall allow customizing dashboard widgets based on user role and preferences.
SRS-188	The system shall refresh dashboard data in real-time or at user-defined intervals.

9.2 Access All Modules

SRS-189	The system shall provide a main navigation menu accessible from all pages allowing quick access to all modules.
SRS-190	The system shall highlight the current active module in the navigation menu.
SRS-191	The system shall display only modules and features that the user has permission to access based on their role.
SRS-192	The system shall support breadcrumb navigation for easy back-tracking.
SRS-193	The system shall provide search functionality to quickly navigate to specific modules or features.

9.3 Manage Users, Roles & Permissions

SRS-194	The system shall allow administrators to view all system users with details including name, email, role, status, and last login.
SRS-195	The system shall allow administrators to create, edit, and delete user accounts (with soft delete for audit trail).
SRS-196	The system shall allow administrators to assign and change user roles.
SRS-197	The system shall allow defining custom roles with granular permission control.
SRS-198	The system shall allow administrators to enable/disable users or reset user passwords.
SRS-199	The system shall maintain complete audit trail of user management actions including who made changes and when.

3. Non-Functional Requirements

3.1 Security

Since the system uses JWT authentication and role-based access, no unauthorized user can access the data. Users must log in with valid credentials, and the system ensures that only authorized roles can perform specific actions.

3.2 Usability

The system interface is simple, user-friendly, and easy to navigate for Admin, Super Admin, and Clients. All modules are clearly named, and forms include validation messages to guide users.

3.3 Availability

The system will remain available at all times when deployed online. It can be downloaded, hosted, or modified as needed. It will remain accessible to users as long as the server is running.

3.4 Performance

The system uses modern technologies like .NET and Angular/React, which provide fast performance. However, speed may vary depending on user device performance, internet connection, and server resources.

3.5 Accuracy

Since the system is fully digital, all calculations (income, expenses, progress) will be accurate as long as correct data is entered. There will be no manual errors related to paperwork.

4. External Interface Requirements

4.1 User Interfaces

The system provides clean and simple user interfaces with separate dashboards for administrators and clients. All modules such as Projects, Clients, Booking, Income, and Expenses are easily accessible from the main menu.

4.2 Hardware Interfaces

The system requires a standard computer or laptop with basic input devices like keyboard and mouse. A stable internet connection is required for accessing the online system.

4.3 Licensing Requirements

The system is free for internal use by Deen Associate and can be hosted on local or cloud servers. It can be modified according to future needs without licensing restrictions.

EXTERNAL INTERFACE REQUIREMENTS

FOR

Deen Associate Management System

VERSION 1.0

Proposed By

Muhammad Junaaid Riaz

27th November, 2025

Revision History

Version	Description	Author	Date
1.0	This document contains External Interface Requirements of Deen Associate Management System	Muhammad Junaid Riaz	27th November 2025

External Interface Requirements

This section describes the external interface requirements of the Deen Associate Management System (DAMS), including user interfaces, hardware interfaces, and licensing requirements.

User Interfaces

The system provides clean and simple user interfaces with separate dashboards for administrators, super administrators, and clients. All modules such as Projects, Clients, Booking, Income, and Expenses are easily accessible from the main menu.

User Interface (Client)

- Dashboard showing current bookings, project details, payment status, and notifications
- Project search and online booking request page
- Booking confirmation and booking history
- Payment receipts and pending dues alerts
- Profile management for updating personal information
- Notifications for booking, payment, or important notices
- Login, registration, and forgot password screens

Admin Interface

- Dashboard displaying new booking requests, confirmed bookings, customers, and finance summary
- Project and unit management (add/update projects, units, status, and media)
- Customer management panel
- Booking request review and confirmed booking management
- Installment and payment management
- Employee management, attendance, tasks, and salary panel
- Expense and manual income management

Super Admin Interface

- Full control over projects, units, staff, clients, bookings, payments, and finance
- Reporting and analytics for booking, payment, income, and expenses
- User role and profile management
- Role-based access management

Hardware Interfaces

The system requires a standard computer or laptop with basic input devices like keyboard and mouse. A stable internet connection is required for accessing the online system.

- Standard computer, laptop, or tablet with keyboard and mouse or touch input
- Minimum recommended configuration: Intel Core i5 or equivalent processor, 8 GB RAM, 256 GB SSD
- Stable internet connection for accessing the web-based system
- Supported web browsers such as Google Chrome, Microsoft Edge, or Mozilla Firefox
- Optional printer for printing booking application forms and payment receipts
- Local or cloud server for hosting the backend API and SQL Server database

Licensing Requirements

The Deen Associate Management System will comply with software licenses used in development. Open source or free tools will be used to ensure legality and cost-effectiveness.

Licensing Details

- Frontend: React.js MIT License (Open Source)
- Backend: .NET 8 Web API MIT License (Open Source)
- Database: Microsoft SQL Server (Commercial / Developer Edition)
- Authentication: JWT (JSON Web Token) (Open Standard)
- UI Styling: Tailwind CSS, Bootstrap Icons (Open Source)
- Development Tools: Visual Studio Code, Postman (Free to use)
- Hosting: Azure, local server, or compatible cloud hosting

Licensing Requirements Table

Software / Tool	Type	License Type	Usage	Remarks
React.js	Open Source	MIT License	Frontend Development	Free to use
.NET 8 Web API	Open Source	MIT License	Backend Development	Secure and scalable API
Microsoft SQL Server	Commercial / Developer	Microsoft Software License	Database Storage	Used for DAMS database
Tailwind CSS	Open Source	MIT License	UI Styling	For responsive design
Visual Studio Code	Open Source	MIT License	Code Editor	Free development environment
Postman	Freeware	Free	API Testing	Backend testing
Azure / Local Server	Cloud / Local Hosting	As per hosting plan	Deployment Environment	Production or development hosting

Design Phase

Design Phase

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

Muhammad Junaaid Riaz

27th November, 2025

USE CASE DIAGRAM

FOR

Deen Associate Management System

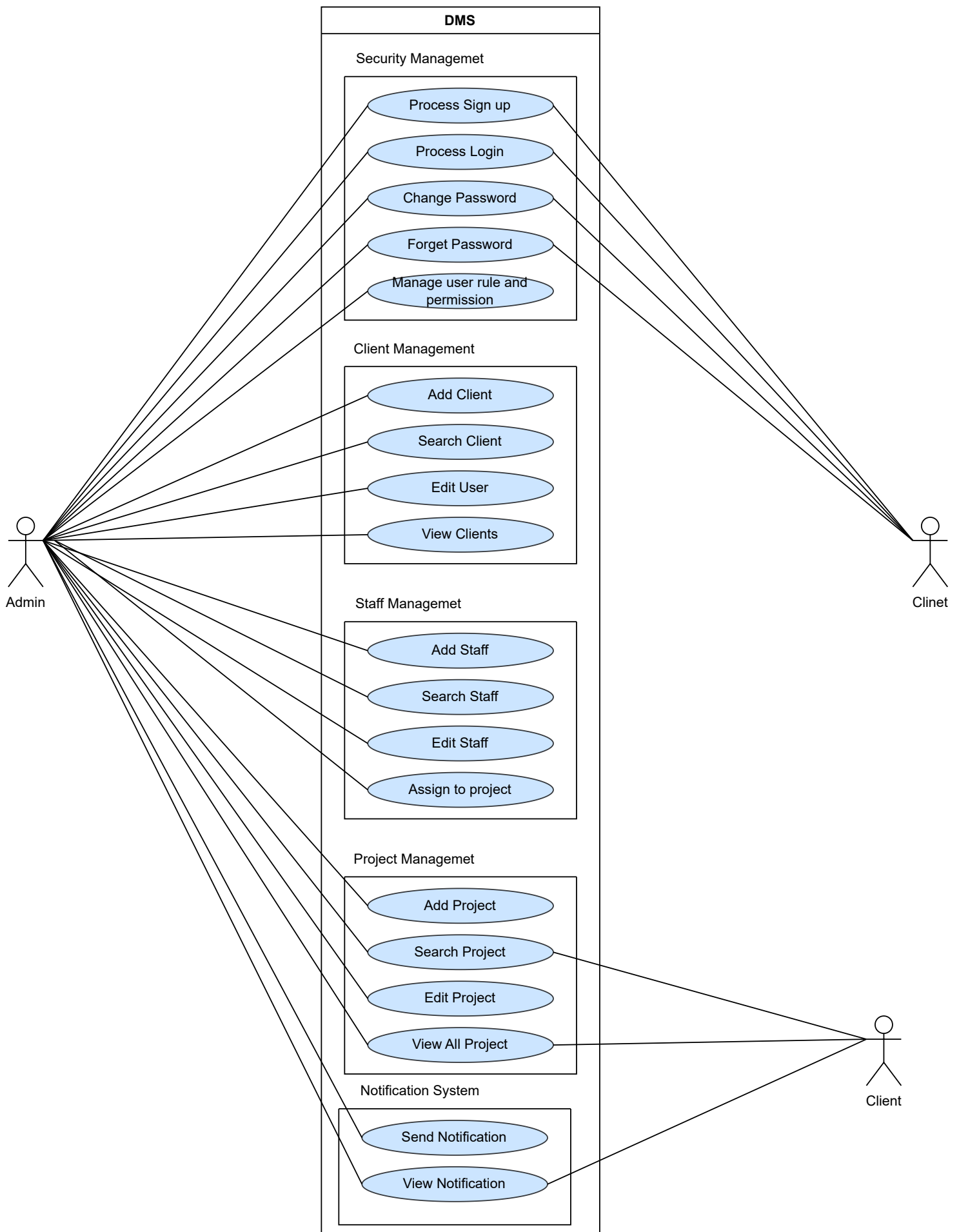
VERSION 1.0

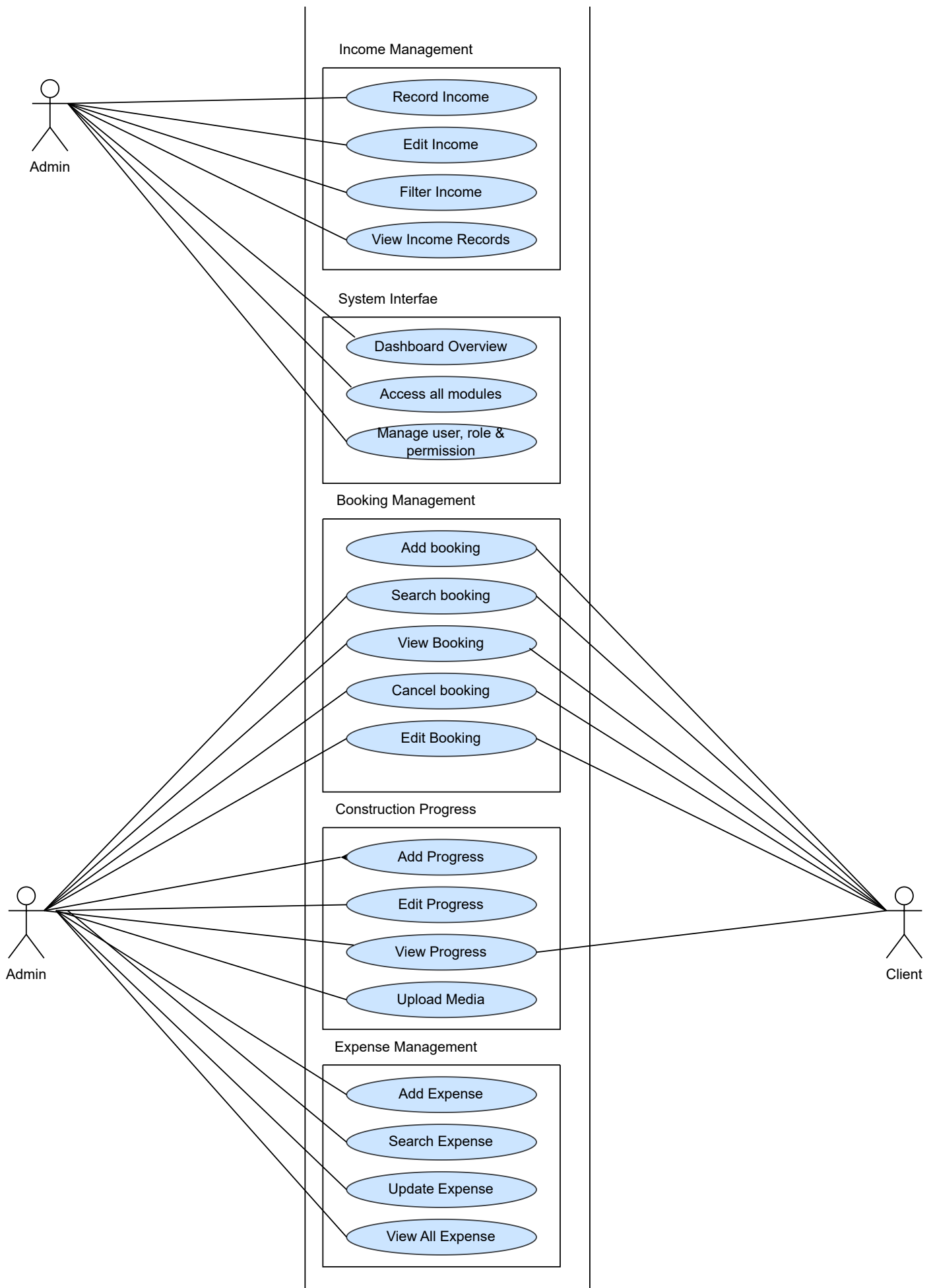
Prepared By

Muhammad Junaaid Riaz

27th November, 2025

3.1 Use Case Diagram





Fully Dressed Use Cases

FOR

Deen Associate Management System

VERSION 1.0

Proposed By

Muhammad Junaid Riaz

27th November, 2025

3.2 Fully Dressed Use Cases

Revision History

Version	Description	Author	Date
1.0	This document covers the fully dressed use cases of Deen Associates Management System	Muhammad Junaid	20 th November 2025

Use case ID: UC-01
Use case Name: Sign up

Use case Prepared by: M junaid Riaz

Use case Prepared on: 30-Nov-25

Use case Description:

This use case describes how a new user registers an account in the system.

Primary Actors: Admin , Clinet

Stake Holders s Interest: New User:Wants to create an account to access the system.

Pre-Condition:

1. System must be in running state
2. Use must not be already registered

Main Success Scenario:

User Action		System Response	
1	User enters required registration information (name, email, phone, password, role).		
		2	System validates all fields (email format, password strength, phone format).
3	User submits form.	4	System checks for duplicate email or phone number.
		5	System saves user information and generates verification email.
		6	System should send varification code to user email and display success message.

Post-Condition: User account is successfully created and verification email is sent.

Extension Points / Alternative way:

1a. Email or phone # already exists
System displays: “Email already registered.”

1b.Invalid email or weak password
System displays: “Enter valid email / strong password.”

1c. Any required field left blank
System will prompt to “Fill the blank fields”

Id. Email service down
System Dislays :“Varification email could not sent , Try again.”

Priority: High

Frequency: Less Frequently Use

Cross-Reference: SRS1, SRS2, SRS3, SRS4, SRS5

Use case ID: UC-02

Use case Name: login

Use case Prepared by: M junaid Riaz

Use case Prepared on: 30-Nov-25

Use case Description:

This use case describes how logged-in into the system.

Primary Actors: <Registered User>

Stake Holders s User:Wants to update password Securely

Interest:

Pre-Condition: 1.User must be logged in
2.System must be operational

Main Success Scenario:

User Action		System Response	
1	User enters email and password.		
		2	System validates credentials.
		3	System generates JWT token for authenticated session.
		4	System logs timestamp and redirects user to dashboard.

Post-Condition: User is successfully logged into the system and authenticated using JWT.

Extension Points / Alternative way: **1a.** Incorrect password:
System displays: “Invalid email or password.”
1b.Account not verified:
System displays: “Please verify your account first.”

Priority: High

Frequency: Very High

Cross-Reference: SRS6, SRS7, SRS8, SRS9

Use case ID: UC-03
caseName: Change Password

Use case Prepared by: M junaid Riaz

Use case Prepared on: 30-Nov-25

Use case Description:

This use case describes how registered user can changed their password.

Primary Actors: <Registered User>

Stake Holders s Interest: User:Wants to update password Securely

Pre-Condition:
1. System must be in running state
2. Use must must be login

Main Success Scenario:

User Action		System Response	
1	User enters current password and new password.		
		2	. System validates current password.
		3	System checks strength of new password.
4	User confirms password change.		
		5	System updates password in database and shows success message.

Post-Condition: Password is upadte successfully

Extension Points / Alternative way:
1a. Wrong current password:
System displays: “Incorrect current password”

1b.Weak New Password:
System displays: “Please Enter Strong password (with some suggestion from system)”

Priority: High

Frequency: Medium

Cross-Reference: SRS10, SRS11, SRS12, SRS13

Use case ID: UC-04
Use case Name: Forget Password

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 30-Nov-25

Use case Description:

This use case describes how to reset their password when they forget it.

Primary Actors: <Registered User>

Stakeholders Interest: User:Wants to recover access to their account securely

Pre-Condition: 1.User must have a registered email
2.System must be in running state.

Main Success Scenario:

User Action		System Response	
1	User clicks "Forget Password" and enters registered email.		
		2	System verifies that the email exists in the database.
		3	System generates a secure, time-limited reset code/link (valid for 5 minutes).
		4	System sends the reset code/link to the user's email.
5	User enters the received code or opens the reset link.		
		6	System verifies the code and allows user to set a new password.
7	User enters new password and submits		
		8	System validates new password strength and updates password.
		9	System sends confirmation email and displays success message.

Post-Condition: User has successfully reset their password and can now log in with the new password.

Extension Points / Alternative way: 1a. User enters email not registered:
System displays: "Email not found"
1b.Email server fails to send reset link:
System displays: "Unable to send email , Try again"

3c. User enters wrong reset code:
System displays: "Invalid or expired code"

Priority: High

Frequency: Medium

Cross-Reference: SRS14, SRS15, SRS16, SRS17

Use case ID: UC-05
Use case Name: Add staff

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:

This use case describes how an admin add new staff member into the system.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to add new staff with correct information

Pre-Condition:
1.System must be in running state
2.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	Admin enters staff details (name, email, phone, designation, skills, department).		
		2	System validates required fields and formats.
		3	System checks for duplicate staff using email/phone number.
		4	System assigns unique staff ID and saves staff data.
		5	System displays “Staff added successfully.”

Post-Condition: Staff member successfully added into system

Extension Points / Alternative way:
1a. Admin leaves any field blank :
System prompts “Fill the blank field.”
1b.Admin enters invalid email/phone:
System prompts “Enter valid data.”
3a. Duplicate entry found :
System prompts “Staff already exists.”

Priority: High

Frequency: Medium

Cross-Reference: SRS32, SRS33, SRS34, SRS35

Use case ID: UC-06
Use case Name: View Staff

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin can see the list of all staff member.

Primary Actors: <Admin>

Stakeholders Interest: Admin:Wants to view staff detail for management.

Pre-Condition:
1.System must be in running state
2.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	Admin navigates to “View Staff.”		
		2	System displays staff list with ID, name, designation, phone, and status.
		3	Admin selects a staff member to view full profile.
		4	System shows detailed staff profile.

Post-Condition: Admin successfully views staff detail.

Extension Points / Alternative way: **1a. No staff exists in the system:**
System prompts “No staff available”.

Priority: High

Frequency: Medium

Cross-Reference: SRS41, SRS42, SRS43, SRS44

Use case ID: UC-07
Use case Name: Edit staff

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin updates existing staff detail.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to view staff detail for management.

Pre-Condition:
1.System in running
2.Admin must be logged in.
3.Staff record must exist.

Main Success Scenario:

User Action		System Response	
1	Admin selects staff to edit.		
		2	System loads staff information.
3	Admin updates fields (contact, designation, skills, dept)		.
		4	System validates updated information.
		5	System saves updated data and shows success message

Post-Condition: Staff information is updated.

Extension Points / Alternative way:
1a. Staff not found:
System prompts “Staff not found”.
3a. Leaves blank:
System prompts “Fill empty black”.
3b. Invalid data format:
System prompts “Enter valid data”.

Priority: High

Frequency: Medium

Cross-Reference: SRS37, SRS38, SRS39, SRS40

Use case ID: UC-08
Use case Name: Search staff

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin search for an specific staff member.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to serach for spetic staff memeber.

Pre-Condition:
1.System in running
2.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	Admin enters search criteria (name, ID, designation, dept)		
		2	System verifies search input
		3	.System displays matching staff list

Post-Condition: Admin successfully view search result

Extension Points / Alternative way:
1a. No matching staff:
System prompts “No staff found”.

3a. Empty input:
System prompts “Fill empty black”.

3b. Invalid data format:
System “Enter search value”.

Priority: Medium

Frequency: High

Cross-Reference: SRS45, SRS46, SRS47, SRS48

Use case ID: UC-09
Use case Name: Assign Staff to project

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:

This use case describes how an admin assign staff to a specific project.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to assign skilled and available staff.

Pre-Condition:
1.System in running
2.Admin must be logged in.
3.Staff and system must exist.

Main Success Scenario:

User Action		System Response	
1	Manager selects project for assignment.		
		2	System displays available staff based on skills and workload.
3	Manager selects staff member and assigns to project.		.
		4	System verifies availability.
		5	System updates assignment and shows success message.

Post-Condition: Staff member successfully assigned to the selected project

Extension Points / Alternative way:
1a. No available staff:
System prompts "No staff available".
3a. staff already assigned:
System prompts "staff is already assigned".
3b. Leaves Blank:
System "Fill empty blank".

Priority: High

Frequency: Medium

Cross-Reference: SRS49, SRS40, SRS51, SRS52,SRS53,SRS54.

Use case ID: UC-10
Use case Name: Add Project

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin add new project in the system.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to create and register new project.

Pre-Condition:
1.System in running
2.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User enters project details (name, location, client, budget, start/end date, description).		
		2	System validates all required fields and checks date logic.
		3	.System checks for duplicate project name or location.
		4	System saves project and assigns a unique Project ID.
		5	System displays “Project added successfully.”

Post-Condition: A new project is added and stored in the system.

Extension Points / Alternative way:
1a. Invalid date (End<start):
System prompts “Enter valid date”.

1b. Leaves Blank:
System “Fill empty blank”.

Priority: High

Frequency: Medium

Cross-Reference: SRS55, SRS56, SRS57, SRS58,SRS59,

Use case ID: UC-11
Use case Name: Edit Project

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin update an existing project.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to updatre existing project..

Pre-Condition:
1.System in running
2.Admin must be logged in.
3.Project must be exists.

Main Success Scenario:

User Action		System Response	
1	User selects project to edit.		
		2	System loads all project details.
	User updates fields (budget, timeline, description, status, etc.).	3	.System checks for duplicate project name or location.
		4	System validates updated information.
		5	System saves updates and displays success message.

Post-Condition: Project data successfully updated.

Extension Points / Alternative way:
1b. **Project not found:**
System prompts "Project does not found".

1b. **Invalid date (End<start):**
System prompts "Enter valid date".

1c. **Leaves Blank:**
System "Fill empty blank".

Priority: High

Frequency: Medium

Cross-Reference: SRS60, SRS61, SRS62, SRS63

Use case ID: UC-12
Use case Name: View Project

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin see list or detail of Projects..

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to see the listing or detail of project.

Pre-Condition:
1.System in running
2.Admin must be logged in.
3.Project must be exists.

Main Success Scenario:

User Action		System Response	
1	User navigates to “View Projects.”		
		2	System displays list of all projects with ID, name, location, status, dates, budget
3	User selects a project to view in detail		..
		4	System shows full project details including staff assignments, expenses, timeline, and progress.

Post-Condition: Admin successfully see the projects information.

Extension Points / Alternative way: 1b. **No project exist:**
System prompts “No Project to found”.

Priority: Medium

Frequency: High

Cross-Reference: SRS64, SRS65, SRS66, SRS67

Use case ID: UC-13
Use case Name: Search Project

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:

This use case describes how an admin search for specific proejct.

Primary Actors: <Admin>

Stake Holders s Admin:Wants to search for projects.
Interest:

Pre-Condition: 3. System in running
4. Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User enters search criteria (project ID, name, client, location, status).		
		2	System validates search input
		3	System displays matching project list

Post-Condition: Admin successfully search and flitered projects.

Extension Points / 1b. No project matches Record:
Alternative way: System prompts “No Project found”.

Priority: Medium

Frequency: High

Cross-Reference: SRS68, SRS69, SRS70, SRS71

Use case ID: UC-14
Use case Name: Add client

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin can add the new client into the system.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants to store complete and accurate information of new client.

Pre-Condition:
5. System in running
6. Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User enters client details (name, contact person, phone, email, address, registration no., payment terms).		
		2	System validates all fields (email format, phone number).
		3	System checks for duplicate client by email, phone, or registration number
		4	System creates a unique Client ID and saves the client record
		5	System shows "Client added successfully."

Post-Condition: A new client is successfully registered in the system.

Extension Points / Alternative way: 1b. **Left blacks:**
System prompts "Fill the black field".

Priority: High

Frequency: Medium

Cross-Reference: SRS72, SRS73, SRS74, SRS75,SRS76

Use case ID: UC-15
Use case Name: Edit client

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

This use case describes how an admin can update existing client in the system.

Primary Actors: <Admin>

Stakeholders Interest: Admin:Wants to updates information of an existing client.

Pre-Condition:
1System in running
2Admin must be logged in.
3Client must be exists.

Main Success Scenario:

User Action		System Response	
1	User selects a client to edit.		
		2	System loads all existing client details
		3	User modifies fields (email, contact person, phone, address, payment terms)
		4	System validates updated data.
		5	System saves updated client details and shows success message

Post-Condition: Client information successfully updated.

Extension Points / Alternative way:
1a. **Left blacks:**
System prompts "Fill the black field".
1b. **Invalid data:**
System prompts "Enter valid data".

Priority: High

Frequency: Medium

Cross-Reference: SRS77, SRS78, SRS79, SRS80

Use case ID: UC-16
Use case Name: View client

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:

This use case describes how an admin view list of client or profile of client.

Primary Actors: <Admin>

Stakeholders Interest: Admin:Wants to updates information of an existing client.

Pre-Condition:

- 1 Admin must be logged in.
- 2 At least one client must be exists.

Main Success Scenario:

User Action		System Response	
1	User opens the “View Clients” page.		
		2	System displays a list of all clients with ID, name, email, phone, address, and status
3	User selects a client to view details.		
		4	System shows full client profile including: associated projects, booking history, status, and financial records

Post-Condition: User successfully view client information.

Extension Points / Alternative way: 1a. **No client availabe:**
System prompts “No client found”.

Priority: Medium

Frequency: High

Cross-Reference: SRS81, SRS82, SRS83, SRS84

Use case ID: UC-17
Use case Name: Search client

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:

This use case describes how an admin search for a client using different filters.

Primary Actors: <Admin>

Stakeholders: Admin:Wants to Search specific client.

Interest:

Pre-Condition:

- 1.System must be running
- 2.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User enters search criteria (name, ID, email, phone, company).		
		2	System validates search input.
		3	System returns matching clients in list format.

Post-Condition: User view filtered list of client.

Extension Points / Alternative way: 1a. **Empty search field:**
System prompts "Enter search value".

Priority: Medium

Frequency: High

Cross-Reference: SRS85, SRS86, SRS87, SRS88

Use case ID: UC-18
Use case Name: Add Booking Request

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:

This use case describes how clients can add their request for booking..

Primary Actors: <Client>

Stake Holders s Admin:Wants to Search specific client.

Interest:

Pre-Condition:

- 1.System must be running
- 2.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	Client selects a project and unit.		
2	Client fills booking request form.		
		3	System validates the form fields
		4	System saves booking with status: Pending Approval.
		5	System notifies Admin of the new booking request.
6	Client see “booking request successfully submitted”.		

Post-Condition: Bookig request is saved in system with pending status and admin will look on this.

Extension Points / Alternative way:

1a. Unit already booked:

System prompts “This unit is not available”.

1b. Client leaves required field empty:

System prompts “Please fill required field ”.

3a. Invalid data in form:

System display “Please Enter valid data”.

Priority: High

Frequency: Frequently used

Cross-Reference: SRS89, SRS90, SRS91, SRS92, SRS93

Use case ID: UC-19
Use case Name: Approved / Reject Booking

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 01-Dec-25

Use case Description:
This use case describes how admin reject or accept request of client and system notify user with booking status.

Primary Actors: <Admin>

Stakeholders Interest:
Admin: Wants to verify booking detail before approval and availability of unit.
Client: Want to check status of request.

Pre-Condition:
1. System must be running
2. Admin must be logged in.
3. Booking request exists in "Pending" Status.

Main Success Scenario:

User Action		System Response	
1	Admin opens Pending Booking Requests.		
2	Admin selects a booking to review.		
		3	System displays full booking details.
4	Admin chooses Approve/Reject.		
		5	System updates status to Approved or Rejected.
		6	System sends notification to client.
		7	Admin sees success confirmation.

Post-Condition: Booking status approved or rejected and client get notified.

Extension Points / Alternative way:
1a. Admin does not select status:
System shows "Kindly select status for request".
1b. booking already processed:
System prompts " This booking already approved or rejected".

Priority: High
Frequency: Frequently used

Cross-Reference: SRS100, SRS101, SRS102, SRS103

Use case ID: UC-20

Use case Name: Search Booking

Use case Prepared by: M junaid Riaz

Use case Prepared on: 01-Dec-25

Use case Description:

Admin searches booking using filters.

Primary Actors: <Admin>

Stake Holders s Admin:Wants to verify booking detail before approval and avability of unit.

Interest: Client: Want to cheack status of request .

Pre-Condition:

1.Admin logged in

Main Success Scenario:

User Action		System Response	
1	Admin enters search criteria (Booking ID, Client, Project, Status, Date Range).		
		2	System fetches and displays matched booking records.

Post-Condition: Search result successfully displayed

Extension Points / Alternative way: 1a. No matchching filter found:
System shows “NO result found”.

Priority: Medium

Frequency: Frequently used

Cross-Reference: SRS97, SR98,SRS99

Use case ID:	UC-21
Use case Name:	Edit booking

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 02-Dec-25

Use case Description:
Admin updates booking detail like dates , recourses or make some correction

Primary Actors: <Admin>

Stake Holders s	Admin:Wants to modify or correct booking details.
Interest:	Client: Want to cheack status of request .

Pre-Condition:

- 1.Admin must be logged in.
- 2.Booking must be exists.

Main Success Scenario:

User Action		System Response	
1	Admin selects a booking to edit.		
		2	System displays editable booking form
3	Admin updates fields.		
4	System validates updates.		
		5	System saves new booking details.
6	System shows success message.		

Post-Condition: Bookig detail updates successfully.

Extension Points/ Alternative way:	3a. Invalid data entered: System prompts “Enter valid dataf”. 4a. Recourse Unavailable: System prompts “selected recourse is not available”.	
	1a. Booking cancel: System display “Please Enter valid da	

Priority: High

Frequency: less Frequently

Cross-Reference: SRS104, SRS105, SRS106, SRS107

Use case ID: UC-22
Use case Name: Cancel booking

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 02-Dec-25

Use case Description:
Admin updates booking detail like dates , recourses or make some correction

Primary Actors: <Admin> <Client>

Stake Holders s Interest: Admin:Can cancel booking with respect to some policies or on client request.
Client: Get notified on cancelling .

Pre-Condition:

- 1.Admin must be logged in.
- 2.Booking must be exists.
- 3.Booking msut not be already cancelled.

Main Success Scenario:

User Action		System Response	
1	Admin selects a booking to cancel.		
		2	2. System shows booking details.
3	Admin clicks Cancel Booking.		
		4	System updates booking to “Cancelled”.
		5	System sends cancellation email to client.
6	System displays success message		

Post-Condition: Booking is cancelled.

Extension Points / Alternative way:

1a. Booking already Cancelled.
System prompts “Booking is already cancelled”.

4a. Admin does not provide cancellation reason:
System prompts “Provide cacellation reason”.

Priority: High

Frequency: Medium

Cross-Reference: SRS100, SRS101, SRS102, SRS103

Use case ID: UC-23
Use case Name: Add Contruction progress

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 03-Dec-25

Use case Description:

This use case discribe how admin can upload progress of projects including discription , task completed, issues or next plan

Primary Actors: <Admin>

Stake Holders s Interest: Admin: Wants to upadte real time construction progress .
Wants to doucment work completed and next steps.

Pre-Condition:

- 1.Admin must be logged in.
- 2.Project must be exist in the system.

Main Success Scenario:

User Action		System Response	
1	User selects a project and opens Add Progress page.		
		2	System displays progress entry form.
3	User enters progress percentage, description, work done, next steps.		
		4	System validates all inputs.
		5	System saves progress entry and timestamps it.
6	System shows success confirmation.		

Post-Condition: A new construction progress entry is recorded and linked to the project.

Extension Points / Alternative way: **3a. Progress percentage exceeds limits (e.g., >100%).**
System prompts “Progress must be between 0 - 100%”.
4a. Required field empty:
System prompts “Please fill all required fields”.

Priority: High

Frequency: Very frequent

Cross-Reference: SRS108, SRS109, SRS110, SRS111

Use case ID: UC-24

Use case Name: Edit Contruction progress

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 03-Dec-25

Use case Description:

User updates previously entered progress details such as percentage, status, description, challenges, or next steps.

Primary Actors: <Admin>

Stakeholders	Admin: Wants updated and accurate project information .
Interest:	

Pre-Condition:

- 1.Admin must be logged in.
- 2.Project entry must exist.

Main Success Scenario:

User Action		System Response	
1	User selects a progress entry to edit.		
		2	System displays current progress details.
3	User updates progress information.		
		4	System validates updated data.
		5	System saves updated progress information.
6	System shows success message.		

Post-Condition: Progress entry is updated and previous version is stored in history.

Extension Points / Alternative way:	3a. Invalid progress value.
	System prompts “Progress must be between 0 - 100%”.
	2a. Progress entry not found:
	System prompts “Progress record dose not exist”.

Priority: Medium

Frequency: Moderate

Cross-Reference: SRS112, SRS113, SRS114, SRS115

Use case ID: UC-25
Use case Name: View Construction progress

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 03-Dec-25

Use case Description:

This use case describes how users view the construction progress of any project along with percentage, timeline, and past updates.

Primary Actors: <Admin> <client>

Stake Holders s Interest: Admin: Wants to oversee overall project development .
Client: Want transparency about development status.

Pre-Condition:

- 1.Admin must be logged in.
- 2.Project must exist with progress entries.

Main Success Scenario:

User Action		System Response	
1	User opens project progress section.		
		2	System displays list of progress entries sorted by date.
3	User selects any progress entry		
		4	System shows detailed progress including percentage, notes, media, timeline.

Post-Condition: Admin and client successfully views progress..

Extension Points / Alternative way: 1a. **No progress recorded.**
System prompts “No progress update for this project”.

Priority: Medium

Frequency: Frequent

Cross-Reference: SRS116, SRS117, SRS118, SRS119

Use case ID: UC-26
Use case Name: Upload progress media

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 03-Dec-25

Use case Description:

User uploads construction-related images or videos for any progress update.

Primary Actors: <Admin>

Stake Holders s	Admin: Wants visual verification and transparency .
Interest:	Client: Wants to see real progress visually.

Pre-Condition:

- 1.Admin must be logged in.
- 2.Project must exist with progress entries.

Main Success Scenario:

User Action		System Response	
1	User opens Upload Media option.		
		2	System shows upload interface.
3	User selects images/videos.		
		4	System validates file type & size.
		5	System uploads and attaches media to progress entry.
5	System shows success message.		

Post-Condition: Media is uploaded and linked with progress entry.

Extension Points/ Alternative way:	4a. Invalid file format. System prompts “Invalid file type. Allowed types: JPG, PNG, MP4”.
---	--

4b. File size to large.
System prompts “Maximum file size is 50MB”.

Priority: Medium

Frequency: Frequent

Cross-Reference: SRS120 SRS121, SRS122, SRS123

Use case ID: UC-27
Use case Name: Send Notification

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description:

This use case describes how an Admin or Project Manager sends notifications to users regarding project updates, reminders, or system alerts.

Primary Actors: <Admin>

Stake Holders s Interest: Admin: Wants to quickly notify users about important updates.
Client: Wants timely notifications about bookings, progress, updates.

Pre-Condition:

- 1.Admin must be logged in.
- 2.Recipient must be exist in system..

Main Success Scenario:

User Action		System Response	
1	User selects notification type & recipients		
		2	System displays notification form.
3	User enters message content & clicks send.		
		4	System validates message and recipients.
		5	System delivers notification via email/SMS/in-app (if configured).
5	System shows “Notification Sent Successfully.”		

Post-Condition: Notification is stored in the notification log.

Extension Points / Alternative way: **3a. Message field empty.**
System prompts “Messge can not be empty”.
2a. No recipientes selected.
System prompts “Please select atleast one recipient”.

Priority: High

Frequency: Frequent

Cross-Reference: SRS125, SRS126, SRS127, SRS128, SRS129

Use case ID:	UC-28
Use case Name:	View Notification

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description:

Users view all notifications received including updates, reminders, and announcements.

Primary Actors: <Admin>

Stake Holders s	Admin: Want to send and monitor all notification
Interest:	Client: Want to stay updated on project progress.

Pre-Condition:

1.Admin and client must be logged in.

Main Success Scenario:

User Action		System Response	
1	Admin open notification center or menu		
		2	System displays list of notification sorted by date.
3	User Click on notification Center.		
		4	System show full detail of notification.

Post-Condition: Notification status change to read..

Extension Points / Alternative way:	1a. No notification available. System prompts “No new notification”.
--	--

Priority: Medium

Frequency: High

Cross-Reference: SRS130, SRS131, SRS132, SRS133,

Use case ID:	UC-29
Use case Name:	Add Expense

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description:

UThis use case discribe how admin add record of new expense including ammout, date , project and ammout.

Primary Actors: <Admin>

Stakeholders	Admin: Want to track all company and project expense.
Interest:	Finance Team: Wants accurate financial data.

Pre-Condition:

- 1.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User opens “Add Expense” page.		
		2	System displays the expense form.
3	User enters all details and uploads supporting documents.		
		4	System validates all required fields and file size/type.
		6	System assigns a unique Expense ID and stores data.
		7	System shows success message “Expense Added Successfully.”

Post-Condition: A new expense record is saved in the system

Extension Points / Alternative way:	3a. Missing required field. System prompts “Please fill required field”.
--	--

3b. Invalid ammount.
System prompts “Please enter positive ammount”.

Priority: High

Frequency: very frequent

Cross-Reference: SRS139, SRS140, SRS141, SRS142, SRS143

Use case ID:	UC-30
Use case Name:	edit Expense

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description:

This use case describe how admin can edit an existing expense entry.

Primary Actors: <Admin>

Stake Holders s	Admin: Want to correct expense mistake.
Interest:	

Pre-Condition:

- 1.Admin must be logged in.
- 2.Expense record must be exits.

Main Success Scenario:

User Action		System Response	
1	User searches and selects an expense to edit.		
		2	System displays current expense details.
3	User updates fields and clicks “Save.”		
		4	System validates new values.
		5	System stores updated expense and logs modification history.
		6	System displays success message.

Post-Condition: Expense record update successfully.

Extension Points/ Alternative way:	1a. Expense not found. System prompts “Expense not found”.
---	--

3a. Invalid ammount.
System prompts “Please enter positive ammount”.

Priority: High

Frequency: very frequent

Cross-Reference: SRS144, SRS145, SRS146, SRS147, SRS148

Use case ID: UC-31
Use case Name: Delete Expense

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case discribe how admin can delete an existing expense entry.

Primary Actors: <Admin>

Stake Holders s Admin: Want to remove incorrct expense.
Interest:

Pre-Condition:
1.Admin must be logged in.
2.Expense record must be exits.

Main Success Scenario:

User Action		System Response	
1	User clicks “Delete Expense.”		
		2	System asks for confirmation.
3	User confirms deletion.		
		4	System performs soft delete and moves record to archive.
		5	System sends notification to project manager.
		6	System shows success message.

Post-Condition: Expense marked as deleted and stored in archive.

Extension Points / **1a. Expense not found.**
Alternative way: System prompts “Expense not found”.
.

Priority: Medium

Frequency: Less frequent

Cross-Reference: SRS149, SRS150, SRS151, SRS152

Use case ID: UC-32
Use case Name: View Expense

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how users view all expenses with filtering, sorting, and detailed viewing.

Primary Actors: <Admin>

Stake Holders s Interest: Admin: Want to monitor overall expense.

Pre-Condition:
1.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User opens “Expenses List.”		
		2	System displays expense table with pagination.
3	User applies filters (type, date, project, amount).		
		4	System updates table based on filters.
		5	System sends notification to project manager.
		6	System shows success message.

Post-Condition: User views expense data successfully.

Extension Points / Alternative way: **1a. Expense not found.**
System prompts “ No Expense records found”.

Priority: Medium

Frequency: Frequent

Cross-Reference: SRS153, SRS154, SRS155, SRS156

Use case ID: UC-33
Use case Name: Profit & loss report

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how the system generates project-wise or date-range P&L reports.

Primary Actors: <Admin>

Stake Holders s Interest: Admin:Wants insight into financial performance

Pre-Condition:
1.Admin must be logged in.
2.Expense record data must be exist.

Main Success Scenario:

User Action		System Response	
1	User opens P&L Report.		
		2	System displays filter options (date range, project).
3	User selects filters and clicks “Generate.”		
		4	System calculates total revenue and expenses.
		5	System shows detailed P&L in charts + tables.
6	User exports report.		
		7	System downloads PDF/Excel file.

Post-Condition: System downloads PDF/Excel file.

Extension Points / Alternative way: 1a. No financial data exists.
System prompts “ No financial data available for selected period.”.

Priority: High

Frequency: Moderate

Cross-Reference: SRS157, SRS158, SRS159, SRS160, SRS161, SRS162

Use case ID:	UC-34
Use case Name:	Record Income

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how a user records a new income entry including amount, type, project/client details, and payment method.

Primary Actors: <Admin>

Stakeholders	Admin: Wants to complete record of all income sources.
Interest:	Finance Team: Wants accurate revenue records for reporting and reconciliation.

Pre-Condition: 1.Admin must be logged in.

Main Success Scenario:

User Action		System Response	
1	User opens the “Record Income” page.		
		2	System loads income entry form.
3	User enters all details (type, amount, date, project/client, payment method).		
		4	System validates all fields.
5	User clicks “Submit.”		
		6	System assigns unique income ID and stores data.
		7	System shows success message “Income Recorded Successfully.”

Post-Condition: A new income entry is added to the system.

Extension Points / Alternative way:	3a. Missing mandatory field. System prompts “ Please fill required field”.
	3b. Invalid ammount. System prompts “ Amount must be greater than 0”.

Priority: High

Frequency: Frequently used

Cross-Reference: SRS163, SRS164, SRS165, SRS167,

Use case ID: UC-35
Use case Name: Edit Income

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how a user modifies an existing income entry and updates relevant fields.

Primary Actors: <Admin>

Stakeholders Interest: Admin: Wants to correct income errors
Finance Team: Needs accurate revenue details.

Pre-Condition:
1.Admin must be logged in.
2.Income record must be exist already.

Main Success Scenario:

User Action		System Response	
1	User opens income list and selects an income to edit.		
		2	System displays existing income details.
3	User updates required fields.		
		4	System validates updated information.
5	User clicks “Save Changes.”		
		6	. System updates entry and logs modification history.
		7	System shows success message.

Post-Condition: A new income entry is added to the system.

Extension Points / Alternative way: **3a. Invalid updated amount.**
System prompts “ Amount must be greater than 0”.

Priority: Medium

Frequency: Moderate

Cross-Reference: SRS168, SRS169, SRS170, SRS171

Use case ID: UC-36
Use case Name: View Income Record

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how users view income records in a tabular, paginated interface.

Primary Actors: <Admin>

Stakeholders Interest: Admin: Wants to review income records.

Pre-Condition:
1.Admin must be logged in.
2.Income record must be exist already.

Main Success Scenario:

User Action		System Response	
1	User opens the “Income Records” page.		
		2	System displays income table with pagination.
3	User clicks an income record to view details.		
		4	System displays detailed income information including payment status.

Post-Condition: User successfully views income details.

Extension Points / Alternative way: **1a. No record found.**
System prompts “ No record available”.

Priority: Medium

Frequency: Frequent

Cross-Reference: SRS172, SRS173, SRS174

Use case ID: UC-37
Use case Name: Filter Income

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how users apply filters to income records to find specific results.

Primary Actors: <Admin>

Stakeholders Interest: Admin: Need customized filter data.

Pre-Condition:
1.Admin must be logged in.
2.Income data must be exist .

Main Success Scenario:

User Action		System Response	
1	User opens income filter panel.		
2	User selects filters (date, project, client, income type, amount range).		.
		3	System processes selected filters.
		4	System displays filtered records and summary totals.
5	User optionally saves filter configuration.		
		6	System saves filter for quick future use.

Post-Condition: Filtred income list amd summarized displayed.

Extension Points / Alternative way: **1a. No Matching record.**
System prompts “ No record found”.

Priority: Medium

Frequency: Frequent

Cross-Reference: SRS175, SRS176, SRS177, SrS178

Use case ID: UC-38
Use case Name: Income Summary

Use case Prepared by: M junaid Riaz **Use case Prepared on:** 04-Dec-25

Use case Description: This use case describes how users generate summary income reports for selected date ranges.

Primary Actors: <Admin>

Stake Holders s Interest: Admin: Wants Performance summararies.

Pre-Condition:
1.Admin must be logged in.
2.Income record must be exist .

Main Success Scenario:

User Action		System Response	
1	User opens “Income Summary Report.”		
		2	.System displays report filter options.
3	. User selects date range or project/client.		
		4	System calculates total received, pending, breakdowns, etc.
5	User exports report.		
		6	System downloads PDF/Excel version.

Post-Condition: Summary report generated and optionally exported.

Extension Points / Alternative way: 1a. No Income data available.
System prompts “ No record found”.

Priority: High

Frequency: Moderate

Cross-Reference: SRS179, SRS180, SRS181, SRS182, SRS183

Use case ID: UC-39

Use case Name: Manage user role & permission

Use case Prepared by: M junaid Riaz

Use case Prepared on: 11-Dec-25

Use case Description: This use case describes how an admin can assign or update the role or permission.

Primary Actors: <Admin>

Stake Holders s Interest: Admin: Wants to control and manage system access.
User: Should only access allowed access modules.

Pre-Condition:
1.Admin must be logged in.
2.Client or user record must be exist. .

Main Success Scenario:

User Action		System Response	
1	Admin opens “User Roles & Permissions” page.		
		2	System displays list of users with current roles.
3	. Admin selects a user to modify		
		4	System loads user profile and editable role options
5	Admin assigns a new role or updates permissions.		
		6	System validates role selection.
7	Admin clicks “Save”.		
		8	System updates role/permissions and logs changes.
		9	System shows “Role updated successfully.

Post-Condition: User role and authorized permissions are updated and stored in the system.

Extension Points / Alternative way:	3a. user not found. System prompts “ User record does not exists”.
Priority:	High
Frequency:	Moderate
Cross-Reference:	SRS10, SRS11, SRS13,

Domain Model

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

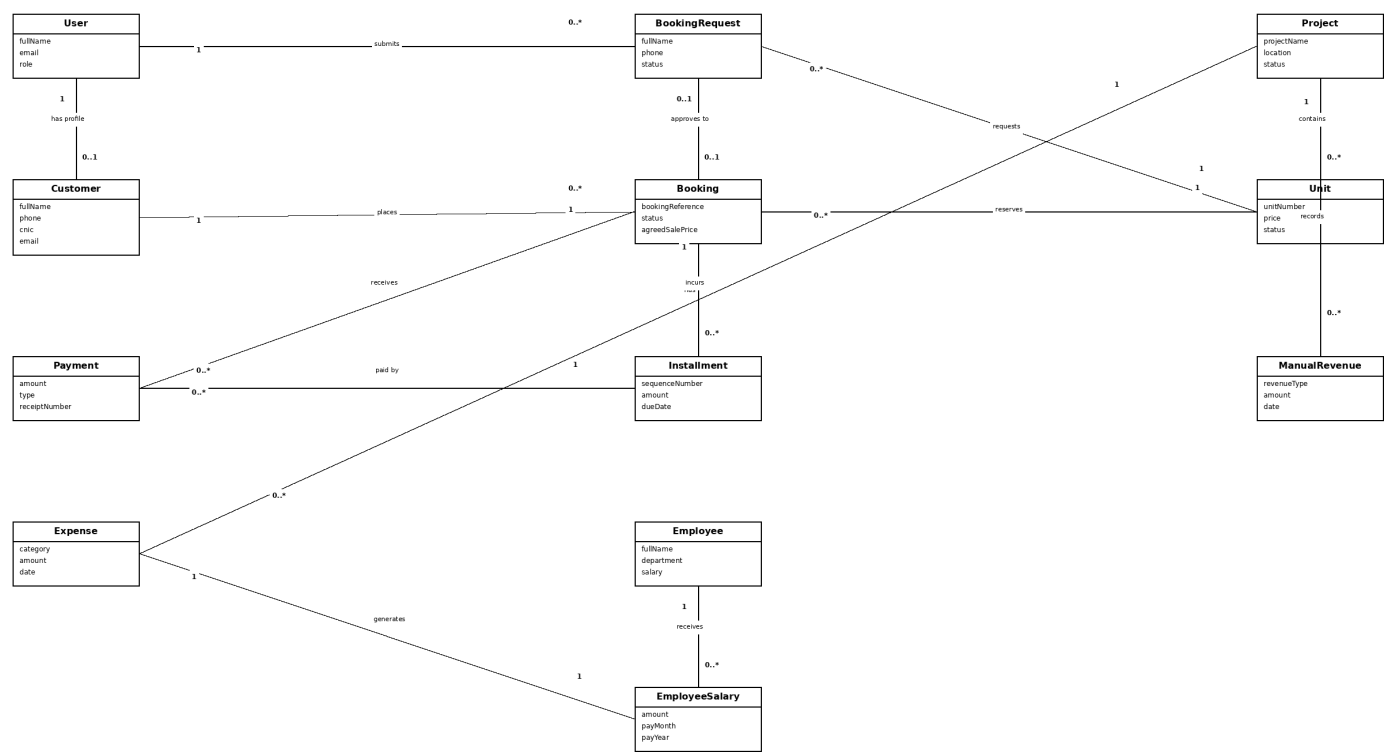
Muhammad Junaaid Riaz

27th November, 2025

3.3 Domain Model

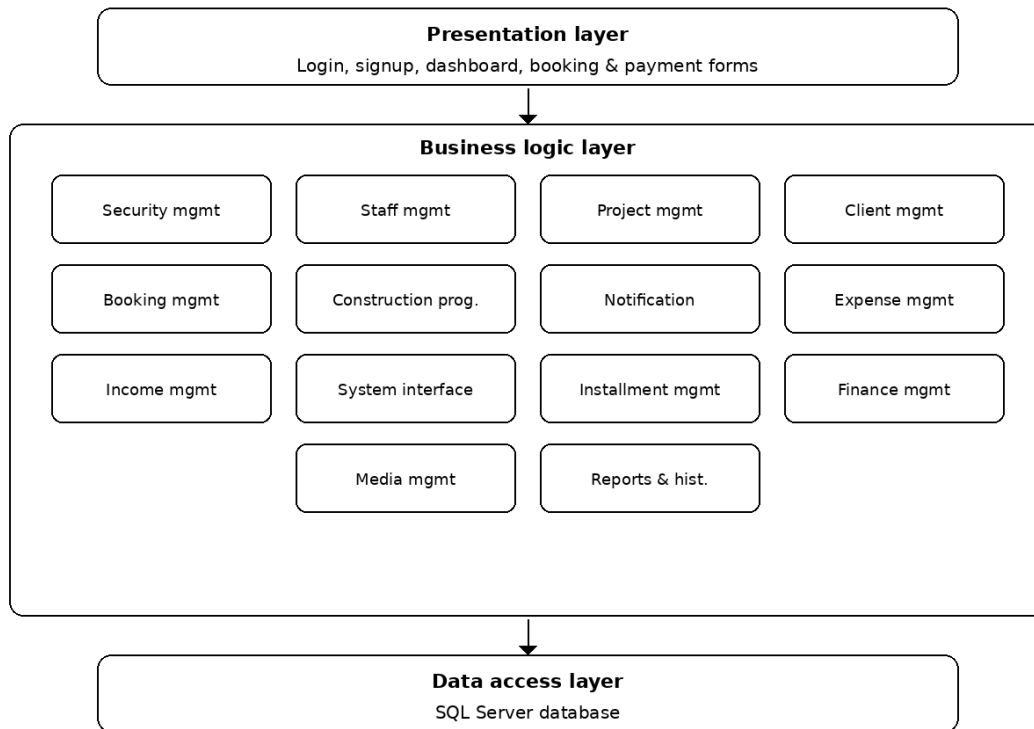
3.3 Domain Model

UML domain model (Larman): conceptual classes with key attributes, associations with multiplicity and names. No operations or navigability arrows.



3.4 Package Diagram

3.4 Package Diagram



System Sequence Diagrams

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

Muhammad Junaaid Riaz

27th November, 2025

3.5 System Sequence Diagram

System Sequence Diagram — User Registration

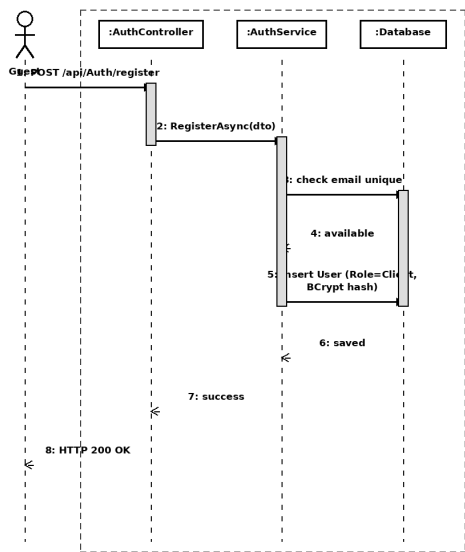


Figure 3.1: User Registration (Guest creates Client account)

System Sequence Diagram — User Login

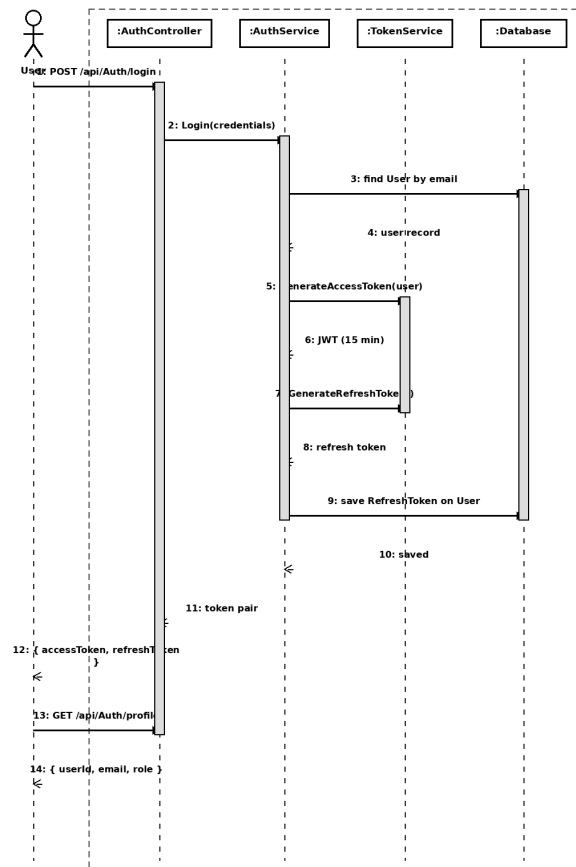


Figure 3.2: User Login and profile load

System Sequence Diagram — Browse Projects

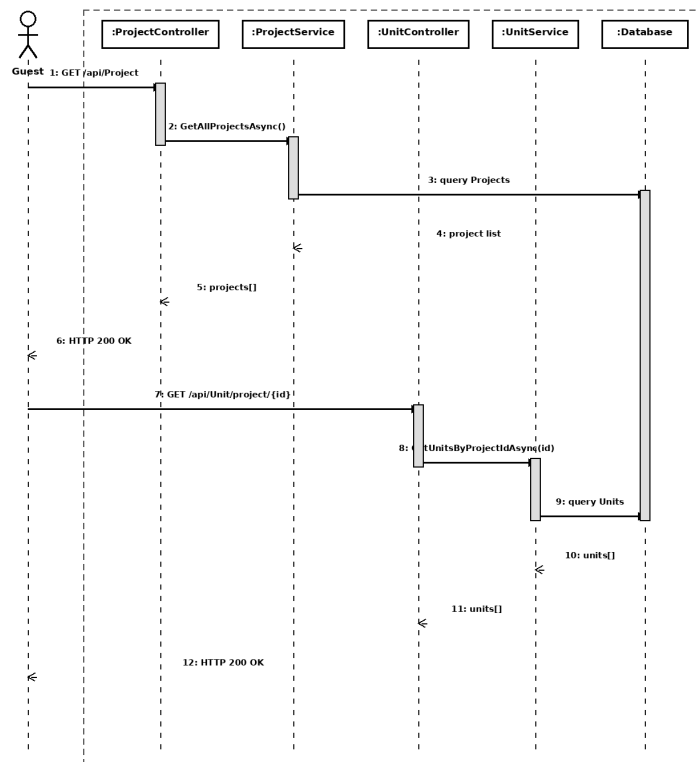


Figure 3.3: Guest browses projects and units

System Sequence Diagram — Submit Booking Request

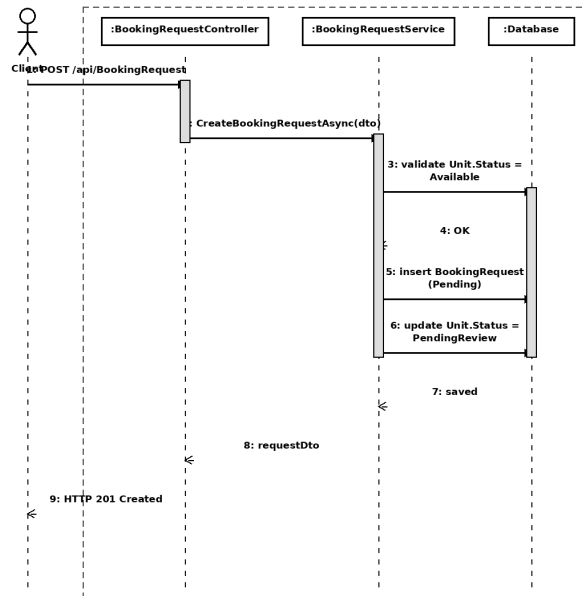


Figure 3.4: Guest/Client submits booking request for a unit

System Sequence Diagram — Approve Booking Request

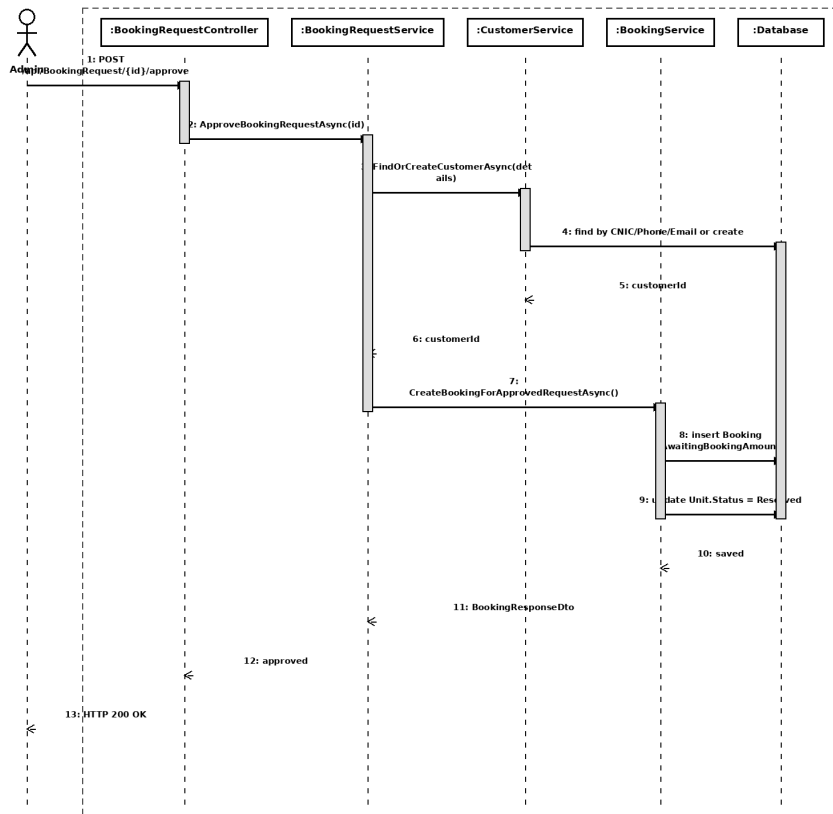


Figure 3.5: Admin approves request and creates confirmed booking

System Sequence Diagram — Reject Booking Request

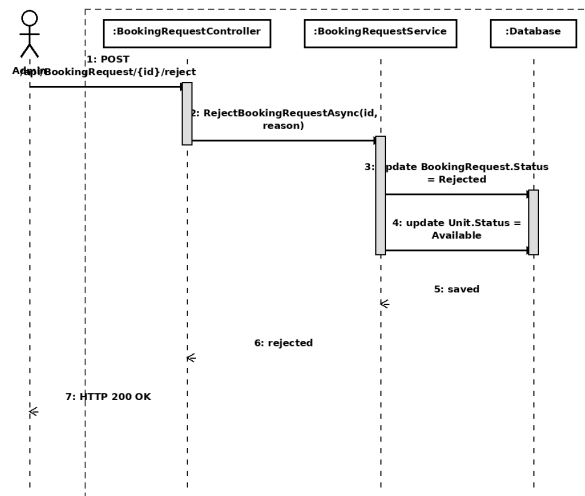


Figure 3.6: Admin rejects booking request and releases unit

System Sequence Diagram — Create Walk-in Booking

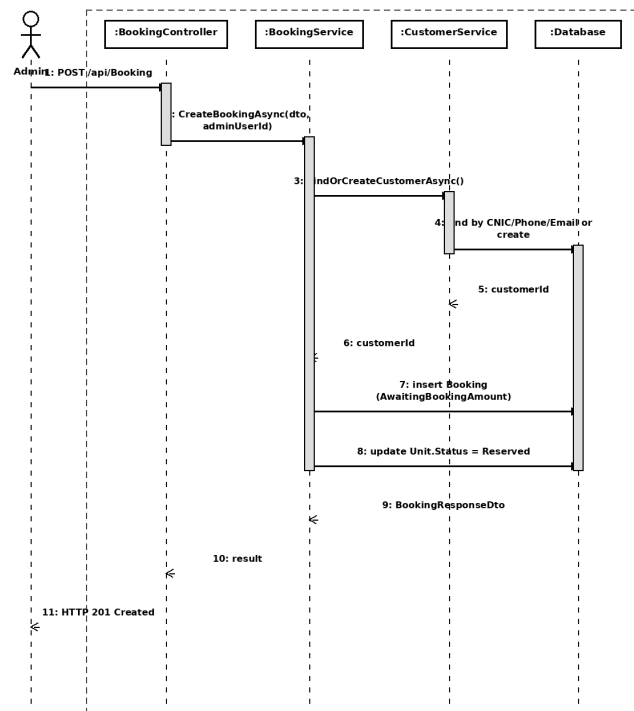


Figure 3.7: Admin creates direct booking for walk-in customer

System Sequence Diagram — Record Booking Amount Payment

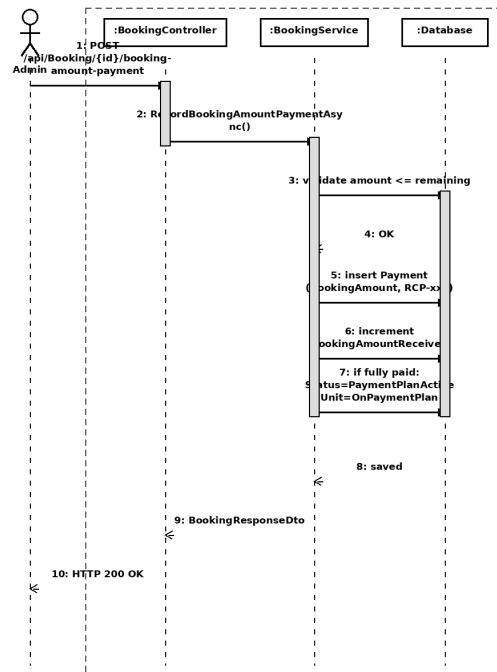


Figure 3.8: Admin records booking amount; status becomes PaymentPlanActive

System Sequence Diagram — Generate Installment Plan

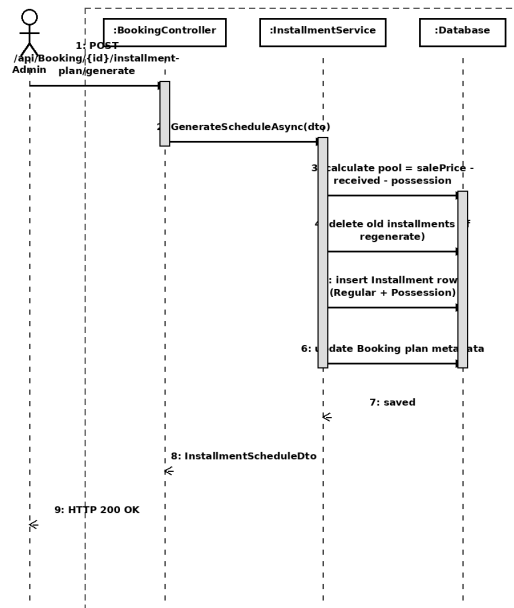


Figure 3.9: Admin generates installment schedule for booking

System Sequence Diagram — Record Installment Payment

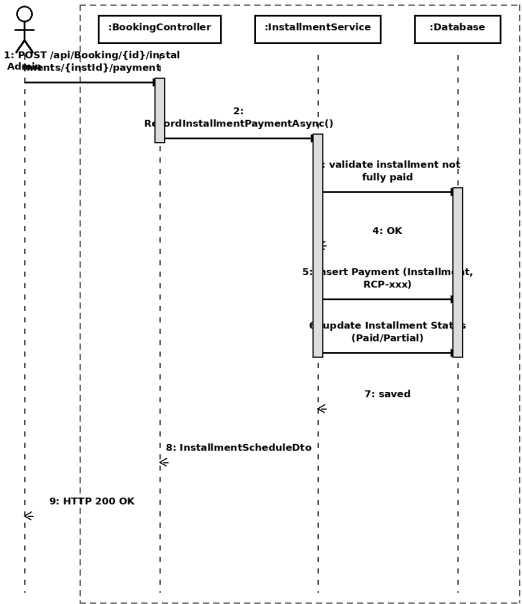


Figure 3.10: Admin records installment payment and generates receipt

System Sequence Diagram — View Payment Receipt

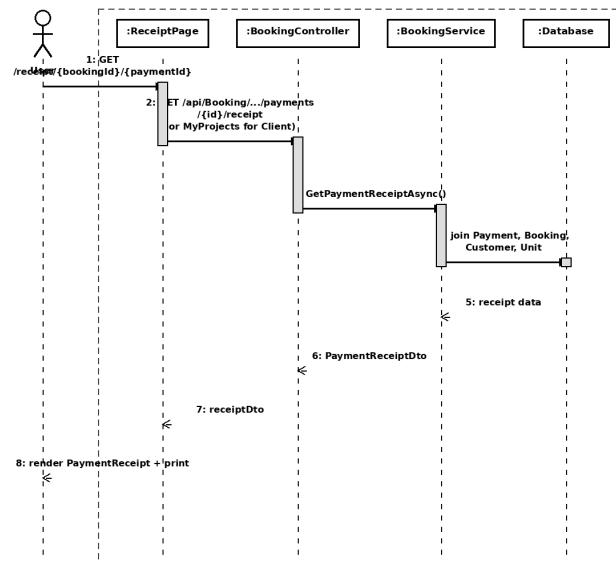


Figure 3.11: Admin or Client views printable payment receipt

System Sequence Diagram — Client My Projects

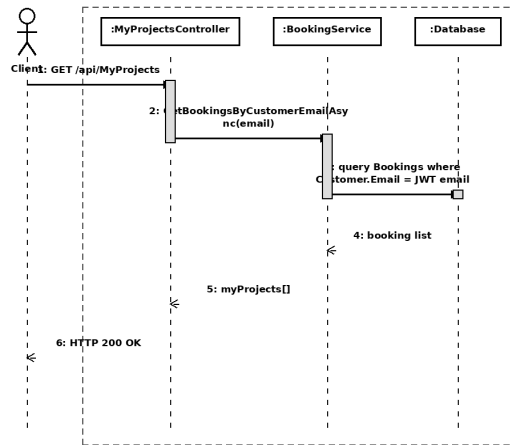


Figure 3.12: Client views own confirmed bookings by email match

System Sequence Diagram — Cancel Booking

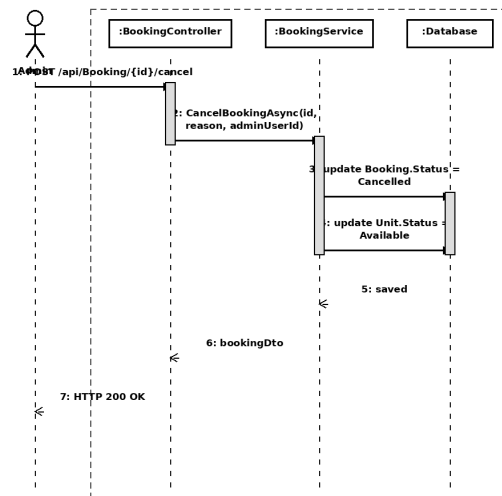


Figure 3.13: Admin cancels booking and releases unit

System Sequence Diagram — Mark Employee Attendance

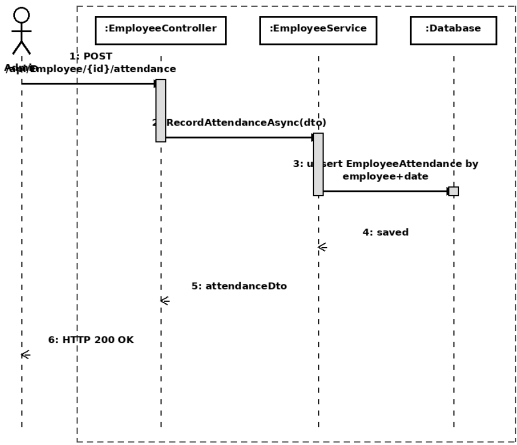


Figure 3.14: Admin records daily attendance for employees

System Sequence Diagram — Generate Employee Salary

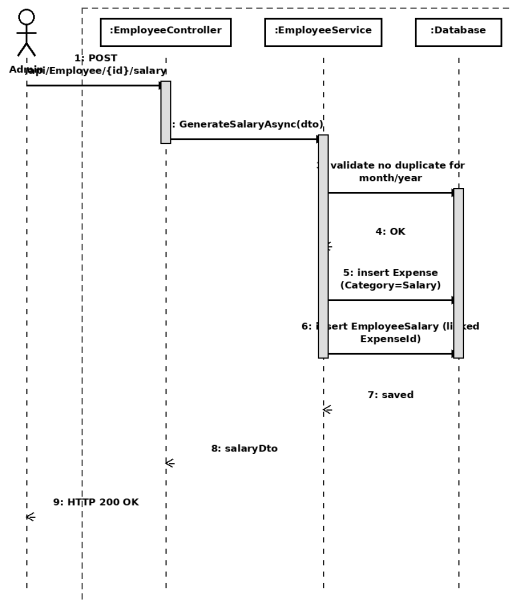


Figure 3.15: Admin generates salary payment and linked expense

System Sequence Diagram — Finance Dashboard

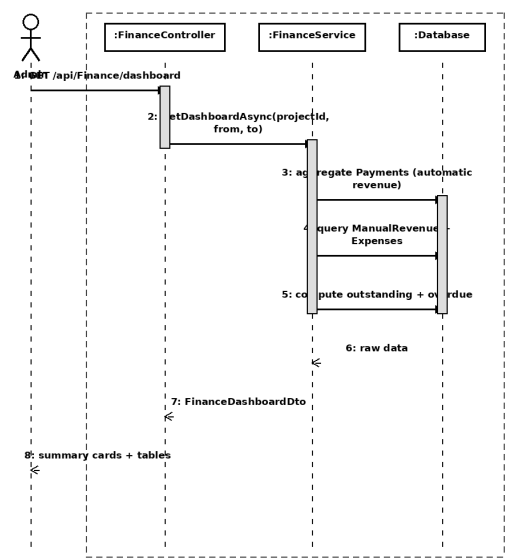


Figure 3.16: Admin loads financial summary and transaction lines

System Sequence Diagram — Add Manual Revenue

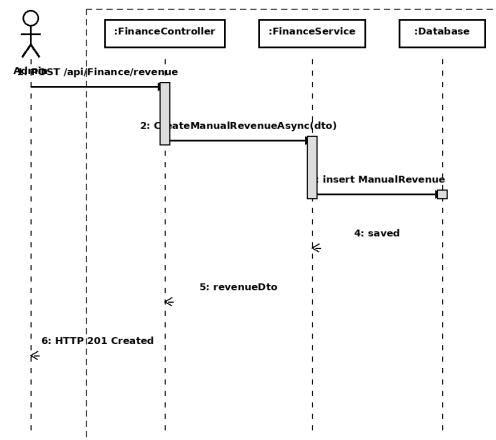


Figure 3.17: Admin adds manual revenue entry to finance ledger

System Sequence Diagram — Add Expense

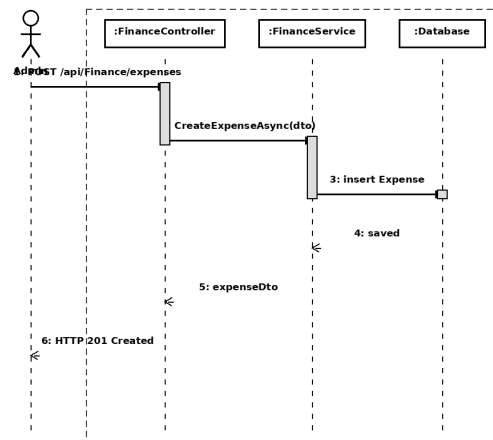
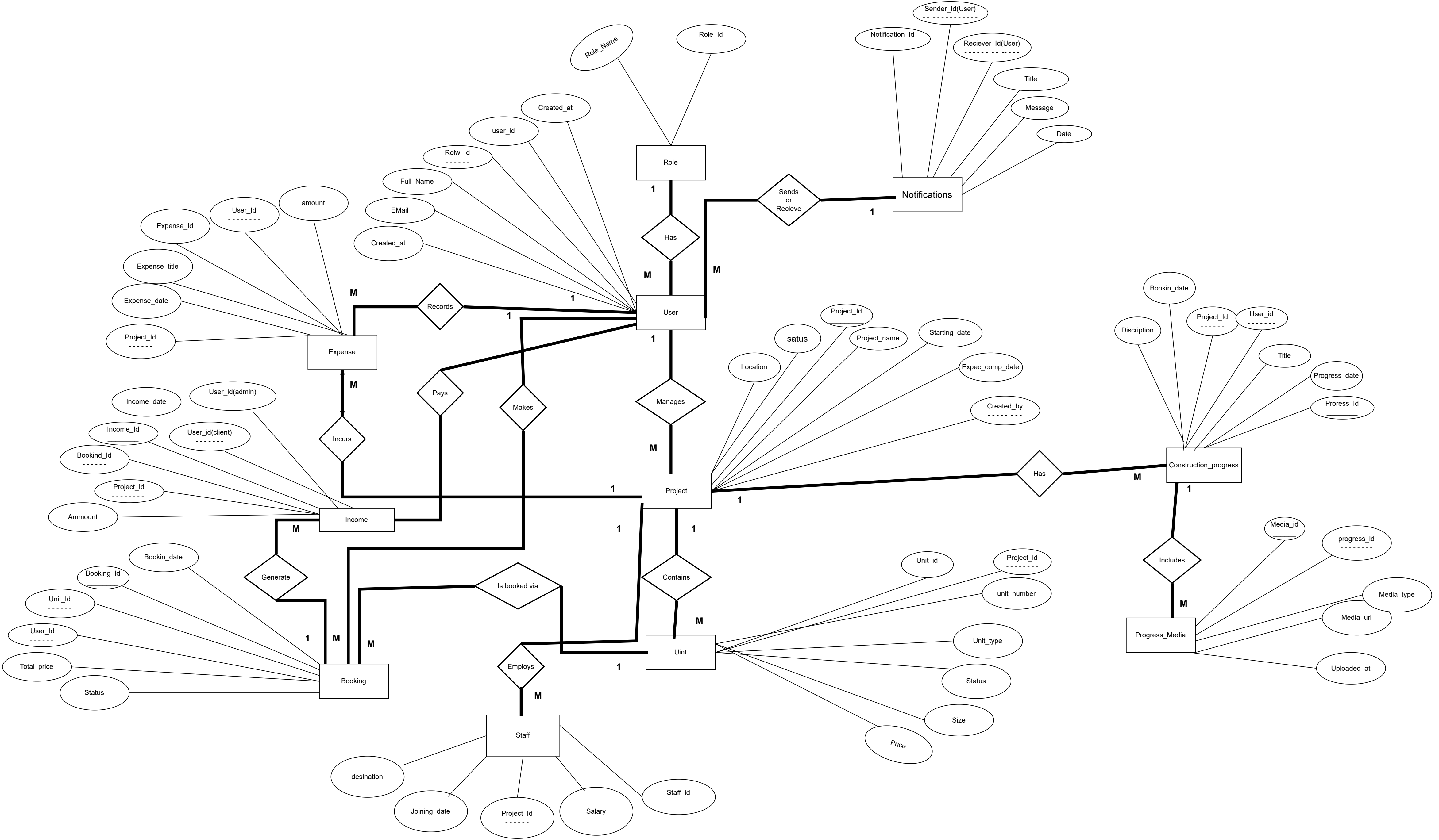


Figure 3.18: Admin adds expense entry to finance ledger

3.6 ER Diagram



Normalization

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

Muhammad Junaaid Riaz

27th November, 2025

3.7 Normalization

Revision History

Version	Description	Author	Date
1.0	This covers the normalization of the system	Junaid Riaz	27 - 12 - 2025

1.Table “Role”:

Role	
Role_Id (pk)	Role_name

Table “Role” is already in 1NF,2NF and 3NF. No further normalization is required.

2. Table “ User”

User					
User_Id(Pk)	Role_Id(Fk)	Full_name	email	Phone	password

Applying 1NF:-

“User”

User			
User_Id(PK)	Role_Id(FK)	Full_name	Password

“User_email”

User_email	
User_Id (PK) (FK)	email(PK)

“User_contact”

User_contact	
User_Id (PK) (FK)	Contact_no(PK)

There are no partial or transitive dependencies in above tables; therefore, no further normalization is required.

3. Table “Project”

Project						
Project_Id(PK)	User_Id(FK)	Project_name	location	Start_date	End_date	Satus

Since the Project table already satisfies the requirements of the higher normal forms, no additional normalization is necessary.

4. Table “ Notification”

Notification					
Notification_Id(PK)	Sender_Id (FK)	Receiver_Id (FK)	Title	Message	Notification_Date

Since the Notification table already satisfies the requirements of the higher normal forms, no additional normalization is necessary.

5. Table “Expense”

Expense					
Expense_Id(PK)	Project_Id(FK)	User_Id(FK)	Title	Date	Ammount

The Expense table is already normalized; therefore, no further normalization is required.

6. Table “Unit”

Unit					
Unit_Id(PK)	Project_Id(FK)	Unit_type	Unit_number	Price	Size

Table “Unit” is already in 1NF, 2NF, and 3NF. No further normalization is required.

7. Table “Booking”

Booking					
Booking_Id(PK)	Unit_Id(FK)	User_Id(FK)	Booking_date	Total_Price	Status

Table “Booking” is already in 1NF, 2NF, and 3NF. No further normalization is required.

8. Table “Income”

Income					
Income_Id(PK)	Booking_Id(FK)	Paid_b(FK)	Recorded_B(FK)	Income_date	Ammount

Table “Income” is already in 1NF, 2NF, and 3NF. No further normalization is required.

9. Table “Staff”

Staff				
Staff_Id(PK)	Project_Id(FK)	Desination	salary	Joining_date

Table “Staff” is already in 1NF, 2NF, and 3NF. No further normalization is required.

10. Table “Progress”

Progress				
Progress_Id(Pk)	Project_Id(FK)	Title	Discription	date

Table “Progress” is already in 1NF, 2NF, and 3NF. No further normalization is required.

11. Progress Media

Progress Media				
Media_Id(PK)	Progress_Id(FK)	Type	Url	Date

Table “Progress_Media” is already in 1NF, 2NF, and 3NF.
No further normalization is required.

3.8 Relational Model

User

Attributes	Data Types	Constraints	Primary Key	Foreign Key
user_id	Int	Unique	Yes	Null
role_Id	Int	Not null	No	Yes
full_name	Varchar(100)	Not null	No	No
password	Varchar(250)	Not null	No	No

Role

Attributes	Data Types	Constraints	Primary Key	Foreign Key
role_id	Int	Unique	Yes	Null
role_name	Varchar(100)	Not null	No	Null

User_email

Attributes	Data Types	Constraints	Primary Key	Foreign Key
user_id	Int	Not null	Yes	Yes
email	Varchar(100)	Not null	yes	Null

User_Contact

Attributes	Data Types	Constraints	Primary Key	Foreign Key
user_id	Int	Not null	Yes	Yes
contact	Varchar(14)	Not null	yes	Null

Project

Attributes	Data Types	Constraints	Primary Key	Foreign Key
project_id	Int	Unique	Yes	Null
user_id	Int	Not null	Null	yes
project_name	Varchar(100)	Not null	Null	Null
location	Varchar(150)	Not null	Null	Null
Start_date	Date	Not null	Null	Null
End_date	Date	Not null	Null	Null
status	Varchar(30)	Not null	Null	Null

User

Notification

Attributes	Data Types	Constraints	Primary Key	Foreign Key
Notification_id	Int	Not null,Unique	Yes	Null
Sender_id	Int	Not null	Null	yes
Reciever_id	Int	Not null	Null	yes
title	Varchar(100)	Not null	Null	Null
message	Text	Not null	Null	Null
date	Date	Not null	Null	Null

Unit

Attributes	Data Types	Constraints	Primary Key	Foreign Key
unit_id	Int	Not null,Unique	Yes	Null
project_id	Int	Not null	Null	yes
unit_type	Varchar(50)	Not null	Null	Null
unit_number	Varchar(20)	Not null	Null	Null
price	Decimal(12,2)	Not null	Null	Null
size	Varchar(30)	Not null	Null	Null

Expense

Attributes	Data Types	Constraints	Primary Key	Foreign Key
Expense_id	Int	Not null,Unique	Yes	Null
project_id	Int	Not null	Null	yes
user_id	Int	Not null	Null	yes
title	Varchar(100)	Not null	Null	Null
date	Date	Not null	Null	Null
amount	Decimal(12 , 2)	Not null	Null	Null

User

Booking

Attributes	Data Types	Constraints	Primary Key	Foreign Key
booking_id	Int	Not null,Unique	Yes	Null
unit_id	Int	Not null	Null	yes
user_id	Int	Not null	Null	yes
status	Varchar(20)	Not null	Null	Null
date	Date	Not null	Null	Null
total_price	Decimal(12 , 2)	Not null	Null	Null

Staff

Attributes	Data Types	Constraints	Primary Key	Foreign Key
staff_id	Int	Not null,Unique	Yes	Null
project_id	Int	Not null	Null	yes
desination	Varchar(50)	Not null	Null	Null
Joining_date	Date	Not null	Null	Null
salary	Decimal(12 , 2)	Not null	Null	Null

Progress

Attributes	Data Types	Constraints	Primary Key	Foreign Key
progress_id	Int	Not null,Unique	Yes	Null
project_id	Int	Not null	Null	yes
title	Varchar(100)	Not null	Null	Null
discription	Text	Not null	Null	Null
date	Date	Not null	Null	Null

Progress Media

Attributes	Data Types	Constraints	Primary Key	Foreign Key
media_id	Int	Not null,Unique	Yes	Null
progress_id	Int	Not null	Null	yes
media_type	Varchar(100)	Not null	Null	Null
url	Varchar(100)	Not null	Null	Null
uploaded_date	Date	Not null	Null	Null

User

Income

Attributes	Data Types	Constraints	Primary Key	Foreign Key
income_id	Int	Not null,Unique	Yes	Nil
booking_id	Int	Not null	Nil	yes
paid_by	Int	Not null	Nil	yes
recorded_by	Int	Not null	Nil	yes
income_date	Date	Not null	Nil	Nil
amount	Decimal(12 , 2)	Not null	Nil	Nil

Chapter # 4

Construction

Class Diagram

FOR

Deen Associate Management System

VERSION 1.0

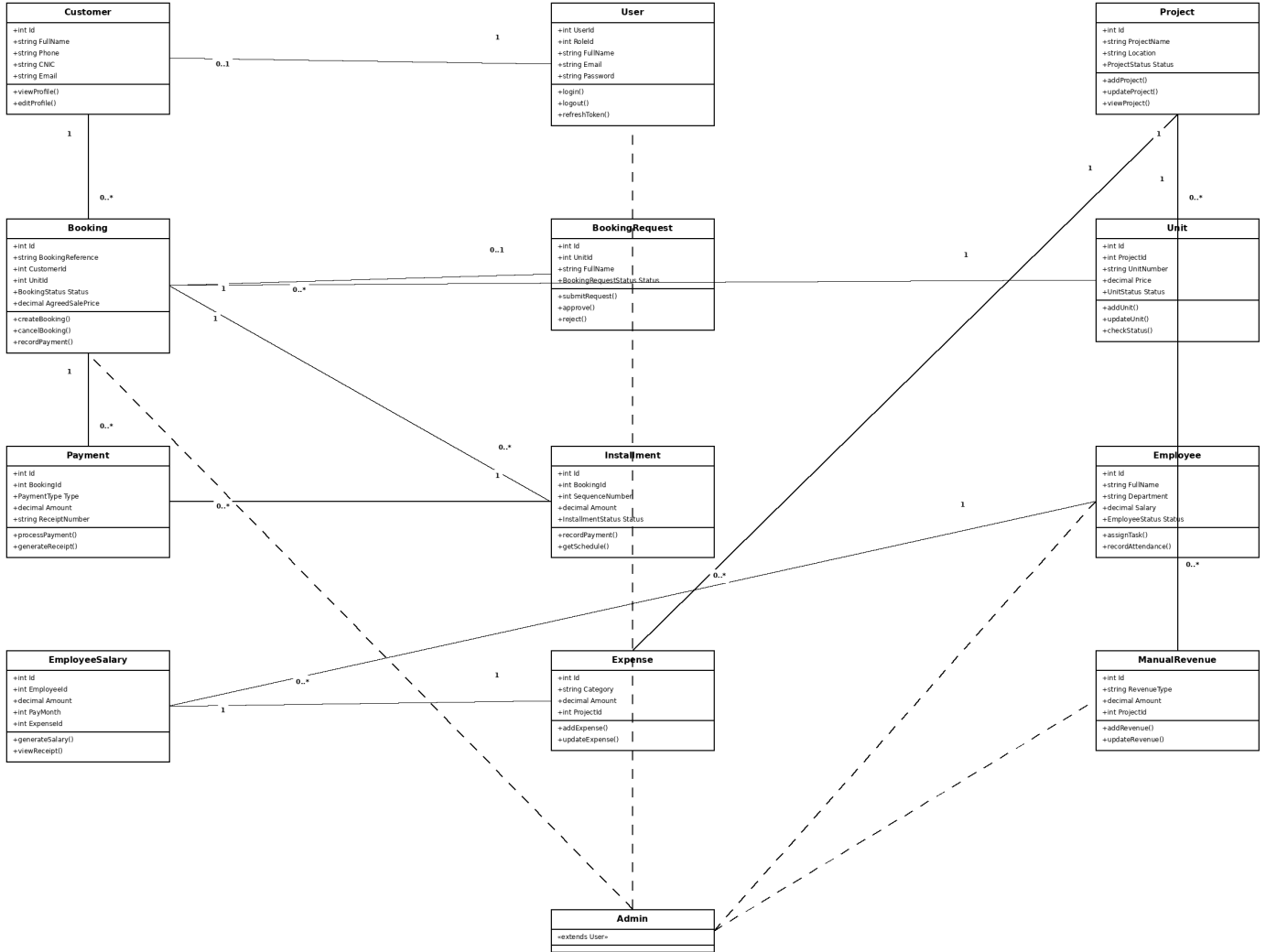
Prepared By

Muhammad Junaid Riaz

27th November, 2025

4.1 Class Diagram

Class Diagram



4.2 Project Code

Project Code

4.2.1 AuthController:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using DAMS.Application.DTOs.Auth;
using DAMS.Application.Interfaces;
using System.Security.Claims;

namespace DAMS.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly IAuthService _authService;

        public AuthController(IAuthService authService)
        {
            _authService = authService;
        }

        [HttpPost("register")]
        public async Task<IActionResult> Register(RegisterRequestDto request)
        {
            await _authService.RegisterAsync(request);
            return Ok(new { message = "Registration completed successfully." });
        }

        [HttpPost("login")]
        public IActionResult Login(LoginRequestDto request)
        {
            var tokens = _authService.Login(request);
            if (tokens == null)
                return Unauthorized("Invalid credentials");
            return Ok(tokens);
        }

        [Authorize]
        [HttpGet("profile")]
        public IActionResult Profile()
        {
            return Ok(new
            {
                userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value,
                email = User.FindFirst(ClaimTypes.Email)?.Value,
                role = User.FindFirst(ClaimTypes.Role)?.Value
            });
        }
    }
}
```

4.2.2 ProjectController:

```
using DAMS.Application.DTOS.ProjectDtos;
using DAMS.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProjectController : ControllerBase
    {
        private readonly IProjectService _projectService;

        public ProjectController(IProjectService projectService)
        {
            _projectService = projectService;
        }

        [Authorize(Roles = "Admin")]
        [HttpPost]
        public async Task<IActionResult> CreateProject(CreateProjectDto dto)
        {
            var adminId =
int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
            var result = await _projectService.CreateProjectAsync(dto,
adminId);
            return Ok(result);
        }

        [Authorize(Roles = "Admin")]
        [HttpPut("{id}")]
        public async Task<IActionResult> UpdateProject(int id, UpdateProjectDto
dto)
        {
            var result = await _projectService.UpdateProjectAsync(id, dto);
            return Ok(result);
        }

        [AllowAnonymous]
        [HttpGet]
        public async Task<IActionResult> GetAllProjects()
        {
            var result = await _projectService.GetAllProjectsAsync();
            return Ok(result);
        }

        [AllowAnonymous]
        [HttpGet("{id}")]
        public async Task<IActionResult> GetProjectById(int id)
        {

```

```

        var result = await _projectService.GetProjectByIdAsync(id);
        if (result == null) return NotFound();
        return Ok(result);
    }
}
}

```

4.2.3 UnitController:

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using DAMS.Application.Interfaces;
using DAMS.Application.DTOS.UnitDtos;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UnitController : ControllerBase
    {
        private readonly IUnitService _unitService;

        public UnitController(IUnitService unitService)
        {
            _unitService = unitService;
        }

        [Authorize(Roles = "Admin")]
        [HttpPost]
        public async Task<IActionResult> Create(CreateUnitDto dto)
        {
            var result = await _unitService.CreateUnitAsync(dto);
            return Ok(result);
        }

        [AllowAnonymous]
        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            var result = await _unitService.GetUnitByIdAsync(id);
            if (result == null) return NotFound();
            return Ok(result);
        }

        [Authorize(Roles = "Admin")]
        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, UpdateUnitDto dto)
        {
            var result = await _unitService.UpdateUnitAsync(id, dto);
            return Ok(result);
        }

        [Authorize(Roles = "Admin")]
    }
}

```

```

        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
        {
            await _unitService.DeleteUnitAsync(id);
            return Ok("Unit deleted successfully");
        }
    }
}

```

4.2.4 CustomerController:

```

using DAMS.Application.DTOS.CustomerDtos;
using DAMS.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize(Roles = "Admin")]
    public class CustomerController : ControllerBase
    {
        private readonly ICustomerService _customerService;

        public CustomerController(ICustomerService customerService)
        {
            _customerService = customerService;
        }

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] CreateCustomerDto
dto)
        {
            var result = await _customerService.CreateCustomerAsync(dto,
GetUserId());
            return Ok(result);
        }

        [HttpGet]
        public async Task<IActionResult> GetAll([FromQuery] string? search)
        {
            var filter = new CustomerFilterDto { SearchTerm = search };
            var result = await _customerService.GetCustomersAsync(filter);
            return Ok(result);
        }

        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            var result = await _customerService.GetCustomerByIdAsync(id);
            if (result == null) return NotFound();
        }
    }
}

```

```

        return Ok(result);
    }

    [HttpPut("{id:int}")]
    public async Task<IActionResult> Update(int id, [FromBody]
UpdateCustomerDto dto)
    {
        var result = await _customerService.UpdateCustomerAsync(id, dto);
        return Ok(result);
    }

    private int GetUserId() =>
int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
    }
}

```

4.2.5 BookingRequestController:

```

using DAMS.Application.DTOs.BookingRequestDtos;
using DAMS.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class BookingRequestController : ControllerBase
    {
        private readonly IBookingRequestService _bookingRequestService;

        [HttpPost]
        public async Task<IActionResult> CreateBookingRequest([FromBody]
CreateBookingRequestDto dto)
        {
            var result = await
_bookingRequestService.CreateBookingRequestAsync(dto, null);
            return Ok(result);
        }

        [HttpGet]
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> GetBookingRequests()
        {
            var result = await
_bookingRequestService.GetBookingRequestsAsync(new BookingRequestFilterDto());
            return Ok(result);
        }

        [HttpPost("{id:int}/approve")]
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> ApproveBookingRequest(int id)

```

```

        {
            var adminUserId =
int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
            var result = await
_bookingRequestService.ApproveBookingRequestAsync(id, adminUserId);
            return Ok(result);
        }

        [HttpPost("{id:int}/reject")]
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> RejectBookingRequest(int id,
[FromBody] RejectBookingRequestDto? dto)
        {
            var adminUserId =
int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
            var result = await
_bookingRequestService.RejectBookingRequestAsync(id, adminUserId,
dto?.RejectionReason);
            return Ok(result);
        }
    }
}

```

4.2.6 BookingController:

```

using DAMS.Application.DTOs.BookingDtos;
using DAMS.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize(Roles = "Admin")]
    public class BookingController : ControllerBase
    {
        private readonly IBookingService _bookingService;

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] CreateBookingDto
dto)
        {
            var result = await _bookingService.CreateBookingAsync(dto,
GetUserId());
            return Ok(result);
        }

        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            var result = await _bookingService.GetBookingByIdAsync(id);

```

```

        if (result == null) return NotFound();
        return Ok(result);
    }

    [HttpPost("{id:int}/cancel")]
    public async Task<IActionResult> Cancel(int id, [FromBody]
CancelBookingDto? dto)
    {
        var result = await _bookingService.CancelBookingAsync(id,
dto?.Reason, GetUserId());
        return Ok(result);
    }

    [HttpPost("{id:int}/booking-amount-payment")]
    public async Task<IActionResult> RecordBookingAmountPayment(int id,
[FromBody] RecordBookingAmountPaymentDto dto)
    {
        var result = await
_bookingService.RecordBookingAmountPaymentAsync(id, dto, GetUserId());
        return Ok(result);
    }

    private int GetUserId() =>
int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));
    }
}

```

4.2.7 EmployeeController:

```

using DAMS.Application.DTOS.EmployeeDtos;
using DAMS.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize(Roles = "Admin")]
    public class EmployeeController : ControllerBase
    {
        private readonly IEmployeeService _employeeService;

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] CreateEmployeeDto
dto)
        {
            var result = await _employeeService.CreateEmployeeAsync(dto);
            return Ok(result);
        }

        [HttpGet]
        public async Task<IActionResult> GetAll()

```

```

        {
            var result = await _employeeService.GetAllEmployeesAsync(null,
null);
            return Ok(result);
        }

        [HttpPost("{employeeId:int}/attendance")]
        public async Task<IActionResult> RecordAttendance(int employeeId,
[FromBody] RecordAttendanceDto dto)
        {
            var result = await
_employeeService.RecordAttendanceAsync(employeeId, dto);
            return Ok(result);
        }

        [HttpPost("{employeeId:int}/salary")]
        public async Task<IActionResult> GenerateSalary(int employeeId,
[FromBody] GenerateSalaryDto dto)
        {
            var result = await _employeeService.GenerateSalaryAsync(employeeId,
dto, null);
            return Ok(result);
        }
    }
}

```

4.2.8 FinanceController:

```

using DAMS.Application.DTOs.ExpenseDtos;
using DAMS.Application.DTOs.FinanceDtos;
using DAMS.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace DAMS.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [Authorize(Roles = "Admin")]
    public class FinanceController : ControllerBase
    {
        private readonly IFinanceService _financeService;

        [HttpGet("dashboard")]
        public async Task<IActionResult> GetDashboard()
        {
            var result = await _financeService.GetDashboardAsync(null, null,
null);
            return Ok(result);
        }

        [HttpPost("expenses")]

```

```

        public async Task<IActionResult> CreateExpense([FromBody]
CreateExpenseDto dto)
        {
            var result = await _financeService.CreateExpenseAsync(dto,
GetUserId());
            return Ok(result);
        }

        [HttpPost("revenue")]
        public async Task<IActionResult> CreateRevenue([FromBody]
CreateManualRevenueDto dto)
        {
            var result = await _financeService.CreateManualRevenueAsync(dto,
GetUserId());
            return Ok(result);
        }

        private int? GetUserId()
        {
            var claim = User.FindFirstValue(ClaimTypes.NameIdentifier);
            return int.TryParse(claim, out var id) ? id : null;
        }
    }
}

```

4.2.9 AuthService:

```

using DAMS.Application.DTOs.Auth;
using DAMS.Application.Interfaces;
using DAMS.Domain.Entities;
using DAMS.Infrastructure.Data;
using Microsoft.EntityFrameworkCore;

namespace DAMS.Application.Services
{
    public class AuthService : IAuthService
    {
        private readonly AppDbContext _context;
        private readonly ITokenService _tokenService;

        public async Task RegisterAsync(RegisterRequestDto request)
        {
            var normalizedEmail = request.Email.Trim().ToLowerInvariant();
            var existingUser = await _context.Users
                .FirstOrDefaultAsync(u => u.Email.ToLower() ==
normalizedEmail);
            if (existingUser != null)
                throw new Exception("Email already exists");

            var clientRole = await _context.Roles
                .FirstOrDefaultAsync(r => r.Role_name.ToLower() == "client");

            var user = new User

```

```

        {
            FullName = request.FullName,
            Email = normalizedEmail,
            Password = BCrypt.Net.BCrypt.HashPassword(request.Password),
            RoleId = clientRole!.RoleId
        };

        _context.Users.Add(user);
        await _context.SaveChangesAsync();
    }

    public AuthResponseDto? Login(LoginRequestDto request)
    {
        var user = _context.Users.FirstOrDefault(u => u.Email == request.Email);
        if (user == null) return null;

        var isValid = BCrypt.Net.BCrypt.Verify(request.Password, user.Password);
        if (!isValid) return null;

        var role = _context.Roles.First(r => r.RoleId == user.RoleId);
        var accessToken = _tokenService.GenerateAccessToken(user, role.Role_name);
        var refreshToken = _tokenService.GenerateRefreshToken();

        user.RefreshToken = refreshToken;
        user.RefreshTokenExpiresAt = DateTime.UtcNow.AddDays(15);
        _context.SaveChanges();

        return new AuthResponseDto
        {
            AccessToken = accessToken,
            RefreshToken = refreshToken,
            ExpiresInMinutes = 15
        };
    }
}

```

4.2.10 BookingService:

```

using DAMS.Application.DTOS.BookingDtos;
using DAMS.Application.Interfaces;
using DAMS.Domain.Entities;
using DAMS.Domain.Enums;
using DAMS.Infrastructure.Data;
using Microsoft.EntityFrameworkCore;

namespace DAMS.Application.Services
{
    public class BookingService : IBookingService
    {

```

```

        private readonly AppDbContext _context;

        public async Task<BookingResponseDto>
        CreateBookingAsync(CreateBookingDto dto, int adminUserId)
        {
            var unit = await _context.Units.FirstOrDefaultAsync(u => u.Id ==
            dto.UnitId);
            if (unit == null) throw new InvalidOperationException("Unit not
            found.");
            if (unit.Status != UnitStatus.Available)
                throw new InvalidOperationException("This unit is not available
            for booking.");

            var booking = new Booking
            {
                CustomerId = dto.CustomerId!.Value,
                UnitId = unit.Id,
                Status = BookingStatus.AwaitingBookingAmount,
                ListPrice = unit.Price,
                AgreedSalePrice = dto.AgreedSalePrice ?? unit.Price,
                BookingAmountRequired = dto.BookingAmountRequired ?? 0m,
                CreatedByUserId = adminUserId,
                CreatedAt = DateTime.UtcNow
            };

            _context.Bookings.Add(booking);
            unit.Status = UnitStatus.Booked;
            await _context.SaveChangesAsync();
            return await GetResponseAsync(booking.Id);
        }

        public async Task<BookingResponseDto> CancelBookingAsync(int id,
        string? reason, int adminUserId)
        {
            var booking = await _context.Bookings.Include(b => b.Unit)
                .FirstOrDefaultAsync(b => b.Id == id);
            if (booking == null) throw new InvalidOperationException("Booking
            not found.");

            booking.Status = BookingStatus.Cancelled;
            booking.Unit.Status = UnitStatus.Available;
            await _context.SaveChangesAsync();
            return await GetResponseAsync(booking.Id);
        }

        public async Task<BookingResponseDto> RecordBookingAmountPaymentAsync(
            int bookingId, RecordBookingAmountPaymentDto dto, int adminUserId)
        {
            var booking = await _context.Bookings.FirstOrDefaultAsync(b => b.Id
            == bookingId);
            if (booking == null) throw new InvalidOperationException("Booking
            not found.");

            booking.BookingAmountReceived += dto.Amount;
            if (booking.BookingAmountReceived >= booking.BookingAmountRequired)

```

```
        booking.Status = BookingStatus.PaymentPlanActive;

        await _context.SaveChangesAsync();
        return await GetResponseAsync(booking.Id);
    }
}
```

Sample Queries

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

Muhammad Junaaid Riaz

27th November, 2025

Revision History

Version	Description	Author	Date
1.0	This document contains sample database queries for Deen Associate Management System (DAMS).	Muhammad Junaid Riaz	27th November, 2025

Sample Queries

This section presents sample queries used by the Deen Associate Management System (DAMS). The application implements these operations using Entity Framework Core (LINQ) in backend services. The SQL statements below show equivalent database operations against Microsoft SQL Server.

Query Summary Table

Query No.	Query Name	Service Method	API Endpoint
1	User Login Verification	AuthService.Login()	POST /api/Auth/login
2	Check Duplicate Email on Registration	AuthService.RegisterAsync()	POST /api/Auth/register
3	Insert New Client User	AuthService.RegisterAsync()	POST /api/Auth/register
4	Retrieve All Projects	ProjectService.GetAllProjectsAsync()	GET /api/Project
5	Retrieve Project by ID	ProjectService.GetProjectByIdAsync()	GET /api/Project/{id}
6	Retrieve Units by Project	UnitService.GetUnitsByProjectIdAsync()	GET /api/Unit/project/{projectId}
7	Search and Filter Customers	CustomerService.GetCustomersAsync()	GET /api/Customer?search=&source=&status=
8	Find Existing Customer (Deduplication)	CustomerService.FindOrCreateCustomerAsync()	Used internally by BookingService / BookingRequestService
9	Insert Booking Request	BookingRequestService.SubmitBookingRequestAsync()	POST /api/BookingRequest
10	List Booking Requests (Admin)	BookingRequestService.GetBookingRequestsAsync()	GET /api/BookingRequest?status=&projectId=&search=
11	Approve Booking Request	BookingRequestService.ApproveBookingRequestAsync()	POST /api/BookingRequest/{id}/approve
12	Create Walk-in Booking	BookingService.CreateBookingAsync()	POST /api/Booking
13	List Bookings with Search and Pagination	BookingService.GetBookingsAsync()	GET /api/Booking?status=&projectId=&search=
14	Client View Own Bookings (My Projects)	BookingService.GetBookingsByCustomerEmailAsync()	GET /api/MyProjects
15	Record Booking Amount Payment	BookingService.RecordBookingPaymentAsync()	POST /api/Booking/{id}/payments

Query No.	Query Name	Service Method	API Endpoint
16	Retrieve Installment Schedule	InstallmentService.GetScheduleAsync()	GET /api/Booking/{id}/installments
17	Record Installment Payment	InstallmentService.RecordInstallmentPaymentAsync()	POST /api/Booking/{id}/installments/{installmentId}/payment
18	Finance Dashboard — Automatic Revenue	FinanceService.GetDashboardAsync()	GET /api/Finance/dashboard?projectId=&from=&to=
19	Finance Dashboard — Manual Revenue	FinanceService.GetDashboardAsync()	GET /api/Finance/dashboard?projectId=&from=&to=
20	Finance Dashboard — Expenses	FinanceService.GetDashboardAsync()	GET /api/Finance/dashboard?projectId=&from=&to=
21	List Employees with Filters	EmployeeService.GetAllEmployeesAsync()	GET /api/Employee?department=&status=
22	Record Employee Attendance	EmployeeService.RecordAttendanceAsync()	POST /api/Employee/{id}/attendance
23	Generate Employee Salary	EmployeeService.GenerateSalaryAsync()	POST /api/Employee/{employeeId}/salary
24	Cancel Booking and Release Unit	BookingService.CancelBookingAsync()	POST /api/Booking/{id}/cancel
25	Booking Request Statistics	BookingRequestService.GetBookingRequestStatsAsync()	GET /api/BookingRequest/stats

Detailed Sample Queries

Query 1: User Login Verification

Description: Verify registered user credentials during login.

Service / Endpoint: AuthService.Login() | POST /api/Auth/login

Equivalent SQL Query:

```
SELECT u.UserId, u.FullName, u.Email, u.Password, r.Role_name
FROM Users u
INNER JOIN Roles r ON u.RoleId = r.RoleId
WHERE u.Email = @Email;
```

Query 2: Check Duplicate Email on Registration

Description: Prevent duplicate client registration using the same email address.

Service / Endpoint: AuthService.RegisterAsync() | POST /api/Auth/register

Equivalent SQL Query:

```
SELECT UserId
FROM Users
WHERE LOWER(Email) = LOWER(@Email);
```

Query 3: Insert New Client User

Description: Register a new client user account with hashed password.

Service / Endpoint: AuthService.RegisterAsync() | POST /api/Auth/register

Equivalent SQL Query:

```
INSERT INTO Users (FullName, Email, Password, RoleId)
VALUES (@FullName, @Email, @HashedPassword, @ClientRoleId);
```

Query 4: Retrieve All Projects

Description: Fetch all real-estate projects for public listing and admin dashboard.

Service / Endpoint: ProjectService.GetAllProjectsAsync() | GET /api/Project

Equivalent SQL Query:

```
SELECT Id, ProjectName, Location, Description, StartingDate,
ExpectedCompletionDate, Status, CreatedAt
FROM Projects
ORDER BY CreatedAt DESC;
```

Query 5: Retrieve Project by ID

Description: Fetch details of a single project.

Service / Endpoint: ProjectService.GetProjectByIdAsync() | GET /api/Project/{id}

Equivalent SQL Query:

```
SELECT Id, ProjectName, Location, Description, StartingDate,
       ExpectedCompletionDate, Status, CreatedAt
FROM Projects
WHERE Id = @ProjectId;
```

Query 6: Retrieve Units by Project

Description: List all units belonging to a selected project.

Service / Endpoint: UnitService.GetUnitsByProjectIdAsync() | GET /api/Unit/project/{projectId}

Equivalent SQL Query:

```
SELECT Id, ProjectId, UnitNumber, UnitType, FloorNumber, Size, Price, Status
FROM Units
WHERE ProjectId = @ProjectId
ORDER BY FloorNumber, UnitNumber;
```

Query 7: Search and Filter Customers

Description: Admin searches customers by name, phone, CNIC, or email with optional filters.

Service / Endpoint: CustomerService.GetCustomersAsync() | GET /api/Customer?search=&source=&status=

Equivalent SQL Query:

```
SELECT c.*, COUNT(b.Id) AS BookingsCount
FROM Customers c
LEFT JOIN Bookings b ON b.CustomerId = c.Id
WHERE (@Source IS NULL OR c.Source = @Source)
AND (@Status IS NULL OR c.Status = @Status)
AND (
  LOWER(c.FullName) LIKE '%' + LOWER(@SearchTerm) + '%'
OR c.Phone LIKE '%' + @SearchTerm + '%'
OR c.CNIC LIKE '%' + @SearchTerm + '%'
OR LOWER(c.Email) LIKE '%' + LOWER(@SearchTerm) + '%'
)
GROUP BY c.Id
ORDER BY c.CreatedAt DESC
OFFSET @Skip ROWS FETCH NEXT @PageSize ROWS ONLY;
```

Query 8: Find Existing Customer (Deduplication)

Description: Locate an existing customer by CNIC, then phone, then email before creating a new record.

Service / Endpoint: CustomerService.FindOrCreateCustomerAsync() | Used internally by BookingService / BookingRequestService

Equivalent SQL Query:

```
SELECT TOP 1 Id FROM Customers WHERE CNIC = @CNIC;
-- if not found:
SELECT TOP 1 Id FROM Customers WHERE Phone = @Phone;
-- if not found:
SELECT TOP 1 Id FROM Customers WHERE Email = @Email;
```

Query 9: Insert Booking Request

Description: Client submits an online booking request for an available unit.

Service / Endpoint: BookingRequestService.SubmitBookingRequestAsync() | POST /api/BookingRequest

Equivalent SQL Query:

```
INSERT INTO BookingRequests
  (UnitId, FullName, Phone, CNIC, Email, Address, Status, UserId, CreatedAt)
VALUES
  (@UnitId, @FullName, @Phone, @CNIC, @Email, @Address, 'Pending', @UserId, GETUTCDATE());
```

Query 10: List Booking Requests (Admin)

Description: Admin views booking requests with project, unit, and status filters.

Service / Endpoint: BookingRequestService.GetBookingRequestsAsync() | GET
/api/BookingRequest?status=&projectId=&search=

Equivalent SQL Query:

```
SELECT br.*, u.UnitNumber, p.ProjectName
FROM BookingRequests br
INNER JOIN Units u ON br.UnitId = u.Id
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE (@Status IS NULL OR br.Status = @Status)
  AND (@ProjectId IS NULL OR u.ProjectId = @ProjectId)
ORDER BY br.CreatedAt DESC
OFFSET @Skip ROWS FETCH NEXT @PageSize ROWS ONLY;
```

Query 11: Approve Booking Request

Description: Admin approves a pending request, creates customer/booking, and marks unit as booked.

Service / Endpoint: BookingRequestService.ApproveBookingRequestAsync() | POST
/api/BookingRequest/{id}/approve

Equivalent SQL Query:

```
UPDATE BookingRequests
SET Status = 'Approved', ReviewedByUserId = @AdminUserId, ReviewedAt = GETUTCDATE()
WHERE Id = @BookingRequestId AND Status = 'Pending';

INSERT INTO Bookings (CustomerId, UnitId, BookingRequestId, Status, ...)
VALUES (@CustomerId, @UnitId, @BookingRequestId, 'AwaitingBookingAmount', ...);

UPDATE Units SET Status = 'Booked' WHERE Id = @UnitId;
```

Query 12: Create Walk-in Booking

Description: Admin creates a booking directly for an available unit.

Service / Endpoint: BookingService.CreateBookingAsync() | POST /api/Booking

Equivalent SQL Query:

```

SELECT u.*, p.ProjectName
FROM Units u
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE u.Id = @UnitId AND u.Status = 'Available';

INSERT INTO Bookings
  (CustomerId, UnitId, Status, AgreedSalePrice, BookingAmountRequired, CreatedByUserId, CreatedAt)
VALUES
  (@CustomerId, @UnitId, 'AwaitingBookingAmount', @AgreedSalePrice, @BookingAmountRequired,
  @AdminUserId, GETUTCDATE());

UPDATE Units SET Status = 'Booked' WHERE Id = @UnitId;

```

Query 13: List Bookings with Search and Pagination

Description: Admin retrieves bookings filtered by status, project, customer, or search term.

Service / Endpoint: BookingService.GetBookingsAsync() | GET /api/Booking?status=&projectId=&search=

Equivalent SQL Query:

```

SELECT b.*, c.FullName, u.UnitNumber, p.ProjectName
FROM Bookings b
INNER JOIN Customers c ON b.CustomerId = c.Id
INNER JOIN Units u ON b.UnitId = u.Id
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE (@Status IS NULL OR b.Status = @Status)
  AND (@ProjectId IS NULL OR u.ProjectId = @ProjectId)
  AND (
    LOWER(b.BookingReference) LIKE '%' + LOWER(@SearchTerm) + '%'
    OR LOWER(c.FullName) LIKE '%' + LOWER(@SearchTerm) + '%'
    OR LOWER(u.UnitNumber) LIKE '%' + LOWER(@SearchTerm) + '%'
  )
ORDER BY b.BookingDate DESC
OFFSET @Skip ROWS FETCH NEXT @PageSize ROWS ONLY;

```

Query 14: Client View Own Bookings (My Projects)

Description: Logged-in client views bookings linked to the same email as the JWT account.

Service / Endpoint: BookingService.GetBookingsByCustomerEmailAsync() | GET /api/MyProjects

Equivalent SQL Query:

```

SELECT b.*, c.FullName, u.UnitNumber, p.ProjectName
FROM Bookings b
INNER JOIN Customers c ON b.CustomerId = c.Id
INNER JOIN Units u ON b.UnitId = u.Id
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE LOWER(c.Email) = LOWER(@JwtEmail)
  AND b.Status <> 'Cancelled'
ORDER BY b.BookingDate DESC;

```

Query 15: Record Booking Amount Payment

Description: Admin records initial booking amount payment and updates booking totals.

Service / Endpoint: BookingService.RecordBookingPaymentAsync() | POST /api/Booking/{id}/payments

Equivalent SQL Query:

```

INSERT INTO Payments
  (BookingId, Type, Amount, PaymentMethod, Reference, PaidAt, ReceiptNumber, CreatedByUserId)
VALUES
  (@BookingId, 'BookingAmount', @Amount, @PaymentMethod, @Reference, @PaidAt, @ReceiptNumber,
  @AdminUserId);

UPDATE Bookings
SET BookingAmountReceived = BookingAmountReceived + @Amount,
  Status = CASE WHEN BookingAmountReceived + @Amount >= BookingAmountRequired
  THEN 'PaymentPlanActive' ELSE Status END
WHERE Id = @BookingId;

```

Query 16: Retrieve Installment Schedule

Description: Fetch installment plan rows for a booking.

Service / Endpoint: InstallmentService.GetScheduleAsync() | GET /api/Booking/{id}/installments

Equivalent SQL Query:

```

SELECT i.*
FROM Installments i
INNER JOIN Bookings b ON i.BookingId = b.Id
WHERE b.Id = @BookingId
ORDER BY i.SequenceNumber;

```

Query 17: Record Installment Payment

Description: Admin records payment against a specific installment.

Service / Endpoint: InstallmentService.RecordInstallmentPaymentAsync() | POST /api/Booking/{id}/installments/{installmentId}/payment

Equivalent SQL Query:

```

INSERT INTO Payments
  (BookingId, InstallmentId, Type, Amount, PaymentMethod, Reference, PaidAt, ReceiptNumber)
VALUES
  (@BookingId, @InstallmentId, 'Installment', @Amount, @PaymentMethod, @Reference, @PaidAt,
  @ReceiptNumber);

UPDATE Installments
SET Status = CASE WHEN @RemainingAfterPayment <= 0 THEN 'Paid' ELSE 'PartiallyPaid' END
WHERE Id = @InstallmentId;

```

Query 18: Finance Dashboard — Automatic Revenue

Description: Aggregate booking and installment payments as automatic revenue.

Service / Endpoint: FinanceService.GetDashboardAsync() | GET /api/Finance/dashboard?projectId=&from=&to=

Equivalent SQL Query:

```

SELECT p.PaidAt, p.Amount, p.Type, p.ReceiptNumber,
       b.BookingReference, c.FullName, pr.ProjectName
FROM Payments p
INNER JOIN Bookings b ON p.BookingId = b.Id
INNER JOIN Customers c ON b.CustomerId = c.Id
INNER JOIN Units u ON b.UnitId = u.Id
LEFT JOIN Projects pr ON u.ProjectId = pr.Id
WHERE b.Status <> 'Cancelled'
      AND (@ProjectId IS NULL OR u.ProjectId = @ProjectId)
      AND (@FromDate IS NULL OR p.PaidAt >= @FromDate)
      AND (@ToDate IS NULL OR p.PaidAt < @ToDateExclusive);

```

Query 19: Finance Dashboard — Manual Revenue

Description: Retrieve manually entered project revenue records.

Service / **Endpoint:** FinanceService.GetDashboardAsync() | GET
 /api/Finance/dashboard?projectId=&from=&to=

Equivalent SQL Query:

```

SELECT mr.Id, mr.Date, mr.RevenueType, mr.Amount, mr.Reference, mr.Description,
       p.ProjectName
FROM ManualRevenues mr
LEFT JOIN Projects p ON mr.ProjectId = p.Id
WHERE (@ProjectId IS NULL OR mr.ProjectId = @ProjectId)
      AND (@FromDate IS NULL OR mr.Date >= @FromDate)
      AND (@ToDate IS NULL OR mr.Date < @ToDateExclusive);

```

Query 20: Finance Dashboard — Expenses

Description: Retrieve project and general expenses including salary expenses.

Service / **Endpoint:** FinanceService.GetDashboardAsync() | GET
 /api/Finance/dashboard?projectId=&from=&to=

Equivalent SQL Query:

```

SELECT e.Id, e.Date, e.Category, e.Amount, e.Vendor, e.Description,
       p.ProjectName
FROM Expenses e
LEFT JOIN Projects p ON e.ProjectId = p.Id
WHERE (@ProjectId IS NULL OR e.ProjectId = @ProjectId)
      AND (@FromDate IS NULL OR e.Date >= @FromDate)
      AND (@ToDate IS NULL OR e.Date < @ToDateExclusive);

```

Query 21: List Employees with Filters

Description: Admin retrieves employees filtered by department and status.

Service / Endpoint: EmployeeService.GetAllEmployeesAsync() | GET /api/Employee?department=&status=

Equivalent SQL Query:

```
SELECT Id, FullName, Department, Salary, Status, JoinDate
FROM Employees
WHERE (@Department IS NULL OR Department = @Department)
AND (@Status IS NULL OR Status = @Status)
ORDER BY FullName;
```

Query 22: Record Employee Attendance

Description: Mark daily attendance for an employee on a selected date.

Service / Endpoint: EmployeeService.RecordAttendanceAsync() | POST /api/Employee/{id}/attendance

Equivalent SQL Query:

```
SELECT Id FROM EmployeeAttendances
WHERE EmployeeId = @EmployeeId AND [Date] = @Date;

-- if not exists:
INSERT INTO EmployeeAttendances (EmployeeId, Date, Status, Notes, RecordedByUserId)
VALUES (@EmployeeId, @Date, @Status, @Notes, @AdminUserId);
```

Query 23: Generate Employee Salary

Description: Create monthly salary record and linked expense entry.

Service / Endpoint: EmployeeService.GenerateSalaryAsync() | POST /api/Employee/{employeeId}/salary

Equivalent SQL Query:

```
SELECT Id FROM EmployeeSalaries
WHERE EmployeeId = @EmployeeId AND PayMonth = @Month AND PayYear = @Year;

INSERT INTO Expenses (Category, Amount, Vendor, Date, ProjectId, CreatedByUserId)
VALUES ('Salary', @Amount, @EmployeeName, @PayDate, @ProjectId, @AdminUserId);

INSERT INTO EmployeeSalaries (EmployeeId, Amount, PayMonth, PayYear, ExpenseId, PayDate)
VALUES (@EmployeeId, @Amount, @Month, @Year, @ExpenseId, @PayDate);
```

Query 24: Cancel Booking and Release Unit

Description: Admin cancels an active booking and returns the unit to Available status.

Service / Endpoint: BookingService.CancelBookingAsync() | POST /api/Booking/{id}/cancel

Equivalent SQL Query:

```
UPDATE Bookings
SET Status = 'Cancelled', UpdatedAt = GETUTCDATE(), InternalNotes = @UpdatedNotes
WHERE Id = @BookingId AND Status NOT IN ('Cancelled', 'PossessionGiven', 'SaleCompleted');

UPDATE Units
SET Status = 'Available', UpdatedAt = GETUTCDATE()
WHERE Id = @UnitId;
```

Query 25: Booking Request Statistics

Description: Count booking requests grouped by status for admin dashboard cards.

Service / **Endpoint:** BookingRequestService.GetBookingRequestStatsAsync() | GET
/api/BookingRequest/stats

Equivalent SQL Query:

```
SELECT Status, COUNT(*) AS RequestCount  
FROM BookingRequests  
GROUP BY Status;
```

Chapter # 5

Testing

Test Case

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

Muhammad Junaid Riaz

25th November, 2025

Test case ID:	TC-01	Test Case Name:	Sign Up
Test Case Description:	This test case describes that a new user can register an account in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User opens the sign up form and enters name, email, phone, password, and role.			
		2- System validates the fields and creates the account successfully.	
3- Steps repeat if another user registers.			
Testing Requirement:			
Testing Condition:	System must be in running state. Email and phone must not already exist.		
Input Data:	Name, email, phone, password, role		
Expected Result:	User account is created successfully.		
Actual Result:	User account is created successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-02	Test Case Name:	Login
Test Case Description:	This test case describes that a registered user can log in in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		

Main success scenario:			
User Action		System Response	
1- User enters email and password on the login page.		2- System verifies credentials and redirects the user to the dashboard.	
Testing Requirement:			
Testing Condition:		System must be in running state. User must have valid login credentials.	
Input Data:		Email and password	
Expected Result:		User is successfully logged into the system.	
Actual Result:		User is successfully logged into the system.	
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-03	Test Case Name:	Logout
Test Case Description:	This test case describes that a logged-in user can log out in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User clicks the Logout button.			
		2- System ends the session and redirects the user to the home	

		page.	
Testing Requirement:			
Testing Condition:	System must be in running state. User must be logged in.		
Input Data:	Click Logout		
Expected Result:	User is logged out successfully.		
Actual Result:	User is logged out successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-04	Test Case Name:	Change Password
Test Case Description:	This test case describes that a logged-in user can change password in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User enters current password and new password.			
		2- System validates the current password and updates the new password.	
Testing Requirement:			
Testing Condition:	System must be in running state. User must be logged in.		
Input Data:	Current password and new password		

Expected Result:	Password is changed successfully.		
Actual Result:	Password is changed successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-05	Test Case Name:	Forget Password
Test Case Description:	This test case describes that a user can reset forgotten password in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User enters registered email on forget password page.		2- System sends reset instructions and allows password reset.	
Testing Requirement:			
Testing Condition:	System must be in running state. Email must exist in the database.		
Input Data:	Registered email address		
Expected Result:	Password reset process completes successfully.		
Actual Result:	Password reset process completes successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

--	--

Test case ID:	TC-06	Test Case Name:	Add Project
Test Case Description:	This test case describes that an admin can add a new project in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters project name, location, description, and dates.		2- System validates the data and saves the new project.	
Testing Requirement:			
Testing Condition:	System must be in running state. Admin must be logged in.		
Input Data:	Project name, location, description, dates		
Expected Result:	New project is added successfully.		
Actual Result:	New project is added successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-07	Test Case Name:	Update Project
Test Case Description:	This test case describes that an admin can update project details in the Deen Associate Management System (DAMS).		

Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects a project and updates its details.		2- System validates and saves the updated project information.	
Testing Requirement:			
Testing Condition:	System must be in running state. Project must already exist.		
Input Data:	Updated project details		
Expected Result:	Project details are updated successfully.		
Actual Result:	Project details are updated successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-08	Test Case Name:	View Project
Test Case Description:	This test case describes that a user can view project details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User selects a project from the projects list.			
		2- System displays complete project details and related units.	

Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Project selection		
Expected Result:	Project details are displayed correctly.		
Actual Result:	Project details are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-09	Test Case Name:	Search Project
Test Case Description:	This test case describes that an admin can search projects in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters search criteria on the projects page.			
		2- System displays matching projects in a list.	
Testing Requirement:			
Testing Condition:	System must be in running state. Admin must be logged in.		
Input Data:	Project name or location		
Expected Result:	Matching projects are displayed.		

Actual Result:	Matching projects are displayed.		
Priority:	Medium	Frequency:	Less frequent
Test Acceptance:	Passed		

Test case ID:	TC-10	Test Case Name:	Upload Project Media
Test Case Description:	This test case describes that an admin can upload project media in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects image or video files for a project.			
		2- System uploads and displays the media in the project gallery.	
Testing Requirement:			
Testing Condition:	System must be in running state. Valid media format must be selected.		
Input Data:	Project media file		
Expected Result:	Project media is uploaded successfully.		
Actual Result:	Project media is uploaded successfully.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

--	--

Test case ID:	TC-11	Test Case Name:	Add Unit
Test Case Description:	This test case describes that an admin can add a unit to a project in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters unit number, type, floor, size, price, and status.			
		2- System validates and saves the new unit.	
Testing Requirement:			
Testing Condition:	System must be in running state. Project must exist.		
Input Data:	Unit details		
Expected Result:	Unit is added successfully.		
Actual Result:	Unit is added successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-12	Test Case Name:	Update Unit
Test Case Description:	This test case describes that an admin can update unit details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		

Main success scenario:			
User Action		System Response	
1- Admin selects a unit and updates its information.			
		2- System validates and saves the updated unit details.	
Testing Requirement:			
Testing Condition:		System must be in running state. Unit must already exist.	
Input Data:		Updated unit details	
Expected Result:		Unit details are updated successfully.	
Actual Result:		Unit details are updated successfully.	
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-13	Test Case Name:	View Unit Status
Test Case Description:	This test case describes that a user can view unit availability status in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User opens a unit detail page.			
		2- System displays unit status such as Available, Booked, or Sold.	
Testing Requirement:			

Testing Condition:	System must be in running state.		
Input Data:	Unit selection		
Expected Result:	Unit status is displayed correctly.		
Actual Result:	Unit status is displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-14	Test Case Name:	Upload Unit Media
Test Case Description:	This test case describes that an admin can upload unit media in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin uploads images or videos for a selected unit.		2- System stores and displays the unit media.	
Testing Requirement:			
Testing Condition:	System must be in running state. Valid media format must be selected.		
Input Data:	Unit media file		
Expected Result:	Unit media is uploaded successfully.		

Actual Result:	Unit media is uploaded successfully.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-15	Test Case Name:	Add Customer
Test Case Description:	This test case describes that an admin can add a customer in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters customer name, phone, CNIC, email, and address.		2- System validates and saves the customer record.	
Testing Requirement:			
Testing Condition:	System must be in running state. Phone or CNIC must be unique.		
Input Data:	Customer details		
Expected Result:	Customer is added successfully.		
Actual Result:	Customer is added successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-16	Test Case Name:	Update Customer
Test Case Description:	This test case describes that an admin can update customer information in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects a customer and updates the details.		2- System validates and saves the updated customer record.	
Testing Requirement:			
Testing Condition:	System must be in running state. Customer must exist.		
Input Data:	Updated customer details		
Expected Result:	Customer information is updated successfully.		
Actual Result:	Customer information is updated successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-17	Test Case Name:	View Customer
Test Case Description:	This test case describes that an admin can view customer details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			

User Action		System Response	
1- Admin selects a customer from the customer list.			
		2- System displays customer profile and booking history.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Customer selection		
Expected Result:	Customer details are displayed correctly.		
Actual Result:	Customer details are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-18	Test Case Name:	Search Customer
Test Case Description:	This test case describes that an admin can search customers in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters name, phone, or CNIC in search field.		2- System displays matching customer records.	
Testing Requirement:			
Testing Condition:	System must be in running state.		

Input Data:	Search keyword		
Expected Result:	Matching customers are displayed.		
Actual Result:	Matching customers are displayed.		
Priority:	Medium	Frequency:	Less frequent
Test Acceptance:	Passed		

Test case ID:	TC-19	Test Case Name:	Submit Booking Request
Test Case Description:	This test case describes that a client can submit a booking request in the Deen Associate Management System (DAMS).		
Primary Actor:	<Client>		
Main success scenario:			
User Action		System Response	
1- Client selects a unit and fills the booking request form.		2- System saves the request with Pending status.	
Testing Requirement:			
Testing Condition:	System must be in running state. Client must be logged in. Unit must be available.		
Input Data:	Client details and unit selection		
Expected Result:	Booking request is submitted successfully.		
Actual Result:	Booking request is submitted successfully.		

Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-20	Test Case Name:	View Booking Requests
Test Case Description:	This test case describes that an admin can view booking requests in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens the booking requests page.			
		2- System displays all pending and reviewed booking requests.	
Testing Requirement:			
Testing Condition:	System must be in running state. Admin must be logged in.		
Input Data:	Open booking requests page		
Expected Result:	Booking requests are displayed correctly.		
Actual Result:	Booking requests are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-21	Test Case Name:	Approve Booking Request
Test Case	This test case describes that an admin can approve a booking request in the Deen		

Description:	Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin reviews a pending booking request and clicks Approve.			
		2- System updates request status and allows booking creation.	
Testing Requirement:			
Testing Condition:	System must be in running state. Booking request must be pending.		
Input Data:	Booking request ID		
Expected Result:	Booking request is approved successfully.		
Actual Result:	Booking request is approved successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-22	Test Case Name:	Reject Booking Request
Test Case Description:	This test case describes that an admin can reject a booking request in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	

1- Admin reviews a pending booking request and clicks Reject.			
		2- System updates request status to rejected with reason.	
Testing Requirement:			
Testing Condition:	System must be in running state. Booking request must be pending.		
Input Data:	Booking request ID and rejection reason		
Expected Result:	Booking request is rejected successfully.		
Actual Result:	Booking request is rejected successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-23	Test Case Name:	Create Confirmed Booking
Test Case Description:	This test case describes that an admin can create a confirmed booking in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects customer, unit, and enters booking financial details.		2- System creates the booking and generates booking reference.	
Testing Requirement:			
Testing Condition:	System must be in running state. Customer and unit must exist.		

Input Data:	Customer, unit, and booking details		
Expected Result:	Confirmed booking is created successfully.		
Actual Result:	Confirmed booking is created successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-24	Test Case Name:	Update Booking
Test Case Description:	This test case describes that an admin can update booking details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens a booking and updates financial or status details.		2- System validates and saves the updated booking.	
Testing Requirement:			
Testing Condition:	System must be in running state. Booking must exist.		
Input Data:	Updated booking details		
Expected Result:	Booking is updated successfully.		

Actual Result:	Booking is updated successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-25	Test Case Name:	View Booking Details
Test Case Description:	This test case describes that an admin can view booking details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects a booking from the bookings list.			
		2- System displays booking, customer, unit, and payment details.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Booking selection		
Expected Result:	Booking details are displayed correctly.		
Actual Result:	Booking details are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-26	Test Case Name:	Cancel Booking
Test Case Description:	This test case describes that an admin can cancel a booking in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects a booking and chooses Cancel Booking.			
		2- System updates booking status to cancelled.	
Testing Requirement:			
Testing Condition:	System must be in running state. Booking must be active.		
Input Data:	Booking ID and cancellation reason		
Expected Result:	Booking is cancelled successfully.		
Actual Result:	Booking is cancelled successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-27	Test Case Name:	View Booking History
Test Case Description:	This test case describes that a user can view booking history in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			

User Action		System Response	
1- User opens booking history or my projects page.			
		2- System displays past and current booking records.	
Testing Requirement:			
Testing Condition:	System must be in running state. User must be logged in.		
Input Data:	Open booking history		
Expected Result:	Booking history is displayed correctly.		
Actual Result:	Booking history is displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-28	Test Case Name:	Record Booking Amount Payment
Test Case Description:	This test case describes that an admin can record booking amount payment in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens a booking and records booking amount payment.			
		2- System saves payment and updates booking amount received.	
Testing Requirement:			
Testing Condition:	System must be in running state.		

	Booking must exist.		
Input Data:	Payment amount and method		
Expected Result:	Booking amount payment is recorded successfully.		
Actual Result:	Booking amount payment is recorded successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-29	Test Case Name:	Generate Installment Plan
Test Case Description:	This test case describes that an admin can generate installment plan in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters installment frequency, count, and start date.		2- System generates installment schedule for the booking.	
Testing Requirement:			
Testing Condition:	System must be in running state. Booking must be confirmed.		
Input Data:	Installment plan details		
Expected Result:	Installment plan is generated successfully.		

Actual Result:	Installment plan is generated successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-30	Test Case Name:	Record Installment Payment
Test Case Description:	This test case describes that an admin can record installment payment in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects an installment and records payment.		2- System updates installment status and booking payment totals.	
Testing Requirement:			
Testing Condition:	System must be in running state. Installment must be pending.		
Input Data:	Installment ID and payment amount		
Expected Result:	Installment payment is recorded successfully.		
Actual Result:	Installment payment is recorded successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-31	Test Case Name:	View Payment History
Test Case Description:	This test case describes that an admin can view payment history in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens payment history for a booking.			
		2- System displays all booking and installment payments.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Booking selection		
Expected Result:	Payment history is displayed correctly.		
Actual Result:	Payment history is displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-32	Test Case Name:	Generate Payment Receipt
Test Case Description:	This test case describes that an admin can generate payment receipt in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	

1- Admin opens a recorded payment and views receipt.			
		2- System displays printable receipt with receipt number.	
Testing Requirement:			
Testing Condition:	System must be in running state. Payment must exist.		
Input Data:	Payment ID		
Expected Result:	Payment receipt is generated successfully.		
Actual Result:	Payment receipt is generated successfully.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-33	Test Case Name:	Print Application Form
Test Case Description:	This test case describes that an admin can print application form in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens application form for a booking.			
		2- System displays printable official application form.	
Testing Requirement:			
Testing Condition:	System must be in running state. Booking must exist.		

Input Data:	Booking ID		
Expected Result:	Application form is displayed correctly.		
Actual Result:	Application form is displayed correctly.		
Priority:	Medium	Frequency:	Less frequent
Test Acceptance:	Passed		

Test case ID:	TC-34	Test Case Name:	Add Employee
Test Case Description:	This test case describes that an admin can add an employee in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters employee name, job title, department, phone, and salary.			
		2- System validates and saves the employee record.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Employee details		
Expected Result:	Employee is added successfully.		
Actual Result:	Employee is added successfully.		

Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-35	Test Case Name:	Update Employee
Test Case Description:	This test case describes that an admin can update employee details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects an employee and updates the details.		2- System validates and saves updated employee information.	
Testing Requirement:			
Testing Condition:	System must be in running state. Employee must exist.		
Input Data:	Updated employee details		
Expected Result:	Employee details are updated successfully.		
Actual Result:	Employee details are updated successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-36	Test Case Name:	View Employee
Test Case	This test case describes that an admin can view employee details in the Deen		

Description:	Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects an employee from the employee list.		2- System displays employee profile and related records.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Employee selection		
Expected Result:	Employee details are displayed correctly.		
Actual Result:	Employee details are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-37	Test Case Name:	Assign Staff to Project
Test Case Description:	This test case describes that an admin can assign staff to a project in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1-			

Admin assigns an employee to a selected project.			
		2- System saves staff-project assignment successfully.	
Testing Requirement:			
Testing Condition:	System must be in running state. Employee and project must exist.		
Input Data:	Employee ID and project ID		
Expected Result:	Staff is assigned to project successfully.		
Actual Result:	Staff is assigned to project successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-38	Test Case Name:	Record Attendance
Test Case Description:	This test case describes that an admin can record employee attendance in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin records attendance status for an employee.			
		2- System saves attendance record for the selected date.	
Testing Requirement:			
Testing Condition:	System must be in running state. Employee must exist.		
Input Data:	Employee ID, date, attendance status		

Expected Result:	Attendance is recorded successfully.		
Actual Result:	Attendance is recorded successfully.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-39	Test Case Name:	Assign Employee Task
Test Case Description:	This test case describes that an admin can assign a task to an employee in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin creates a task with title, description, and due date.			
		2- System assigns the task to the selected employee.	
Testing Requirement:			
Testing Condition:	System must be in running state. Employee must exist.		
Input Data:	Task details		
Expected Result:	Employee task is assigned successfully.		
Actual Result:	Employee task is assigned successfully.		

Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-40	Test Case Name:	Update Task Status
Test Case Description:	This test case describes that an admin can update employee task status in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens a task and updates its status.			
		2- System saves the updated task status.	
Testing Requirement:			
Testing Condition:	System must be in running state. Task must exist.		
Input Data:	Task ID and new status		
Expected Result:	Task status is updated successfully.		
Actual Result:	Task status is updated successfully.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-41	Test Case Name:	Generate Salary
Test Case Description:	This test case describes that an admin can generate employee salary in the Deen Associate Management System (DAMS).		

Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects employee and salary month/year.		2- System generates salary record for the employee.	
Testing Requirement:			
Testing Condition:	System must be in running state. Employee must exist.		
Input Data:	Employee ID, month, year, amount		
Expected Result:	Salary is generated successfully.		
Actual Result:	Salary is generated successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-42	Test Case Name:	View Salary History
Test Case Description:	This test case describes that an admin can view salary history in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens salary history for an employee.			
		2-	

		System displays past salary records.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Employee selection		
Expected Result:	Salary history is displayed correctly.		
Actual Result:	Salary history is displayed correctly.		
Priority:	Medium	Frequency:	Less frequent
Test Acceptance:	Passed		

Test case ID:	TC-43	Test Case Name:	Add Expense
Test Case Description:	This test case describes that an admin can add an expense in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin enters expense category, amount, vendor, and date.		2- System validates and saves the expense record.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Expense details		
Expected Result:	Expense is added successfully.		

Actual Result:	Expense is added successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-44	Test Case Name:	Update Expense
Test Case Description:	This test case describes that an admin can update an expense in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin selects an expense and updates its details.			
		2- System validates and saves the updated expense.	
Testing Requirement:			
Testing Condition:	System must be in running state. Expense must exist.		
Input Data:	Updated expense details		
Expected Result:	Expense is updated successfully.		
Actual Result:	Expense is updated successfully.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-45	Test Case Name:	View Expense
Test Case Description:	This test case describes that an admin can view expense records in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens the finance or expense list page.			
		2- System displays all expense records.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Open expense list		
Expected Result:	Expense records are displayed correctly.		
Actual Result:	Expense records are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-46	Test Case Name:	Add Manual Income
Test Case Description:	This test case describes that an admin can add manual income in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			

User Action		System Response	
1- Admin enters income type, amount, project, and date.		2- System validates and saves the manual income record.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Manual income details		
Expected Result:	Manual income is added successfully.		
Actual Result:	Manual income is added successfully.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-47	Test Case Name:	View Finance Dashboard
Test Case Description:	This test case describes that an admin can view finance dashboard in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin opens the finance dashboard.			
		2- System displays income, expense, and summary reports.	
Testing Requirement:			
Testing Condition:	System must be in running state. Admin must be logged in.		

Input Data:	Open finance dashboard		
Expected Result:	Finance dashboard is displayed correctly.		
Actual Result:	Finance dashboard is displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-48	Test Case Name:	Browse Projects
Test Case Description:	This test case describes that a client can browse available projects in the Deen Associate Management System (DAMS).		
Primary Actor:	<Client>		
Main success scenario:			
User Action		System Response	
1- Client opens the projects page.			
		2- System displays all available projects.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Open projects page		
Expected Result:	Projects are displayed correctly.		
Actual Result:	Projects are displayed correctly.		
Priority:	Medium	Frequency:	Frequent

Test Acceptance:	Passed
-------------------------	--------

Test case ID:	TC-49	Test Case Name:	View Project Details
Test Case Description:	This test case describes that a client can view project details in the Deen Associate Management System (DAMS).		
Primary Actor:	<Client>		
Main success scenario:			
User Action		System Response	
1- Client selects a project from the list.			
		2- System displays project information and available units.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Project selection		
Expected Result:	Project details are displayed correctly.		
Actual Result:	Project details are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-50	Test Case Name:	View My Projects
Test Case Description:	This test case describes that a client can view my projects page in the Deen Associate Management System (DAMS).		

Primary Actor:	<Client>		
Main success scenario:			
User Action		System Response	
1- Client opens the My Projects page.			
		2- System displays projects and units linked to the client.	
Testing Requirement:			
Testing Condition:	System must be in running state. Client must be logged in.		
Input Data:	Open My Projects page		
Expected Result:	Client projects are displayed correctly.		
Actual Result:	Client projects are displayed correctly.		
Priority:	Medium	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-51	Test Case Name:	Manage User Role
Test Case Description:	This test case describes that a super admin can manage user roles in the Deen Associate Management System (DAMS).		
Primary Actor:	<Super Admin>		
Main success scenario:			
User Action		System Response	
1- Super Admin updates role for a system user.			
		2- System saves the updated role and access permissions.	

Testing Requirement:			
Testing Condition:	System must be in running state. Super Admin must be logged in.		
Input Data:	User ID and role		
Expected Result:	User role is updated successfully.		
Actual Result:	User role is updated successfully.		
Priority:	High	Frequency:	Less frequent
Test Acceptance:	Passed		

Test case ID:	TC-52	Test Case Name:	Invalid Login Attempt
Test Case Description:	This test case describes that the system handles invalid login attempts in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User enters incorrect email or password.			
		2- System displays an invalid credentials error message.	
Testing Requirement:			
Testing Condition:	System must be in running state.		
Input Data:	Invalid email or password		
Expected Result:	Login is rejected and error message is shown.		

Actual Result:	Login is rejected and error message is shown.		
Priority:	High	Frequency:	Frequent
Test Acceptance:	Passed		

Test case ID:	TC-53	Test Case Name:	Duplicate Email Registration
Test Case Description:	This test case describes that the system prevents duplicate email registration in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin> <Client>		
Main success scenario:			
User Action		System Response	
1- User tries to register with an email that already exists.			
		2- System displays duplicate email error and blocks registration.	
Testing Requirement:			
Testing Condition:	System must be in running state. Email must already exist.		
Input Data:	Existing email address		
Expected Result:	Duplicate registration is prevented.		
Actual Result:	Duplicate registration is prevented.		
Priority:	High	Frequency:	Less frequent
Test Acceptance:	Passed		

Test case ID:	TC-54	Test Case Name:	Duplicate Customer Entry
Test Case Description:	This test case describes that the system prevents duplicate customer records in the Deen Associate Management System (DAMS).		
Primary Actor:	<Admin>		
Main success scenario:			
User Action		System Response	
1- Admin tries to add a customer with existing phone or CNIC.			
		2- System displays duplicate entry error and blocks save.	
Testing Requirement:			
Testing Condition:	System must be in running state. Customer phone or CNIC must already exist.		
Input Data:	Duplicate phone or CNIC		
Expected Result:	Duplicate customer entry is prevented.		
Actual Result:	Duplicate customer entry is prevented.		
Priority:	High	Frequency:	Less frequent
Test Acceptance:	Passed		

Chapter # 6

User Manual

User Manual

FOR

Deen Associate Management System

VERSION 1.0

Prepared By

Muhammad Junaaid Riaz

27th November, 2025

User Manual

Deen Associate Management System (DAMS)

1. Introduction

The Deen Associate Management System (DAMS) is a web-based real estate management platform used to publish projects, manage property units, handle customer booking requests, confirm sales, track installment schedules, record payments, manage employees, and monitor company finances. The same application serves two primary audiences: clients (customers) who browse projects and track their own bookings, and administrators who operate the full back-office workflow.

This manual explains how to use DAMS screen by screen. Each section clearly states whether the screen is for clients, administrators, or both. Screens marked Shared are visible to visitors and logged-in users; admin-only screens require an account with the Admin role.

2. User Roles

2.1 Client (Customer)

A client account is created through self-registration. After login, the navigation bar shows Home, Projects, About, Contact, and My Projects. Clients can browse available units, submit booking requests, and—once a booking is confirmed—view their payment progress, installment schedule, application form, and receipts from My Projects.

- Browse projects and units without administrative controls.
- Submit booking requests for available units.
- Track confirmed bookings, payments, and documents in My Projects.
- Cannot access Requests, Bookings (admin list), Customers, Employees, or Finance.

2.2 Administrator

An administrator account has full operational access. After login, the navigation bar expands to include Requests, Bookings, Customers, Employees, and Finance in addition to the public pages. Admins review booking requests, confirm bookings, generate installment plans, record payments, print official forms and receipts, manage staff attendance and salaries, and maintain the finance ledger.

- Review and approve or reject booking requests.
- Create and manage confirmed bookings and installment schedules.
- Record booking and installment payments with printable receipts.
- Manage customers, employees, attendance, salaries, and finance entries.

- Upload and maintain project media and unit inventory.

3. Getting Started

3.1 Opening the Application

Open the application URL provided by your organization (for example, <http://localhost:5173> during development). The home page introduces Deen Associate and provides quick links to Projects and Contact.

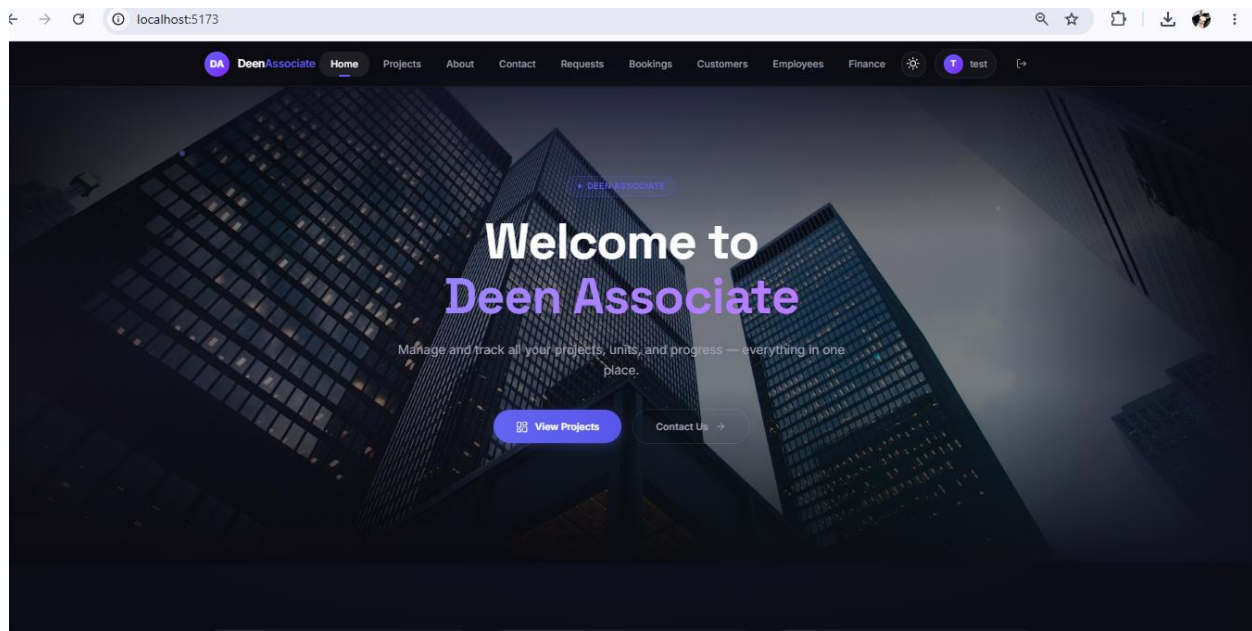
Home Page

Access level: Guest / Client / Admin (public landing page)

The home page welcomes users to Deen Associate and highlights the main entry points. Use View Projects to browse inventory, or Contact Us to reach the sales team. The top navigation remains available on every page.

- Open the application in a modern browser (Chrome, Edge, or Firefox recommended).
- Click View Projects to browse available real estate projects.
- Use Log in or Sign Up in the top-right corner when you need an account.

Figure 1: Home Page



3.2 Registration and Login

New customers create a client account using Sign Up. Existing users sign in with Log in. Administrator accounts are typically created by system staff and are not available through public self-registration.

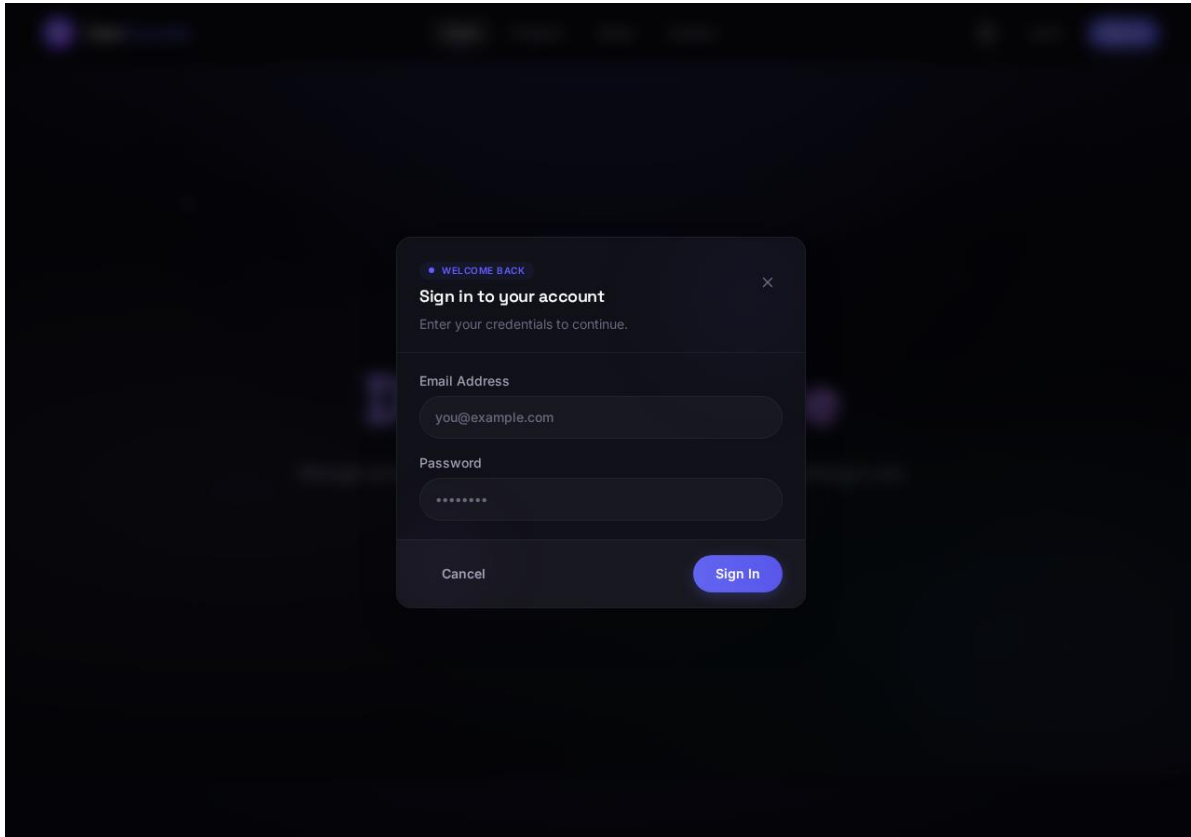
Login Dialog

Access level: Guest → Client or Admin after successful authentication

The login dialog collects email and password. On success, the system stores a secure session token and loads the navigation appropriate to the user's role.

- Click Log in on the navigation bar.
- Enter your registered email address and password.
- Submit the form to access role-based pages.

Figure 2: Login Dialog



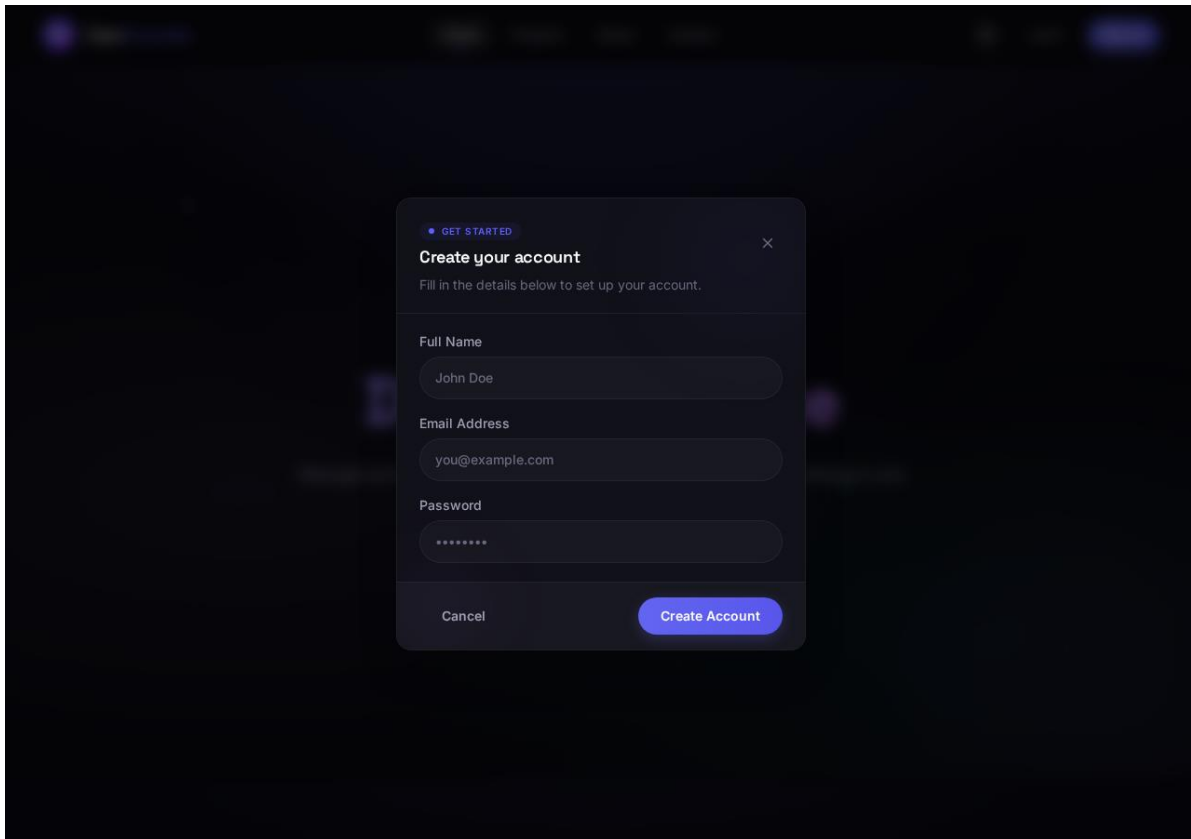
Sign Up Dialog

Access level: Guest → new Client account

The sign-up dialog creates a new customer account. Provide full name, email, and password. After registration, return to Log in with the same credentials.

- Click Sign Up on the navigation bar.
- Enter full name, email, and password.
- Complete registration, then log in with the new account.

Figure 3: Sign Up Dialog



3.3 Theme and Profile Controls

The sun/moon icon in the navigation bar toggles light and dark mode. When logged in, the profile chip shows the account initial and email prefix; the adjacent button logs the user out.

4. Client User Guide

The following screens support the customer journey: discovering projects, reviewing units, submitting booking interest, and tracking confirmed purchases.

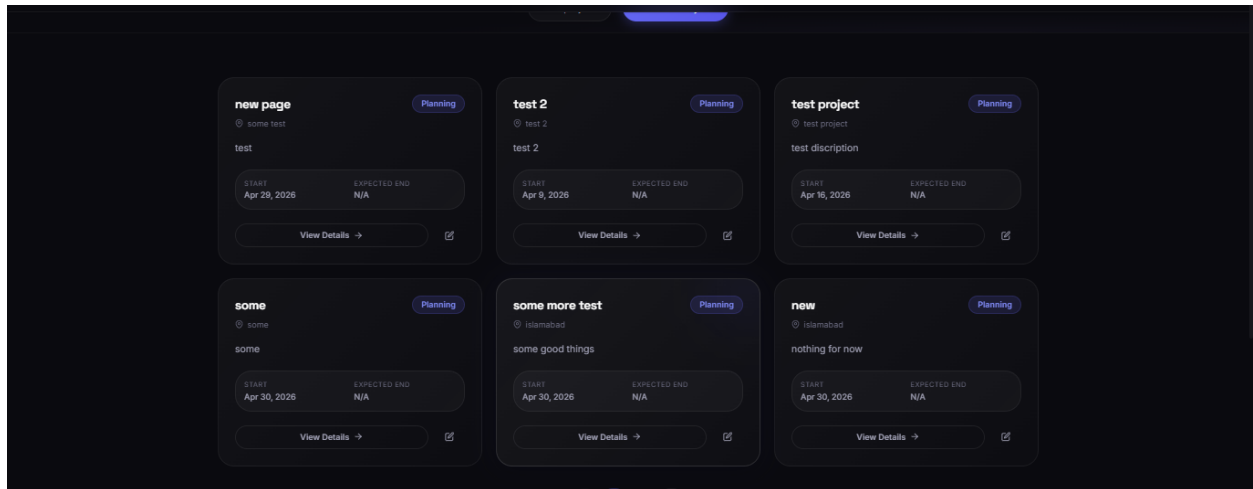
Projects Listing

Access level: Guest / Client / Admin (browse mode)

The Projects page lists all published developments. Each card shows the project name, location, status, and summary information. Select a project to open its detail view.

- Click Projects in the navigation bar.
- Review available developments and their current status.
- Open a project to inspect units and media.

Figure 4: Projects Listing



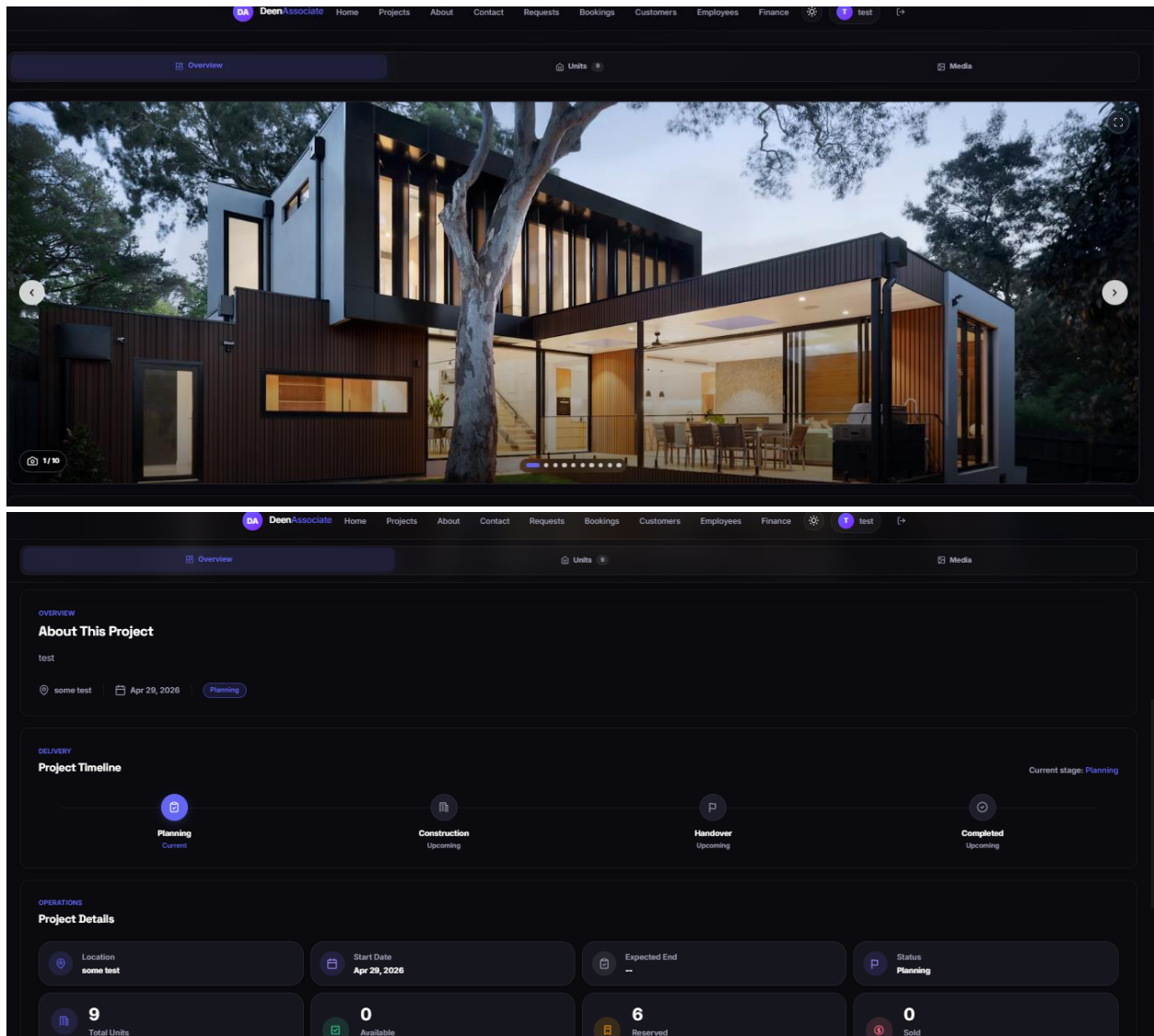
Project Detail — Overview Tab

Access level: Guest / Client / Admin

The Overview tab describes the selected project, including location, launch date, and current delivery stage on the timeline (Planning, Construction, Handover, Completed). This helps clients understand project progress before booking.

- Open a project from the Projects list.
- Read the About This Project section and metadata row.
- Review the Project Timeline to see the current stage.

Figure 5: Project Detail — Overview Tab



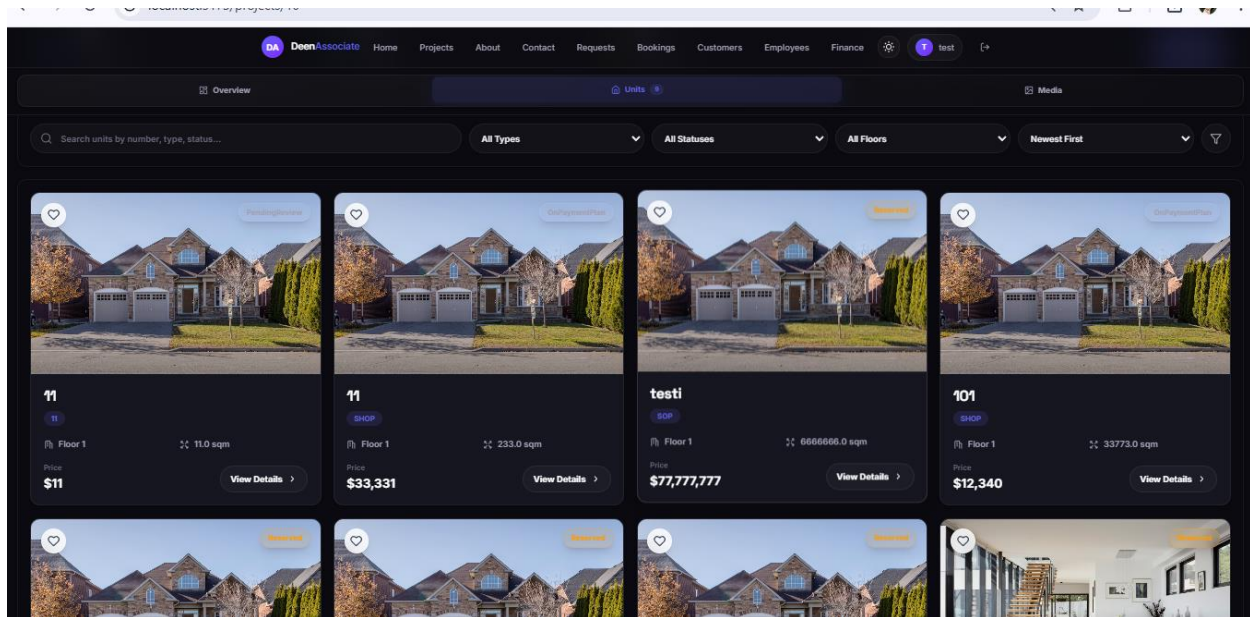
Project Detail — Units Tab

Access level: Guest / Client / Admin

The Units tab displays all units in the project as cards with photos, type, floor, area, and availability status (for example Available, Reserved, OnPaymentPlan). Clients use this screen to choose a unit before submitting a booking request.

- Switch to the Units tab on a project page.
- Use search and filters for type, status, floor, or sort order.
- Open a unit card to view full details and start a booking request.

Figure 6: Project Detail — Units Tab



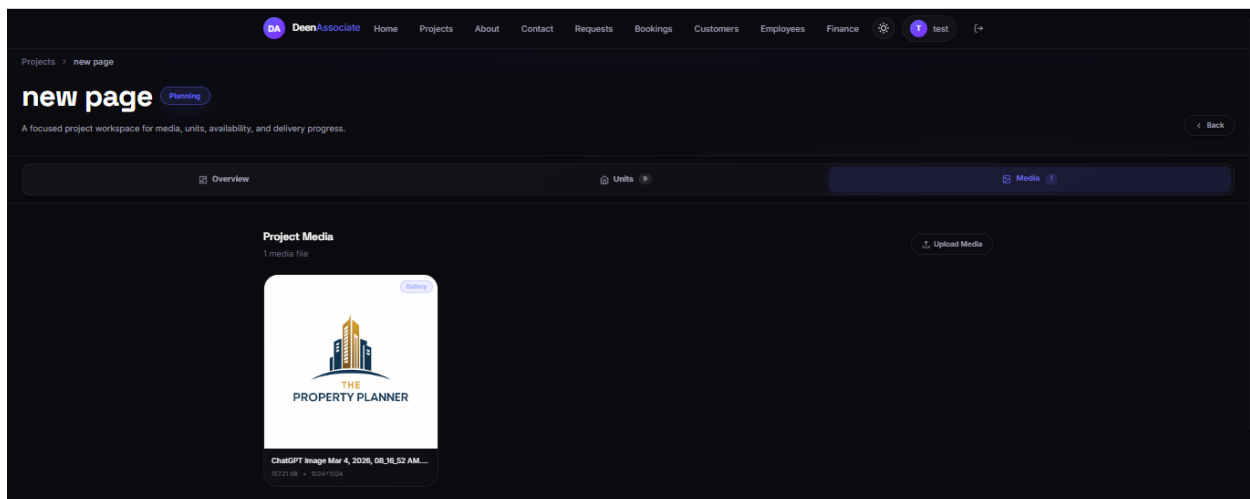
Project Detail — Media Tab

Access level: Guest / Client / Admin

The Media tab shows gallery images and uploaded files for the project. Clients use this section to review marketing visuals, floor plans, or brochures before making a purchase decision.

- Switch to the Media tab.
- Browse uploaded gallery items and file details.
- Use carousel controls where available for full-size viewing.

Figure 7: Project Detail — Media Tab



From a unit detail page, an authenticated client can submit a booking request. The request enters the admin Requests queue for review. Once approved and converted into a confirmed booking, the unit appears under My Projects.

My Projects

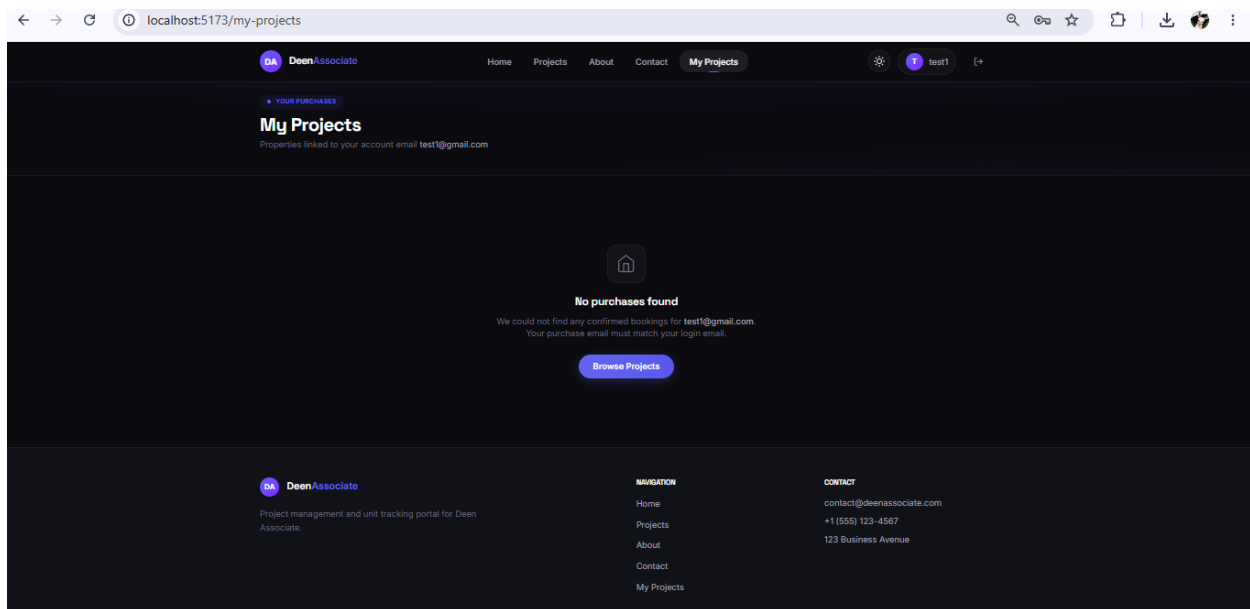
Access level: Client only

My Projects is the client dashboard for confirmed bookings. Each row or card shows booking reference, project, unit, agreed price, booking amount progress, installment totals, and current status (for example PaymentPlanActive).

- Log in with a client account.
- Click My Projects in the navigation bar.
- Select a booking to view payment history, schedule, forms, and receipts.

Administrators are redirected away from My Projects because they manage bookings from the admin Bookings module instead.

Figure 8: My Projects



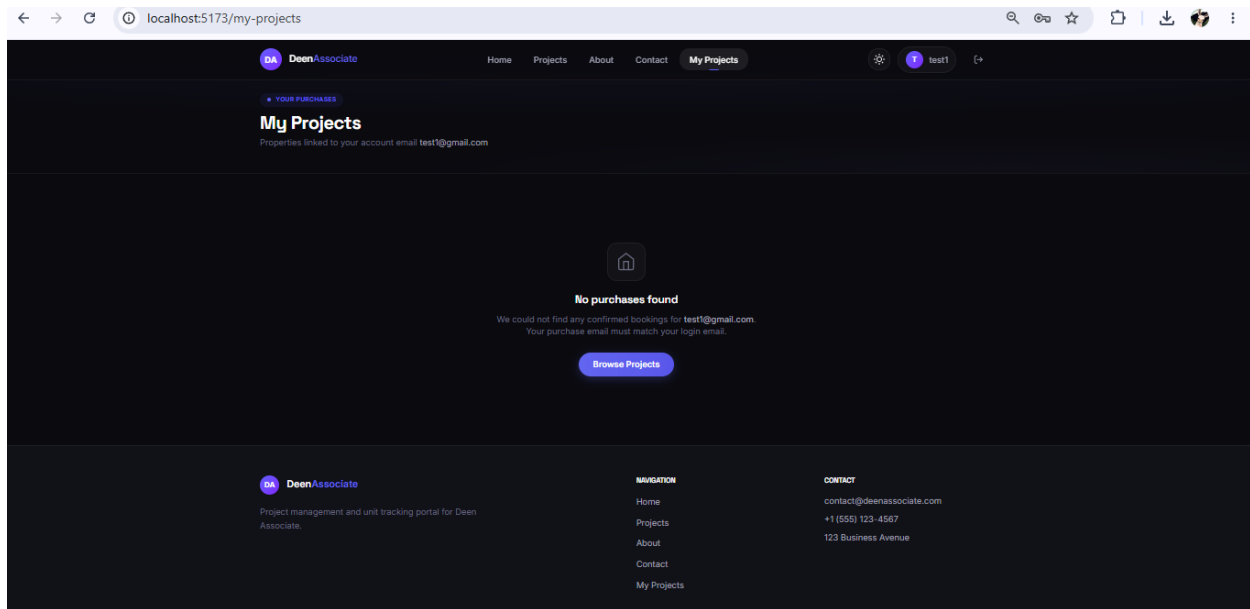
Application Form (Client View)

Access level: Client (own booking) · Admin (any booking)

The application form is the official customer record for a confirmed booking. It includes serial number, date, unit details, applicant information, and next of kin fields. Clients access it from their booking detail; administrators open it via Print Application Form on the admin booking page.

- Open the booking from My Projects (client) or Confirmed Bookings (admin).
- Choose Print Application Form or the equivalent view action.
- Use Print / Save as PDF in the browser for a downloadable copy.

Figure 9: Application Form (Client View)



Payment Receipt (Client View)

Access level: Client (own payments) · Admin (all payments)

Every recorded payment generates a printable receipt with receipt number, amount in words, payment type (Down Payment or Installment), mode of payment, and reference details. Clients can open receipts linked to their bookings; administrators open them after recording a payment.

- Open the booking or installment history.
- Select the receipt for a completed payment.
- Print or save the receipt as PDF for your records.

Figure 10: Payment Receipt (Client View)

The screenshot shows a web application interface for 'DeenAssociate'. The header includes a navigation menu with 'Home', 'Projects', 'About', 'Contact', 'Requests', 'Bookings', 'Customers', 'Employees', 'Finance', and 'My Projects'. The 'My Projects' section is active, displaying a receipt for 'FLORIA HEIGHTS' project by 'Seven Ventures'. The receipt includes fields for Receipt No., Date, Apartment Type, Apartment Number, Floor, Size, Block, Received From, Amount Received, Mode of Payment, and Payment Reference No. The receipt is titled 'PAYMENT RECEIPT' and includes a 'Print / Save as PDF' button.

About and Contact Pages

Access level: Guest / Client / Admin (public information)

The About and Contact pages provide company information and a way to reach Deen Associate. These pages do not require login.

- Use About to read company background and mission content.
- Use Contact to find phone, email, or inquiry options.

Figure 11: About and Contact Pages

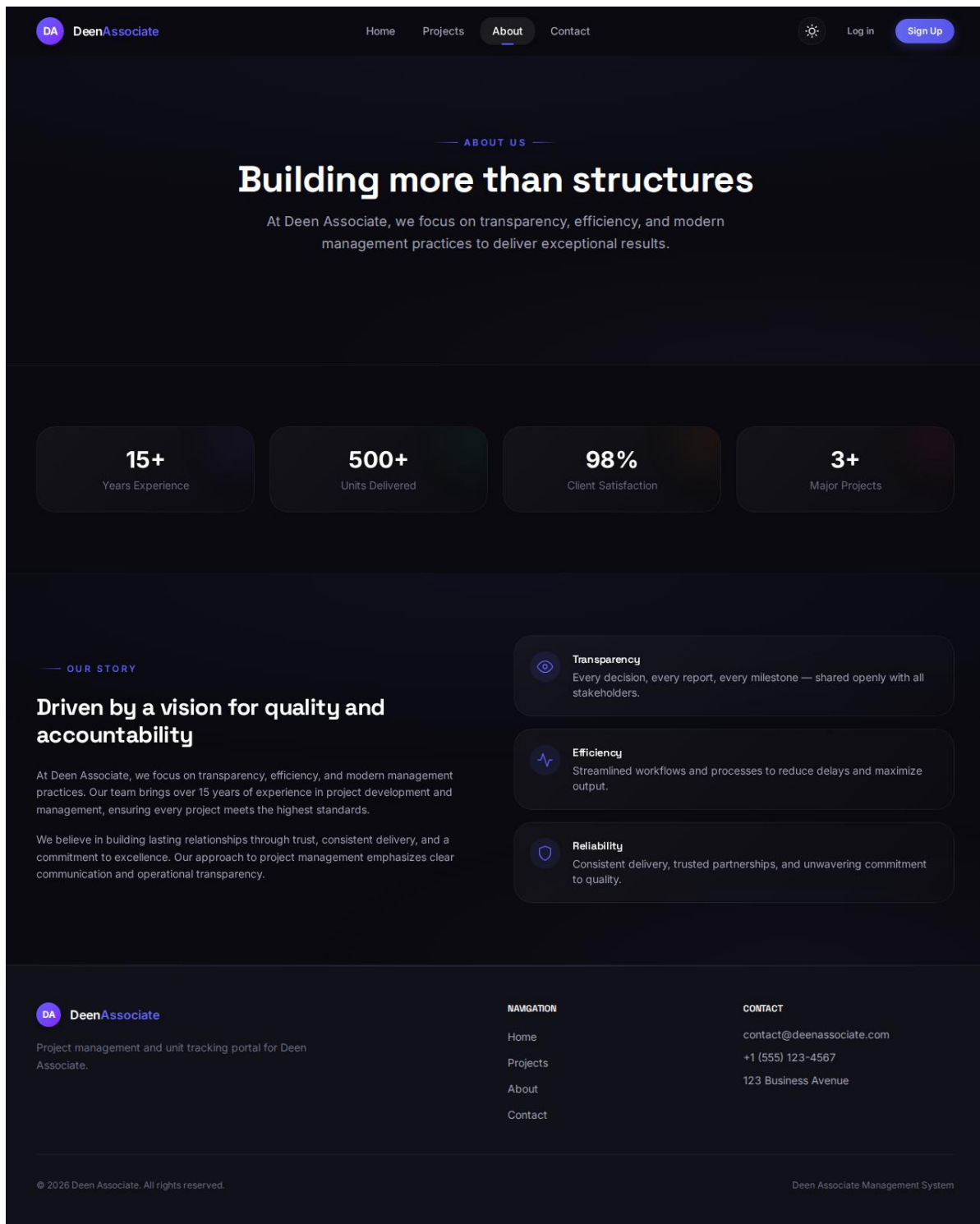
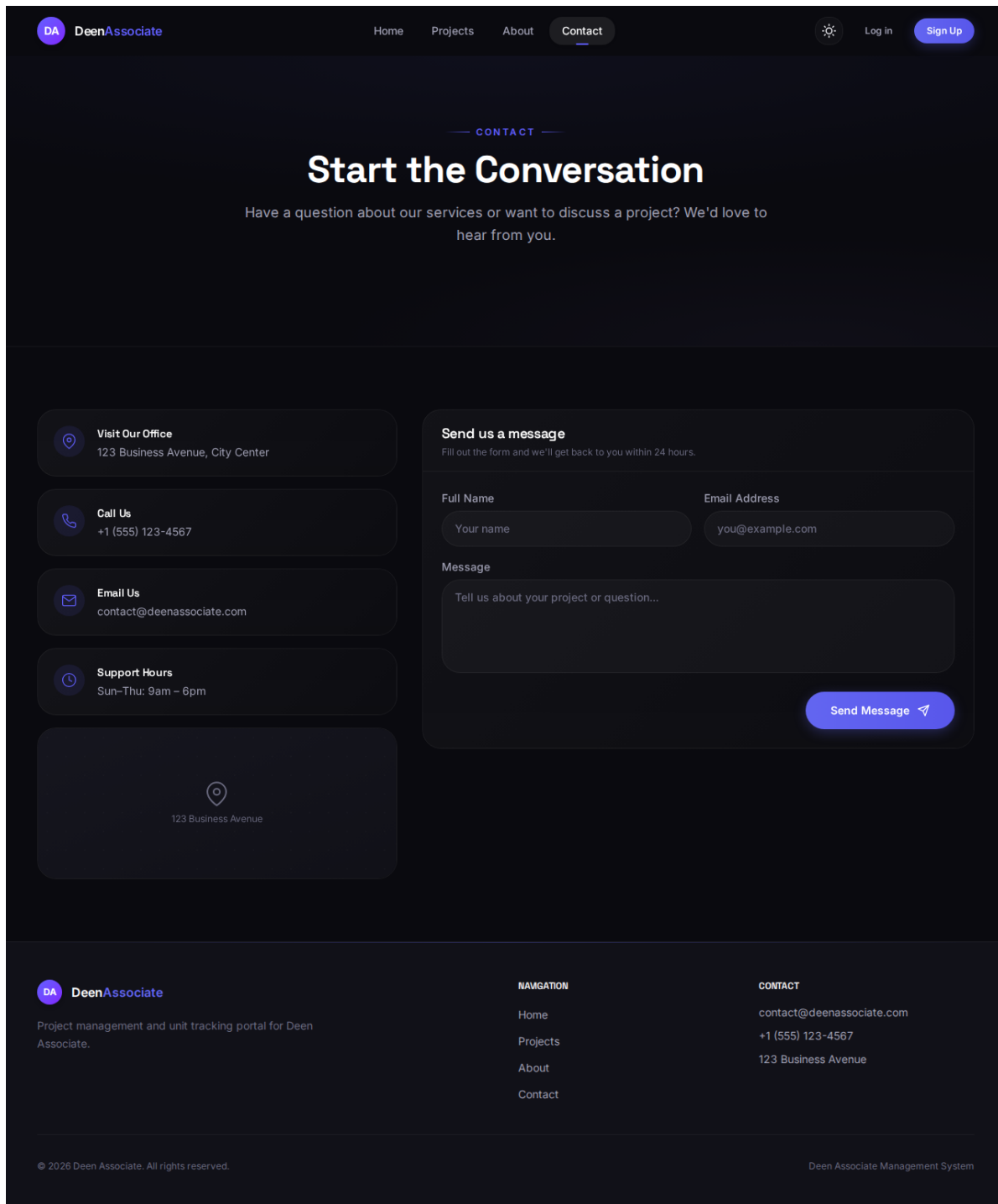


Figure 12: Contact Page



5. Administrator User Guide

Administrators use the expanded navigation bar to run daily operations. The sections below follow the typical workflow: requests → confirmed bookings → payments → documents → staff → finance.

5.1 Booking Requests and Confirmed Bookings

Requests lists incoming booking inquiries submitted by clients. After verification, an admin approves the request and creates a confirmed booking. The Bookings module then tracks the full financial lifecycle of that sale.

Confirmed Booking Detail and Installment Setup

Access level: Admin only

The booking detail page (example: BK-000007) is the control center for a confirmed sale. Summary cards show agreed sale price, booking amount received/remaining, and installment pool. The Regenerate Installment Plan section lets the admin negotiate final terms: sale price, discount, frequency, number of installments, start date, and optional possession amount.

- Open Bookings and select a confirmed booking.
- Review financial summary cards at the top of the page.
- Enter or adjust plan fields in Regenerate Installment Plan.
- Check the preview line (for example, 12 installments × amount).
- Click Regenerate Schedule to rebuild the installment table.
- Use Print Application Form or Cancel Booking when needed.

The Payment Plan Active badge indicates an installment schedule is currently in force for the booking.

Figure 13: Confirmed Booking Detail and Installment Setup

BK-000008

Junaid Riaz · new page · Unit 12

Payment Plan Active

Print Application Form

Cancel Booking

Agreed Sale Price

12,343

Booking Amount Received

1,234.3 / 1,234.3

Booking Amount Remaining

0

Installment Pool (preview)

11,108.7

Installment Schedule

Generated 8/25/2026 · Total 11,108.7 · Paid 925.73 · Remaining 10,182.97

#	Type	Due Date	Amount	Paid	Remaining	Status	Notes
1	Regular	7/25/2026	925.73	925.73	0	Paid	—
2	Regular	8/25/2026	925.73	0	925.73	Pending	— Record Payment
3	Regular	9/25/2026	925.73	0	925.73	Pending	— Record Payment
4	Regular	10/25/2026	925.73	0	925.73	Pending	— Record Payment
5	Regular	11/25/2026	925.73	0	925.73	Pending	— Record Payment
6	Regular	12/25/2026	925.73	0	925.73	Pending	— Record Payment
7	Regular	1/25/2027	925.73	0	925.73	Pending	— Record Payment
8	Regular	2/25/2027	925.73	0	925.73	Pending	— Record Payment
9	Regular	3/25/2027	925.73	0	925.73	Pending	— Record Payment

Record Installment Payment Dialog

Access level: Admin only

The Record Installment Payment dialog captures collection details: amount (defaults to remaining balance), payment method (Cash, bank transfer, etc.), optional reference, payment date, and notes. Saving updates the schedule and creates a receipt in the finance ledger.

- Click Record Payment on an installment row.
- Confirm or edit the amount and payment method.
- Enter reference and payment date if applicable.
- Click Record Payment to post the transaction.
- Open the generated receipt to print for the customer.

Figure 15: Record Installment Payment Dialog

Installment Schedule
Generated 6/25/2026 Total: 10,087.7 Paid: 925.73 Remaining: 9,162.07

#	Type	Due Date	Am	Notes
1	Regular	7/25/2026	\$2	—
2	Regular	8/25/2026	\$2	—
3	Regular	9/25/2026	\$2	—
4	Regular	10/25/2026	\$2	—
5	Regular	11/25/2026	\$2	—
6	Regular	12/25/2026	\$2	—
7	Regular	1/25/2027	\$2	—
8	Regular	2/25/2027	\$2	—
9	Regular	3/25/2027	\$2	—
10	Regular	4/25/2027	\$2	—
11	Regular	5/25/2027	\$25.73	0 \$25.73 Pending

Record Installment Payment
Installment 2 · Due 8/25/2026 · Remaining 925.73

Amount
925.73

Payment Method
Cash

Reference (optional)

Payment Date
06/25/2026

Notes (optional)

Record Payment Cancel

5.2 Customers Module

Customers stores registered client profiles linked to bookings. Administrators use this module to review contact details, view booking history, and maintain customer records. Access it from the Customers link in the admin navigation bar.

- Search customers by name, email, or phone.
- Open a customer to inspect linked bookings and profile fields.
- Use customer records when creating manual bookings or verifying identity.

5.3 Employees Module

Employees is an admin-only HR area with three tabs: Team, Attendance, and Salaries.

Employees — Team Tab

Access level: Admin only

The Team tab lists all employees with position, department, status, assignments, phone, and join date. Use Add Employee to onboard staff and View Details to open a full profile.

- Click Employees in the navigation bar.
- Ensure the Team tab is selected.
- Search or filter by status and department.
- Add or open an employee record as needed.

Figure 16: Employees — Team Tab

EMPLOYEE	POSITION	DEPARTMENT	STATUS	ASSIGNED PROJECT	ASSIGNED TASK	PHONE	JOINED	ACTIONS
ahmad No email	security guard	section	Active	Unassigned	Unassigned	03018809431	Jul 7, 2023	View Details
Junaid Riaz rizajunaid54@gmail.com	test	test	Active	Unassigned	Unassigned	03018809431	Apr 4, 2025	View Details

Employees — Attendance Tab

Access level: Admin only

The Attendance tab records daily presence. Select a date, mark each employee Present or Absent, optionally capture a reason, then Save a row or Save All.

- Switch to the Attendance tab.
- Choose the attendance date.
- Mark Present or Absent for each employee.
- Click Save on a row or Save All to store the sheet.

Figure 17: Employees — Attendance Tab

EMPLOYEE	DEPARTMENT	PRESENT	ABSENT	REASON	
ahmad security guard	section	☐	☐	---	Save
Junaid Riaz test	test	☐	☐	---	Save

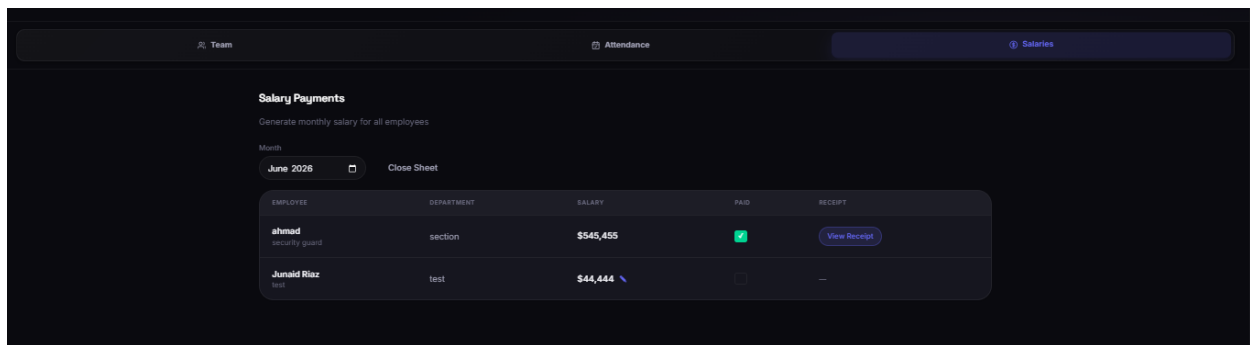
Employees — Salaries Tab

Access level: Admin only

The Salaries tab generates monthly payroll. Select the month, review each employee's salary, mark Paid when disbursed, and View Receipt for processed salary payments. Salary payments also appear in Finance as expense entries.

- Switch to the Salaries tab.
- Choose the payroll month.
- Confirm salary amounts (editable where permitted).
- Mark Paid and open the receipt after payment.

Figure 18: Employees — Salaries Tab



5.4 Finance Module

Finance gives administrators a company-wide view of revenue, expenses, profit, outstanding receivables, and overdue installments. Revenue includes automatic entries from booking and installment payments plus manually entered income.

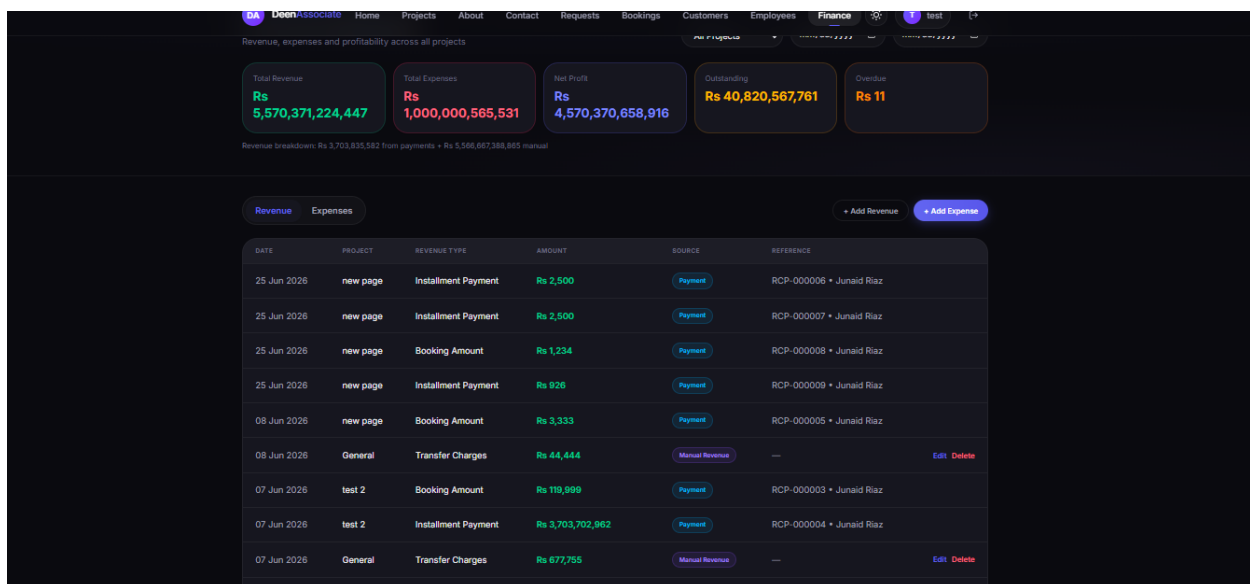
Finance Dashboard — Revenue View

Access level: Admin only

The Revenue view shows summary cards and a detailed table. Automatic Payment rows link to receipts (for example, RCP-000006). Manual Revenue rows can be edited or deleted.

- Click Finance in the navigation bar.
- Review Total Revenue, Expenses, Net Profit, Outstanding, and Overdue cards.
- Keep the Revenue tab selected to audit incoming money.
- Use Add Revenue for manual income not tied to a booking payment.

Figure 19: Finance Dashboard — Revenue View



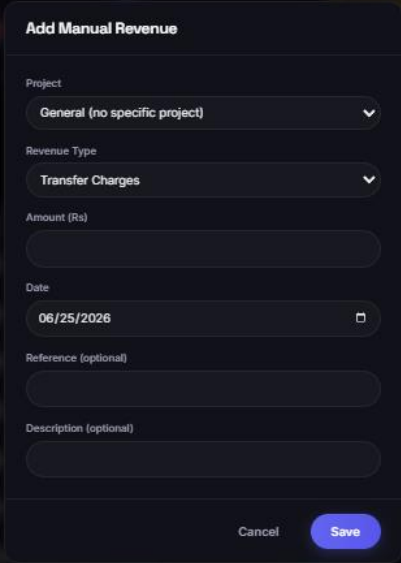
Add Manual Revenue Dialog

Access level: Admin only

Manual revenue records non-booking income such as transfer charges, documentation fees, parking charges, or other project income. Select project (or General), revenue type, amount, date, and optional reference/description.

- On the Finance page, click Add Revenue.
- Choose project and revenue type.
- Enter amount and transaction date.
- Save to post the entry to the revenue ledger.

Figure 20: Add Manual Revenue Dialog

A screenshot of the 'Add Manual Revenue' dialog box. The dialog has a title bar 'Add Manual Revenue'. It contains several input fields: 'Project' with a dropdown menu showing 'General (no specific project)', 'Revenue Type' with a dropdown menu showing 'Transfer Charges', 'Amount (Rs)' with a text input field, 'Date' with a date picker showing '06/25/2026', 'Reference (optional)' with a text input field, and 'Description (optional)' with a text input field. At the bottom right, there are two buttons: 'Cancel' and 'Save'.

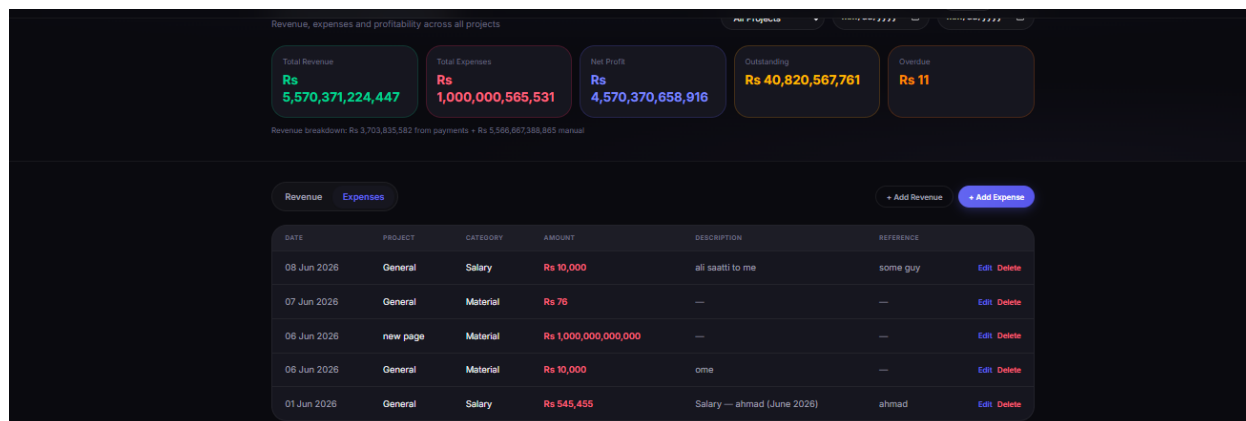
Finance Dashboard — Expenses View

Access level: Admin only

The Expenses view lists operational costs such as salaries and materials. Each row shows date, project, category, amount, description, and reference. Admins can Add Expense or edit/delete manual expense rows.

- Switch the Finance toggle to Expenses.
- Review categories and amounts.
- Click Add Expense for new costs.
- Use Edit or Delete on manual expense entries when corrections are required.

Figure 21: Finance Dashboard — Expenses View



5.5 Project and Media Management

Administrators maintain project content through the same project detail pages clients use to browse, but with additional create/edit/upload capabilities available from admin workflows (project setup, unit inventory, and media uploads). The Overview, Units, and Media tabs shown in Section 4 also apply to admin maintenance—admins ensure descriptions, statuses, and gallery assets remain accurate for client-facing pages.

6. Quick Reference — Who Can Access What

Screen / Module	Guest	Client	Admin
Home, About, Contact	Yes	Yes	Yes
Projects & unit browsing	Yes	Yes	Yes
Sign Up / Log in	Yes	Yes	Yes
Submit booking request	No	Yes	Yes
My Projects	No	Yes	No
Requests	No	No	Yes
Bookings (admin list & detail)	No	No	Yes
Record installment payment	No	No	Yes
Customers	No	No	Yes
Employees	No	No	Yes
Finance	No	No	Yes
Application form & receipt (own booking)	No	Yes	Yes (all)

7. Tips and Troubleshooting

- If pages appear empty after login, confirm the backend API is running and the browser has network access.
- Clients who cannot see My Projects should verify the booking is confirmed and linked to their account.
- Installment totals must be regenerated after changing agreed price or discount on a booking.

- Use Print / Save as PDF for application forms and receipts rather than screenshots for official records.
- Finance outstanding and overdue cards update when installment payments are recorded against bookings.
- Log out from the profile menu when using a shared computer.